

Структуры данных и алгоритмы

Ваше Имя

3.3. 3.3. Summary	49
4. Section 4. DRAKON - Visual Algorithmic Language	50
4.1. 4.1. Algorithm Representation Visualization	50
4.2. 4.2. Interface of DrakonTech.	52
4.3. 4.3. Creating a Drakon-Diagram.	54
4.4. 4.4. Basic Structures of Drakon	

Algorithms	57
4.5. 4.5. From Drakon-Diagram to Program	
Code	60
4.6. 4.6. Execution of the Generated Code....	62
4.7. 4.7. Summary	62
5. Section 5. Complexity and Stability Analysis	

1. Welcome to the Data Structures and Algorithms

5.1. 5.1. Requirements for Algorithms	64
5.2. 5.2. Theoretical Analysis of Algorithm	

All the things we know and understand are made up of something called data. Data represents relationships related to objects and processes within a specific domain. To leverage data for solving particular problems, it's essential to organize it and establish its meaning in the context of those challenges. Structuring and processing data lay the groundwork for deriving insights, uncovering patterns, and generating new knowledge.

To structure this data in order to obtain new knowledge or new meanings, it is necessary to process them according to a certain plan of action. In a stricter definition, a data structure is a form of representation of the properties and relationships of a subject area, focused on the expression of a description of data by means of formal languages. Technologically, a data structure is a container in which information is arranged in a certain way that establishes a way to place data in computer memory for quick and efficient access to them.

In turn, algorithms are sets of step-by-step instructions for solving computational problems by processing data structures. Data structures and the processes of their processing according to various algorithms are inextricably linked with each other. In some problems, the choice of algorithm is dictated by the data structure, in others, the data structure determines the choice of algorithm. One of the founders of the formation of computer science as an academic science, N. Wirth, formulated this connection in the title of the book: "Algorithms + data structures = program" [Wirth].

The variety of physical, abstract, and virtual worlds and the processes taking place in them predetermine the emergence of many problem situations, the effective solution of which involves the choice of data structures and algorithms for their processing. A solution is considered efficient if it solves the problem within known resource constraints. First of all, this refers to the total space available for storing data, as well as the time allotted for the execution of each algorithm. When choosing a data structure for solving a problem, one should first determine the basic operations that must be supported, secondly, to quantify the resource constraints for each operation of the algorithm (occupied memory and execution time of operations).

A data structure requires a certain amount of memory for each stored data element, a certain amount of time to perform one basic operation, and a certain amount of programming effort. Each task has limitations on available space and time. Each solution to a problem uses the underlying operations in some relative proportion, and the data structure selection process must take this into account. Only after a thorough analysis of the characteristics of the problem being solved, it is possible to determine the best data structure for it.

Based on the considerations presented, the Handbook provides basic information about the most commonly used data structures and algorithms, since they form the basis of the professional activity of a programmer of any level. Considerable attention is paid to the issues of evaluating the efficiency of data structures and algorithms in terms of occupied memory and the execution time of basic operations. The organization of data structures and the study of basic algorithms is carried out in the JavaScript programming language and is accompanied by explanatory illustrations. In a number of important cases, the algorithm visualization technology based on the Drakon graphical language is used.

Particular situation is given to the engineering judgment involved in choosing data structures and algorithms. Readers are encouraged to consider the operations required by a task, their relative frequency, the available computing resources, and the constraints of the execution environment.	68
5.5.5.3. Algorithm Stability	69
5.6. Summary	69
This perspective connects theoretical complexity analysis with practical questions: how data is organized in memory, how memory access affects performance, and which trade-offs are acceptable for a particular application.	70
6.1.6.1. Linear Data Searching	71

6.2. 6.2. Binary Search for Data in a Sorted

The aim is to help readers develop the ability to justify their choices and recognize the conditions under which those choices should be reconsidered.

SECTION 1 reveals the meaning of the main concepts in the discipline Data Structures and Algorithms: data, data type, abstract data types, algorithms. A high level of competence in the development of efficient algorithms is necessary for every programmer, not only when solving practical problems related to the use of IT technologies. Knowledge in the field of data structure and algorithms is the most important component in an interview when applying for a job in a large IT company.

SECTION 2 describes the basics of the JavaScript programming language in sufficient amount for acquaintance and practical mastery of knowledge and skills in the field of data structure and algorithms. Of course, basic concepts and a certain level of programming are assumed for reading this chapter.

SECTION 3 gives a brief description of the visual language DRAKON, which allows for the representation of complex algorithms in the form of a DRAKON-diagram using icons, corresponding to the basic constructions of the JavaScript language. The technology of automated transformation of the DRAKON-diagram into the program code is given in the editor DrakonTech, with subsequent implementation in the shell of Visual Studio Code. Actually, this is a hybrid, two-stage approach: in the first step a DRAKON-diagram is made, the icons of which are filled with constructions of the language JavaScript, on the second step - automatically generated software code, implemented in any software environment

SECTION 4 presents JavaScript language constructs that implement the basic abstract data types described in Section 1: array, slice, list etc. Explanations are given regarding the use of appropriate JavaScript language constructs.

SECTION 5 provides basic insights into the analysis of algorithm complexity and the evaluation and effectiveness of algorithms from the perspective of computer memory (space) and time costs. Theoretical ideas about estimating the complexity of algorithms are decisive when choosing the most effective one.

SECTION 6 deals with basic sorting algorithms. Each algorithm is represented by a corresponding DRAKON-diagram, accompanied by necessary explanations about the processing process and estimation of complexity.

SECTION 7 discusses basic algorithms for searching elements in different data structures (slices, lists). In addition, the section includes a description and DRAKON-diagram of hash algorithms that significantly reduce search time by reducing the number of element value comparisons.

SECTION 8 provides conceptual information about binary trees – nonlinear data structures, including binary search trees and self-balancing trees. This section describes the basic DRAKON-

diagrams of algorithms for inserting new ones, finding nodes with specified values and deleting nodes.

6.3.6.3. Searching and Related Operations
in a Singly Linked List
SECTION 9 provides basic information about the most complex data structures – graphs. Basic concepts of graph theory and their basic types are given. Algorithms of representation of graphs, their traversal, basic sorting algorithms on vertices are considered.

7.1.7.1. General Characteristics of Sorting
SECTION 10 brings together the preceding material through an engineering approach to selecting data structures and algorithms. It presents a five-step selection procedure that connects application requirements and operation profiles with the capabilities and limitations of candidate structures. It then examines how data is represented in computer memory, explaining the practical significance of memory hierarchy, cache behavior, and data locality, with attention to JavaScript. The section helps readers assess the resources available to them and understand why theoretical complexity alone is insufficient to predict practical performance.

Chapter 1. Section 1. Abstract Data Types

Algorithms	83
7.2. 7.2. Bubble Sort	93
7.3. 7.3. Selection Sort	95
7.4. 7.4. Insertion Sort	97
7.5. 7.5. Quick Sort	100
7.6. 7.6. Merge Sort	102

This section introduces the foundational concepts of Abstract Data Types (ADTs) in computer science. It covers the separation of interface from implementation, enabling modular software design and algorithmic reasoning. The section provides a detailed classification and overview of both linear ADTs (arrays, sets, linked lists, stacks, queues) and non-linear ADTs (associative arrays, trees, graphs), detailing their defining characteristics, core operations, primary advantages, limitations, and real-world applications.

1.1. 1.1. ADT Concept

In computer science, **data** is a representation of facts, values, measurements, or symbols that can be stored, transmitted, and processed by a computer system.

A **data type** defines a set of possible values and a set of operations that can be performed on those values. For example, an integer type defines whole numbers and operations such as addition, subtraction, multiplication, and comparison.

An **abstract data type** (ADT) is a mathematical model of a data structure that specifies:

- what kind of values can be stored;
- what operations are allowed;
- what behavior these operations must have.

An ADT describes the logical behavior of a data structure independently of its underlying data representation or physical memory layout.

For example, a stack is defined by the operations **push**, **pop**, and **peek**, and by the rule that the last inserted element is the first one removed. This rule is independent of whether the stack is implemented using an array, a linked list, or another internal representation.

The concept of abstract data types was formalized in the 1970s, while Donald Knuth's **The Art of Computer Programming** played a major role in systematizing data structures and algorithms. ADTs became one of the foundations of modular programming, software design, and algorithm analysis.

The main purpose of ADTs is to separate **interface** from **implementation**. The interface defines what operations are available, while the implementation defines how these operations are actually performed.

This separation allows programmers and engineers to reason about data structures at different levels of abstraction.

1.2. Classification of Abstract Data Types

7.7. 7.7. Shell Sort 105

7.8. 7.8. Heap Sort 108

7.9. 7.9. Comparison of Sorting Algorithms 111

8. Section 8. Binary Tree Algorithms 112

8.1. 8.1. Representation of Binary Trees 112

8.2. 8.2. Binary Search Tree 114

8.3. 8.3. Binary Search Tree 114

8.4. 8.4. Binary Search Tree 114

8.5. 8.5. Binary Search Tree 114

8.6. 8.6. Binary Search Tree 114

8.7. 8.7. Binary Search Tree 114

8.8. 8.8. Binary Search Tree 114

8.9. 8.9. Binary Search Tree 114

8.10. 8.10. Binary Search Tree 114

8.11. 8.11. Binary Search Tree 114

8.12. 8.12. Binary Search Tree 114

8.13. 8.13. Binary Search Tree 114

8.14. 8.14. Binary Search Tree 114

8.15. 8.15. Binary Search Tree 114

8.16. 8.16. Binary Search Tree 114

8.17. 8.17. Binary Search Tree 114

8.18. 8.18. Binary Search Tree 114

8.19. 8.19. Binary Search Tree 114

8.20. 8.20. Binary Search Tree 114

8.21. 8.21. Binary Search Tree 114

8.22. 8.22. Binary Search Tree 114

8.23. 8.23. Binary Search Tree 114

8.24. 8.24. Binary Search Tree 114

8.25. 8.25. Binary Search Tree 114

8.26. 8.26. Binary Search Tree 114

8.27. 8.27. Binary Search Tree 114

8.28. 8.28. Binary Search Tree 114

8.29. 8.29. Binary Search Tree 114

8.30. 8.30. Binary Search Tree 114

8.31. 8.31. Binary Search Tree 114

8.32. 8.32. Binary Search Tree 114

8.33. 8.33. Binary Search Tree 114

8.34. 8.34. Binary Search Tree 114

8.35. 8.35. Binary Search Tree 114

8.36. 8.36. Binary Search Tree 114

8.37. 8.37. Binary Search Tree 114

8.38. 8.38. Binary Search Tree 114

8.39. 8.39. Binary Search Tree 114

8.40. 8.40. Binary Search Tree 114

8.41. 8.41. Binary Search Tree 114

8.42. 8.42. Binary Search Tree 114

8.43. 8.43. Binary Search Tree 114

8.44. 8.44. Binary Search Tree 114

8.45. 8.45. Binary Search Tree 114

8.46. 8.46. Binary Search Tree 114

8.47. 8.47. Binary Search Tree 114

8.48. 8.48. Binary Search Tree 114

8.49. 8.49. Binary Search Tree 114

8.50. 8.50. Binary Search Tree 114

8.51. 8.51. Binary Search Tree 114

8.52. 8.52. Binary Search Tree 114

8.53. 8.53. Binary Search Tree 114

8.54. 8.54. Binary Search Tree 114

1.3. Linear Abstract Data Types

Linear abstract data types organize data as a sequence of elements. The logical order of elements is one of their defining properties.

Linear structures are widely used because many computational tasks naturally involve sequences: lists of records, input streams, execution histories, buffers, tasks, commands, and ordered

8.3.8.3. Self-Balancing Binary Trees (AVL Trees).....	120
1.3.1.1.3.1. Arrays	138
8.4.8.4. Red-Black Trees.....	138
8.5.8.5. Comparison of Tree Structures.....	185
8.6.8.6. Conclusion.....	187

9. Section 9. Graphs and Graph Algorithms ... 190

In its fundamental or static form, an array has a fixed size and stores elements in contiguous memory locations. This allows direct random access to any element using its index.

If the base address of the array is known, the address of an element can be calculated using the formula:

$$\text{address} = \text{base_address} + \text{index} * \text{element_size}$$

This property makes arrays one of the most important and efficient data structures in computer science.

Characteristics of Arrays

- **Indexed access** - Each element can be accessed directly by its numeric index.
- **Contiguous memory model** - Classical arrays store elements in adjacent memory locations.
- **Fixed size in the classical model** - The number of elements is usually defined at creation time.
- **Homogeneous storage in low-level languages** - In languages such as C, arrays usually store elements of the same type.
- **Efficient traversal** - Arrays are well suited for sequential iteration.

Main Operations on Arrays

- `access(index)` - Returns the element stored at the specified index.
- `update(index, value)` - Replaces the element at the specified index.
- `insert(index, value)` - Inserts a new element at a specified position.
- `delete(index)` - Removes an element from a specified position.
- `search(value)` - Finds the position of a value in the array.
- `length()` - Returns the number of elements.

Advantages of Arrays

- **Fast random access** - Any element can be accessed directly by index.
- **Memory locality** - Contiguous storage improves cache efficiency on modern hardware.
- **Simple structure** - Arrays are easy to understand and implement.
- **Efficient iteration** - Sequential traversal is usually fast.

Limitations of Arrays

- **Expensive insertion and deletion in the middle** - Elements may need to be shifted.

- **Fixed size in the classical model** - Resizing may require allocating a new array.
 - **Wasted memory** - If the allocated size is larger than needed, memory may be unused.
 - **Limited flexibility** - Arrays are less flexible than linked structures for frequent structural changes.
- In modern programming languages, arrays are often implemented as dynamic arrays that can grow and shrink at runtime. JavaScript arrays are dynamic and can store values of different types, but the abstract concept of an array remains based on indexed access.

Applications of Arrays

Arrays are used in many areas of computing, including:

- storing ordered collections of values;
- implementing matrices and multidimensional data;
- representing tables and buffers;
- implementing stacks, queues, heaps, and hash tables;
- supporting efficient iteration in algorithms.

1.3.2. Sets

A **set** is an abstract data type that stores a collection of unique values. Its defining characteristic is the uniqueness invariant: a set does not allow duplicate elements.

In the mathematical sense, sets are unordered collections. They do not provide indexed access and do not require positional ordering. However, some programming language implementations preserve insertion order during iteration. For example, JavaScript's **Set** object preserves insertion order.

Although sets are not linear in the same strict sense as arrays or linked lists, they are often discussed alongside collection-based ADTs because they store individual elements and support membership operations.

Characteristics of Sets

- **Unique elements** - Each value can appear only once.
- **Membership-based access** - The primary operation is checking whether an element belongs to the set.
- **Dynamic size** - Sets can grow or shrink as elements are added or removed.
- **No indexed access** - Elements are not normally accessed by position.

Main Operations on Sets

- **contains(element)** - Checks whether an element exists in the set.
- **add(element)** - Checks whether a value exists in the set.
- **remove(element)** - Removes a value from the set.

9.5.9.6) Returns the number of unique elements.	271
10. Section 10) Data Structures Selection in	271
Practice	328
• <code>Intersection(Set)</code> - Returns a set containing only common elements.	
10.1. 10.1. Data Structure Selection	
• <code>difference(set)</code> - Returns a set containing elements from one set that are not in another.	
Roadmap	329
10.2. 10.2. Memory Hierarchy and Its	
Advantages of Sets	

- **Efficient deduplication** - Sets automatically prevent duplicate values.
- **Fast membership testing** - Many implementations provide efficient lookup.
- **Clear semantics** - Sets directly model mathematical collections of unique elements.
- **Useful for algorithmic state tracking** - Sets are frequently used to track visited elements.

Limitations of Sets

- **No positional access** - Sets do not usually support indexing.
- **No key-value association** - Sets store values, not pairs.
- **Implementation-dependent ordering** - Mathematical sets are unordered, even if some implementations preserve insertion order.

Applications of Sets

Sets are used for:

- removing duplicates from data;
- checking membership efficiently;
- representing mathematical sets;
- tracking visited nodes in graph traversal;
- filtering data streams;
- comparing collections.

1.3.3. 1.3.3. Linked Lists

A **linked list** is a linear abstract data type in which elements are stored as nodes. Each node contains data and one or more references to other nodes.

Unlike arrays, linked lists do not require contiguous memory. Elements can be stored in different memory locations and connected through references.

The first node of a linked list is usually called the **head**. The last node usually points to `null`, unless the list is circular.

Types of Linked Lists

There are several common types of linked lists.

- **Singly linked list** - Each node stores data and a reference to the next node.
- **Doubly linked list** - Each node stores data, a reference to the next node, and a reference to the previous node.
- **Circular linked list** - The last node points back to the first node.
- **Circular doubly linked list** - Each node has previous and next references, and the list forms a closed cycle.

Main Operations on Linked Lists

- **insertFirst(value)** - Inserts a new node at the beginning.
- **insertLast(value)** - Inserts a new node at the end.
- **insertAfter(node, value)** - Inserts a new node after a specified node.
- **delete(value)** - Removes a node containing the specified value.
- **find(value)** - Searches for a node containing the specified value.
- **traverse()** - Visits all nodes in sequence.

Advantages of Linked Lists

- **Dynamic size** - The structure can grow and shrink at runtime.
- **Efficient insertion and deletion at known positions** - Nodes can be relinked without shifting many elements.
- **No need for contiguous memory** - Nodes may be stored anywhere in memory.
- **Useful for implementing other ADTs** - Stacks, queues, and adjacency lists can be based on linked lists.

Limitations of Linked Lists

- **No direct random access** - Accessing an element requires traversal from the head or tail.
- **Additional memory overhead** - Each node stores references in addition to data.
- **Lack of spatial locality** - Non-contiguous allocation reduces CPU cache performance due to pointer chasing.
- **More complex implementation** - Pointer or reference management can introduce errors.

Applications of Linked Lists

Linked lists are used for:

- implementing stacks and queues;
- representing adjacency lists in graphs;
- managing dynamic memory;
- building undo and redo systems;
- implementing polynomial and sparse matrix representations;

- creating circular buffers and scheduling structures.

1.3.4. 1.3.4. Stacks

A **stack** is a linear abstract data type that follows the **Last In, First Out** principle, commonly abbreviated as **LIFO**.

This means that the last element inserted into the stack is the first element removed from it.

The insertion operation is called **push**, and the removal operation is called **pop**.

Main Operations on Stacks

- **push(value)** - Adds a value to the top of the stack.
- **pop()** - Removes and returns the top value.
- **peek()** - Returns the top value without removing it.
- **isEmpty()** - Checks whether the stack is empty.
- **size()** - Returns the number of elements.

Characteristics of Stacks

- **Restricted access** - Elements are accessed only from the top.
- **LIFO behavior** - The most recently added element is removed first.
- **Simple interface** - Stack operations are small and clearly defined.
- **Useful for nested structures** - Stacks naturally model recursion and nested expressions.

Applications of Stacks

Stacks are used in:

- function call management;
- recursion;
- expression evaluation;
- syntax parsing;
- undo operations;
- browser history;
- depth-first search;
- backtracking algorithms.

1.3.5. 1.3.5. Queues

A **queue** is a linear abstract data type that follows the **First In, First Out** principle, commonly abbreviated as **FIFO**.

This means that the first element inserted into the queue is the first element removed from it.

The insertion operation is usually called **enqueue**, and the removal operation is called **dequeue**.

Main Operations on Queues

- **enqueue(value)** - Adds a value to the rear of the queue.
- **dequeue()** - Removes and returns the front value.
- **front()** - Returns the front value without removing it.
- **isEmpty()** - Checks whether the queue is empty.
- **size()** - Returns the number of elements.

Common Variations

- **Simple queue** - Elements are inserted at the rear and removed from the front.
- **Circular queue** - The last position connects back to the first position, improving fixed-buffer utilization.
- **Priority queue** - Elements are removed according to priority rather than insertion order.
- **Deque** - A double-ended queue that allows insertion and removal at both ends.

Applications of Queues

Queues are used in:

- task scheduling;
- print job management;
- breadth-first search;
- buffering data streams;
- handling asynchronous events;
- managing requests in operating systems and servers.

1.4. 1.4. Non-Linear Abstract Data Types

Non-linear abstract data types organize elements in structures that are not simple sequences.

Instead of a single logical order, they represent hierarchical, associative, or network-like relationships.

The most important non-linear ADTs are associative arrays, trees, and graphs.

1.4.1. 1.4.1. Associative Arrays

An **associative array** is an abstract data type that stores a collection of key-value pairs. Each key appears at most once in the collection and is used to access its associated value.

Associative arrays are also known as **maps**, **dictionaries**, or **symbol tables**, depending on the programming language and context.

Unlike regular arrays, associative arrays:

- use keys instead of numeric indexes;
- associate each key with a value;
- usually do not guarantee positional ordering, depending on implementation;
- are optimized for key-based lookup.

An associative array follows these rules:

- a value can be accessed using its key;
- the same value may be stored under multiple different keys;
- a single key can map to only one value at a time;
- keys must be distinguishable from one another.

Main Operations on Associative Arrays

- `insert(key, value)` - Adds a key-value pair.
- `find(key)` - Returns the value associated with the key.
- `remove(key)` - Deletes the key-value pair.
- `contains(key)` - Checks whether a key exists.
- `size()` - Returns the number of key-value pairs.

Hash-Based Associative Arrays

«Associative arrays are most commonly implemented using hash tables (providing $O(1)$ average-time complexity) or balanced search trees (providing $O(\log n)$ worst-case guarantees).

A hash table uses a **hash function** to convert a key into an index in an internal array. This makes lookup, insertion, and deletion efficient on average.

The idea of hashing was significantly developed by Hans Peter Luhn in 1953. Later, hashing became one of the fundamental techniques for building efficient symbol tables, dictionaries, caches, and databases.

Applications of Associative Arrays

Associative arrays are used for:

- dictionaries and maps;
- frequency counting;
- caching;
- indexing records by identifiers;

- symbol tables in compilers;
- configuration storage;
- database indexing concepts.

1.4.2. 1.4.2. Trees

A **tree** is a non-linear abstract data type that represents hierarchical relationships between elements.

A tree consists of nodes connected by edges. One node is designated as the **root**. Every node except the root has exactly one parent, and a node may have zero or more children.

Trees are acyclic, which means they do not contain cycles.

Hierarchical Model

Tree terminology includes the following concepts:

- **Root** - The top node of the tree.
- **Parent** - A node that has one or more children.
- **Child** - A node connected below another node.
- **Leaf** - A node with no children.
- **Internal node** - A node with at least one child.
- **Subtree** - A tree formed by a node and its descendants.
- **Depth** - The distance from the root to a node.
- **Height** - The length of the longest path from a node to a leaf.

Fundamental Properties

A tree with n nodes has exactly $n - 1$ edges.

There is exactly one path from the root to any node.

Because of these properties, trees are useful for representing hierarchical data, decision processes, and sorted collections.

Structural Classification

Common types of trees include:

- **Binary tree** - Each node has at most two children.
- **Binary search tree** - A binary tree where left descendants are smaller and right descendants are greater than the node.
- **AVL tree** - A self-balancing binary search tree.
- **B-tree** - A balanced tree structure widely used in databases and file systems.

- **Trie** - A tree used to store strings by prefixes.
- **Heap** - A tree-based structure used to implement priority queues.

Historical Context

Tree structures became central to computer science with the development of symbolic computation, parsing, searching, and file organization.

John McCarthy's work on Lisp in 1960 popularized recursive list and tree-like structures in programming language theory.

Donald Knuth later systematized tree algorithms in **The Art of Computer Programming**, making trees one of the core topics in the study of data structures.

Applications of Trees

Trees are used in:

- file systems;
- database indexes;
- syntax trees in compilers;
- decision trees;
- search algorithms;
- priority queues;
- autocomplete systems;
- hierarchical document models.

1.4.3. 1.4.3. Graphs

A **graph** is a non-linear abstract data type that represents relationships between objects.

A graph consists of a set of **vertices** and a set of **edges**. Vertices represent entities, while edges represent connections or relationships between them.

Graphs are among the most general and powerful structures in computer science.

Relational Model

A graph is commonly written as:

$$G = (V, E)$$

where:

- **V** is a set of vertices;
- **E** is a set of edges connecting vertices.

If edges have direction, the graph is called a **directed graph** or **digraph**.

If edges have no direction, the graph is called an **undirected graph**.

Typological Spectrum

Common types of graphs include:

- **Undirected graph** - Edges have no direction.
- **Directed graph** - Edges have direction.
- **Weighted graph** - Edges have associated weights or costs.
- **Unweighted graph** - Edges have no weights.
- **Connected graph** - There is a path between every pair of vertices.
- **Disconnected graph** - Some vertices cannot be reached from others.
- **Cyclic graph** - The graph contains at least one cycle.
- **Acyclic graph** - The graph contains no cycles.
- **Directed acyclic graph** - A directed graph with no directed cycles.
- **Multigraph** - Multiple edges may connect the same pair of vertices.
- **Hypergraph** - An edge may connect more than two vertices.

Mathematical Foundations

Graph theory began with Leonhard Euler's solution to the Seven Bridges of Königsberg problem in 1736.

This problem is widely considered the origin of graph theory. Euler showed that the possibility of walking through all bridges exactly once depends on the degree of the vertices.

Later, graph theory became essential in network analysis, optimization, transportation, scheduling, and computer science.

In the twentieth century, researchers such as Paul Erdős contributed significantly to random graph theory and combinatorics.

Graph algorithms also became central to operations research. For example, Lester Ford and Delbert Fulkerson developed important methods related to network flows in the 1950s.

Abstract Operations

Common graph operations include:

- `addVertex(vertex)` - Adds a new vertex.
- `addEdge(source, target)` - Adds an edge between vertices.
- `removeVertex(vertex)` - Removes a vertex and its incident edges.
- `removeEdge(source, target)` - Removes an edge.
- `getNeighbors(vertex)` - Returns adjacent vertices.

- `hasPath(source, target)` - Checks whether a path exists.
- `traverse(start)` - Visits vertices according to a traversal strategy.

Graph Representations

Graphs are commonly represented using:

- **Adjacency matrix** - A two-dimensional matrix that indicates whether edges exist between vertices.
- **Adjacency list** - A collection where each vertex stores a list of its neighbors.
- **Edge list** - A list of all edges in the graph.

Each representation has different memory and performance characteristics.

An adjacency matrix is simple and provides fast edge lookup, but it can consume large amounts of memory for sparse graphs.

An adjacency list is usually more memory-efficient for sparse graphs and is widely used in practical graph algorithms.

Applications of Graphs

Graphs are used in:

- computer networks;
- social networks;
- transportation systems;
- dependency analysis;
- compilers;
- recommendation systems;
- search engines;
- routing algorithms;
- project scheduling;
- artificial intelligence.

1.5. 1.5. Summary

Abstract data types provide a formal way to describe data structures independently of implementation details.

The main idea of an ADT is to define what operations are available and what behavior they must have, while hiding the internal representation.

Linear ADTs, such as arrays, linked lists, stacks, and queues, organize data sequentially.

Set-like ADTs focus on uniqueness and membership.

Associative arrays organize data as key-value pairs.

Trees represent hierarchical relationships.

Graphs represent general relationships between entities.

Understanding these abstract models is essential before studying concrete implementations, algorithmic complexity, memory behavior, and performance on real computer architectures.

Chapter 2. Section 2. JavaScript Programming Language Overview

2.1. 2.1. What Is the JavaScript Programming Language?

This section provides a foundational overview of the JavaScript programming language, tailored for its application in the context of data structures and algorithms. We will explore the core syntax, data types, and control flow mechanisms essential for implementing and understanding various data structures and algorithmic concepts. Special attention will be given to JavaScript's dynamic nature and object-oriented capabilities, which play a crucial role in efficient algorithm design and data manipulation. This overview aims to equip readers with the necessary JavaScript skills to effectively follow the subsequent chapters on advanced data structures and algorithms.

Since this book takes a hybrid approach to the software implementation of algorithms, it is essential to briefly describe the fundamental concepts and constructs of its components: the visual algorithmic language DRAKON and the JavaScript programming language. This approach is particularly promising in the field of education, as it fosters the formation and development of algorithmic and computational thinking. It is also highly applicable in computer science, particularly in the study of data structures and algorithms.

This section assumes prior familiarity with basic programming concepts in at least one modern high-level language. At the same time, the description of JavaScript in this book is limited to the constructs necessary for understanding data structure algorithms.

This section focuses on the core programming constructs in JavaScript, such as variables, arrays, objects, maps, logical operators, loops, functions, recursion, and more. Special emphasis is placed on data types, both primitive and complex. JavaScript prototypes and closures are also briefly discussed, as they play a key role in the language.

JavaScript (JS) is a high-level, dynamically typed programming language that was originally created by Brendan Eich in 1995 while working at Netscape Communications. Unlike Go, JavaScript is an interpreted language with automatic memory management (garbage collection) and is primarily used for web development, although its use has expanded significantly in recent years.

Below are the most important features of JavaScript:

- **Lightweight and flexible** – JavaScript is dynamically typed, meaning there is no need to explicitly define variable types. This makes it a flexible and beginner-friendly language.
- **Interpreted language** – JavaScript code is executed line by line by an engine (V8,

SpiderMonkey, or Chakra). This makes development more interactive but can also impact performance.

- **Prototype-based object model** – JavaScript does not use classical OOP (like in Java or C++). Instead, it relies on prototypes and constructor functions. With the introduction of ES6 classes, JavaScript provides syntactic sugar for working with objects in an OOP-like style.
- **Asynchronous programming** – JavaScript has non-blocking execution using callbacks, promises, and async/await. This makes it well-suited for handling asynchronous operations such as network requests.
- **Functional programming features** – JavaScript supports higher-order functions, closures, and anonymous functions. Functions in JavaScript are first-class citizens, meaning they can be passed as arguments, returned from other functions, and stored in variables.
- **Event-driven and single-threaded** – JavaScript relies on an event loop and asynchronous callbacks to manage concurrent operations.
- **Extensive ecosystem** – JavaScript has a massive standard library and an even larger third-party ecosystem through npm (Node Package Manager). This makes it one of the most versatile languages in modern development.

While JavaScript was originally designed for client-side web development, it has grown into a general-purpose language thanks to Node.js, which allows JavaScript to run on servers, IoT devices, and even embedded systems.

2.2. 2.2. Program Structure in JavaScript

JavaScript is a dynamically typed, interpreted programming language that is widely used for web development, backend services, and even desktop and mobile applications. A JavaScript program consists of a series of statements, functions, and modules that define its behavior.

Unlike some other programming languages, JavaScript does not require a package declaration or an explicit main() function. Instead, execution starts from the top of the script and proceeds line by line.

The basic structure of a JavaScript program can be organized as follows:

```
import fs from 'fs';

function main() {
  console.log(`Hello, World!`);
}

main();
```

Here import fs from 'fs' – JavaScript allows importing external libraries or built-in modules.

Function definition (function main()) – Functions encapsulate logic and can be reused.

Execution order – The script runs from top to bottom, and functions are called explicitly.

JavaScript supports two types of comments:

```
// This is a single-line comment
/*
This is a multi-line comment.
It can span multiple lines.
*/
```

The specifics of organizing the structure of a JavaScript program in the DRAKON environment are presented in Section 3. We will only note that in DRAKON technology, the program is structured through the `module()` function, which is a container for the main logic of the program. Inside `module()`, the `main()` function is usually called, which controls the main process of code execution.

```
function module() {
  function main() {
    console.log("Hello, World!");
  }

  main(); // Call the main function
}

// Call the module
module();
```

2.3. 2.3. Variables and Constants in JavaScript

In JavaScript, variables are used to store data values, similar to other programming languages. However, JavaScript is dynamically typed, meaning you do not need to explicitly declare the type of a variable — it is determined automatically based on the assigned value.

JavaScript provides three ways to declare variables:

- **var** – the traditional way (not recommended for modern code).
- **let** – for block-scoped variables that can be reassigned.
- **const** – for variables that cannot be reassigned after initialization.

A variable in JavaScript is declared using the `let` or `const` keyword, followed by an identifier (variable name).

```
let i = 10; // Integer
let s = "Hello"; // String
let f = true; // Boolean
```

Variables in JavaScript do not require type definitions, as types are inferred automatically.

```
let a = 25; // Number (integer)
let x = 32.5; // Number (floating point)
let z = "Enter a new value"; // String
```

If no initial value is assigned, JavaScript sets variables to undefined, unlike Go, which initializes them with default values.

In JavaScript, multiple variables can be declared and initialized in one line, separated by commas:

```
let a = 100, b = 200, c = 300;
```

JavaScript allows reassigning variables to different types:

```
let value = 10; // Initially a number
value = "text"; // Now a string
value = true; // Now a boolean
```

JavaScript allows implicit variable declaration (not recommended):

```
x = 2.5; // Automatically declared as a global variable (avoid this!)
```

Variable names in JavaScript must follow certain rules:

- Can contain letters, digits, \$, and _ (but no spaces).
- Cannot start with a digit.
- Are case-sensitive (myVar and myvar are different).
- Use camelCase naming convention (userName, totalPrice).

Constants are declared using const, meaning their values cannot be changed after initialization:

```
const item = "name";
const n = 25;
const y = 45.5;
```

2.4.2.4. Input and Output

In JavaScript, input and output operations differ significantly from those in compiled languages. Since JavaScript was initially designed for the web, it does not have built-in functions for reading user input from the terminal. Similarly, JavaScript outputs data using console.log(), which prints values to the console.

2.4.1. 2.4.1. Output in JavaScript

JavaScript provides several ways to display information:

a) Basic Output to Console

```
console.log("Hello"); // Output without a newline
console.log("Hello", 23); // Output multiple values with spaces
console.log(`The salary is ${32000}`); // Using template literals
```

It should be noted that `console.log()` automatically adds a newline unless specified otherwise.

b) Formatted Output Using `console.log()`

JavaScript provides formatted output:

```
console.log("Hello %d", 23); // Equivalent to Go's fmt.Printf("Hello %d", 23)
console.log("Name: %s, Salary: %d", "Smith", 32000);
```

c) Using Template Literals (Recommended)

Template strings provide an easy way to interpolate variables and expressions into strings.

```
let name = "Smith";
let salary = 32000;
console.log(`The salary of ${name} is ${salary}`);
```

Template literals (``) automatically replace variables inside `${}`.

2.4.2. 2.4.2. Input in JavaScript

JavaScript does not have built-in input functions for reading user data from the terminal. Instead, input depends on the execution environment:

a) Input in the Browser (Using `prompt()`)

In client-side JavaScript, `prompt()` can be used to get user input:

```
let name = prompt("Enter last name:");
let salary = prompt("Enter salary:");
console.log(`Salary of ${name} is ${salary}`);
```

The function `prompt()` always returns a string, so numbers must be converted using `parseInt()` or `parseFloat()`:

```
let salary = parseInt(prompt("Enter salary:"));
```

b) Input in Node.js (Using readline)

If running JavaScript in Node.js, we use the readline module to handle user input:

```
const readline = require("readline");

const q = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

q.question("Enter last name: ", function(name) {
  q.question("Enter salary: ", function(salary) {
    console.log(`Salary of ${name} is $$${salary}`);
    q.close();
  });
});
```

To Summarize:

Feature	JavaScript (Browser)	JavaScript (Node.js)
Output (Console)	console.log()	console.log()
Formatted Output	Template literals (``)	Template literals (``)
Basic Input	prompt()	readline module
Number Input	parseInt(prompt())	parseInt(input)

2.5.2.5. Decision-Making Operators in JavaScript

In programming, decision-making operators allow a program to choose different execution paths based on conditions. JavaScript provides several control structures for decision-making:

- **Conditional branching:** if-else, switch-case
- **Loops for repeating actions:** for, while, do-while
- **Control flow statements:** break, continue, return

2.5.1.2.5.1. Conditional Statement if-else

The if-else statement allows executing different blocks of code depending on a condition.

Syntax of if-else in JavaScript:

```
if (condition) {
```

```
// Code executes if the condition is true
} else {
// Code executes if the condition is false
}
```

For single-line conditions, curly braces {} can be omitted.

Example of if-else Usage:

```
let ID = "Apple";

if (ID === "Apple") {
console.log("Enter your login and password");
} else if (ID === "Google") {
console.log("Your operating system is not supported");
} else {
console.log("Input error");
}
```

Key Points:

- Use === instead of == to check for equality (strict comparison).
- The else if block allows multiple conditions.
- The else block executes if none of the conditions are met.

A representation of this construct in DRAGON technology is shown in the next section.

2.5.2. switch Statement

The switch statement provides a simpler way to compare a variable with multiple values.

Syntax of switch-case in JavaScript:

```
switch (expression) {
case value1:
// Code for case 1
break;
case value2:
// Code for case 2
break;
default:
// Code if no cases match
}
```

The break statement prevents fall-through execution.

Example of switch-case Usage:

```
let ID = "Apple";

switch (ID) {
case "Apple":
console.log("Enter your username and password");
break;
case "Google":
console.log("Your operating system is not supported");
break;
default:
console.log("Input error");
}
```

Key Points:

- The switch statement checks the ID variable.
- The break statement ensures only one case executes.
- The default block runs if none of the cases match.

2.5.3. 2.5.3. Loops in JavaScript

JavaScript supports multiple loop types for repeating actions.

a) Classic for Loop

```
for (let i = 0; i < 8; i++) {
console.log(i);
}
```

b) while Loop (Preconditioned Loop)

Executes as long as the condition is true:

```
let count = 10;

while (count > 0) {
console.log(count);
count--;
}
```

c) do-while Loop

Ensures the loop executes at least once, even if the condition is false.

```
let num = 5;
```



```
do {  
  console.log(num);  
  num--;  
} while (num > 0);
```

To Summarize:

Feature	JavaScript Equivalent
Conditional statement	if-else
Multiple condition checking	switch-case
Loop with counter	for loop
Loop with condition	while loop
Guaranteed execution loop	do-while loop

2.6.2.6. Looping Through Ranges in JavaScript

In JavaScript, loops allow iterating over collections such as arrays, objects, and maps. JavaScript provides several ways to loop through a range of values or iterate over arrays, objects, and maps.

2.6.1. 2.6.1. Iterating Over Arrays Using for-of

JavaScript provides the for-of loop to iterate over arrays and other iterable objects.

Syntax of for-of in JavaScript:

```
for (let element of iterable) {  
  // Code to execute for each element  
}
```

Example: Summing an Array

```
let nums = [2, 3, 4];  
let sum = 0;  
  
for (let num of nums) {  
  sum += num;  
}  
  
console.log("Sum:", sum);
```

2.6.2. 2.6.2. Iterating Over an Array with Indexes (forEach)

If we need both the index and value, we can use forEach:

```
let nums = [2, 3, 4];

nums.forEach((num, index) => {
  if (num === 3) {
    console.log("Index:", index);
  }
});
```

2.6.3. 2.6.3. Iterating Over Objects Using for-in

JavaScript does not have a built-in range keyword, but it provides for-in to iterate over object properties (similar to key-value pairs).

Example: Iterating Over an Object

```
let capitals = {
  France: "Paris",
  Germany: "Berlin"
};

for (let key in capitals) {
  console.log(`${key} -> ${capitals[key]}`);
}
```

2.6.4. 2.6.4. Iterating Over Maps Using forEach() and for-of

For Map objects, we use forEach() or for-of with .entries().

Example: Iterating Over a Map

```
let capitals = new Map([
  ["France", "Paris"],
  ["Germany", "Berlin"]
]);

for (let [key, value] of capitals.entries()) {
  console.log(`${key} -> ${value}`);
}
```

2.6.5. 2.6.5. Iterating Over Keys Only

If we only need the keys:

```
for (let key of capitals.keys()) {
  console.log("Key:", key);
}
```

```
}
```

To Summarize:

JavaScript Construct	Usage
for-of	Iterating over arrays (values only)
forEach()	Iterating over arrays with index and value
for-in	Iterating over object keys
.entries() with for-of	Iterating over maps (key-value pairs)
.keys() with for-of	Iterating over map keys only

2.7. 2.7. Data Types in JavaScript

Data types define the kind of values that variables can store in a program. They also determine the operations that can be performed on the data.

JavaScript has dynamic typing, meaning that variables do not have a fixed type but are assigned a type based on their value.

The main categories of data types in JavaScript include:

- **Primitive Types:** Numbers, strings, booleans, null, undefined, Symbol, and BigInt
- **Reference Types:** Objects, arrays, functions, and maps

2.7.1. 2.7.1. Primitive Data Types

a) Numeric Data Types

JavaScript uses a single Number type for both integers and floating-point values.

Example: Declaring Numeric Variables

```
let integerNumber = 42; // Integer
let floatingPoint = 3.14; // Floating-point number
```

JavaScript also supports BigInt, a special type for working with very large integers.

```
let bigNumber = 9007199254740993n; // BigInt type (suffix 'n' is required)
```

Numeric Limits

The maximum and minimum values for numbers are stored in constants:

```
console.log(Number.MAX_SAFE_INTEGER); // 9007199254740991
```

```
console.log(Number.MIN_SAFE_INTEGER); // -9007199254740991
console.log(Number.MAX_VALUE); // 1.7976931348623157e+308
console.log(Number.MIN_VALUE); // 5e-324
```

Key Points:

- Number type covers both integers and floating-point values.
- BigInt allows working with very large integers beyond Number.MAX_SAFE_INTEGER.
- JavaScript automatically converts between integer and floating-point when needed.

b) String Variables and Constants

A string in JavaScript is a sequence of characters enclosed in single quotes ('), double quotes ("), or backticks (`) for template literals).

Declaring String Variables

```
let greeting = "Hello, World"; // Double quotes
let name = 'John'; // Single quotes
let message = `Hello, ${name}`; // Template literals (ES6+)
```

Key Features:

- Strings are immutable (modifying a string creates a new string).
- Template literals allow embedding variables using \${} inside backticks (`).

String Length and Character Access:

```
console.log(greeting.length); // 12
console.log(greeting[0]); // 'H'
console.log(greeting.charCodeAt(0)); // Unicode code of 'H' (72)
```

Unicode Representation (Hex Output)

To get the hexadecimal representation of characters:

```
for (let i = 0; i < greeting.length; i++) {
  console.log(greeting.charCodeAt(i).toString(16));
}
```

Key Features:

- The length property returns the number of characters in a string.
- charCodeAt(i) provides the Unicode value of a character.

c) Boolean Variables and Constants

Boolean values are either true or false. They are commonly used in comparisons and conditional logic.

Declaring Boolean Variables

```
let isJavaScriptFun = true;
let isWeekend = false;
```

Boolean Expressions:

```
let x = 5, y = 8;
console.log("x == y:", x == y); // false
console.log("x != y:", x != y); // true
console.log("x < y:", x < y); // true
console.log("x > y:", x > y); // false
console.log("x <= y:", x <= y); // true
console.log("x >= y:", x >= y); // false
```

Key Features:

- Booleans are mainly used for conditional logic (if statements, loops, etc.).
- The comparison operators (==, !=, <, >, <=, >=) return boolean values.

Summary of Primitive Data Types:

Data Type	Example	Description
Number	let x = 10;	Holds integer or floating-point values.
BigInt	let y = 12345678901234567890n;	Used for extremely large integers.
String	let s = "Hello";	Sequence of characters.
Boolean	let isTrue = true;	Holds true or false.
Undefined	let x;	Variable is declared but not assigned a value.
Null	let y = null;	Represents an intentional absence of value.

2.8. 2.8. Composite Data Types in JavaScript

JavaScript provides composite data types to store and manipulate collections of data and complex structures. The most commonly used composite types are:

- **Arrays** – Ordered collections of elements.
- **Objects** – Key-value pairs that represent complex data structures.

- **Maps** – Key-value storage with enhanced capabilities.

2.8.1. 2.8.1. Arrays

An array is an ordered collection of elements, where each element is indexed numerically, starting from 0. Arrays in JavaScript are dynamic, meaning they can grow and shrink in size.

Declaring and Initializing Arrays

```
let numbers = [10, 20, 30, 40]; // Array with 4 elements
console.log(numbers[0]); // Access first element (10)
console.log(numbers.length); // Array length (4)
```

Key Features:

- Arrays are zero-indexed (array[0] is the first element).
- The .length property returns the number of elements.
- Arrays can store multiple types of values:

```
let mixedArray = [42, "text", true, null];
```

Adding and Removing Elements

```
let fruits = ["Apple", "Banana"];
fruits.push("Orange"); // Add element to the end
fruits.pop(); // Remove last element
fruits.unshift("Grapes"); // Add to the beginning
fruits.shift(); // Remove first element
console.log(fruits);
```

□ JavaScript arrays are flexible – they can grow, shrink, and store mixed data types.

2.8.2. 2.8.2. Objects

An object is a collection of key-value pairs, where each key is a property that stores a value.

Declaring an Object

```
let employee = {
  firstName: "Max",
  lastName: "Smith",
  age: 42,
  phone: 123456789,
  salary: 34000,
  address: "Amarillo"
};
```

```
console.log(`Employee: ${employee.firstName} ${employee.lastName}`);
console.log(`Salary: ${employee.salary}`);
```

Key Features:

- Properties are accessed using dot notation (object.property) or bracket notation (object["property"]).
- Objects store heterogeneous data (unlike arrays).

Updating and Adding Properties

```
employee.age = 43; // Updating a property
employee.department = "IT"; // Adding a new property
console.log(employee);
```

2.8.3. 2.8.3. Map

```
let capitals = new Map([
  ["France", "Paris"],
  ["Germany", "Berlin"]
]);

console.log(capitals.get("France")); // Access value by key
capitals.set("Spain", "Madrid"); // Add new key-value pair
console.log(capitals);
```

Maps vs. Objects:

- Objects use only strings as keys, while Maps can use any type of key.
- Maps maintain key order, whereas objects do not guarantee order.

Summary of Composite Data Types:

Data Type	Example	Description
Array	let arr = [1, 2, 3];	Ordered collection of elements.
Object	let obj = { name: "Max" };	Collection of key-value pairs.
Map	let map = new Map();	Advanced key-value storage.

2.9. 2.9. Inheritance, Encapsulation, and Polymorphism in JavaScript

2.9.1. 2.9.1. General Features of OOP

Inheritance, encapsulation, and polymorphism are the three pillars of object-oriented programming (OOP) that play a key role in modern programming languages. They help create more modular, flexible, and maintainable code.

Core OOP Principles and Their Roles:

Inheritance

- Allows creating new classes (subclasses) based on existing ones (superclasses), inheriting their properties and methods.
- Promotes code reuse and the creation of class hierarchies.
- In JavaScript, inheritance is implemented through prototypes or using classes (ES6).

Encapsulation

- Hides internal implementation details of a class and provides access to them only through public methods.
- Protects data from unauthorized access and enhances code reliability.
- In JavaScript, encapsulation is achieved through closures or modules.

Polymorphism

- Enables objects of different classes with a common interface to behave differently.
- Makes code more flexible and adaptable to changes.
- In JavaScript, polymorphism is implemented through dynamic typing and method overriding.

2.9.2. 2.9.2. Implementing OOP Principles Using Factory Functions

Factory functions provide a flexible and powerful way to implement OOP principles in JavaScript. Using factory functions helps to prevent the problems of classical prototypical inheritance. With the help of factory functions, you can use object composition, which in many cases is a more flexible solution than inheritance.

Encapsulation with Factory Functions

In this book, the specified OOP principles are implemented based on the use of factory functions.

```
function createPerson(name, age) {
  let _name = name; // private variable
  let _age = age; // private variable

  return {
    getName: function() {
      return _name;
    },
    getAge: function() {
```



```

return _age;
},
setAge: function(newAge) {
  if (newAge > 0 && newAge < 150) {
    _age = newAge;
  }
}
};
}

const person = createPerson("Ivan", 30);
console.log(person.getName()); // Output: Ivan
person.setAge(31);
console.log(person.getAge()); // Output: 31
// console.log(person._age); // Error: _age is not accessible from outside

```

Composition (Alternative to Inheritance) with Factory Functions

```

const canBark = (state) => ({
  bark: () => console.log("Woof!")
});

const canWalk = (state) => ({
  walk: () => console.log("Walking...")
});

const createDog = (name) => {
  let state = {
    name: name
  };
  return Object.assign(
    {},
    state,
    canBark(state),
    canWalk(state)
  );
};

const dog = createDog("Bobik");
dog.bark(); // Output: Woof!
dog.walk(); // Output: Walking...

```

Polymorphism with Factory Functions

```

const createCircle = (radius) => ({
  calculateArea: () => Math.PI * radius * radius
});

const createSquare = (side) => ({

```

```
calculateArea: () => side * side
});

function printArea(shape) {
  console.log("Area:", shape.calculateArea());
}

const circle = createCircle(5);
const square = createSquare(4);

printArea(circle); // Output: Area: 78.53981633974483
printArea(square); // Output: Area: 16
```

2.10. 2.10. Summary

This section provided a comprehensive overview of the JavaScript programming language and the DRAKON visual algorithmic language, establishing the practical foundation for implementing data structures and algorithms in this book.

JavaScript's core syntax, dynamic typing, flexible object model, and functional programming capabilities (such as first-class functions and closures) make it a versatile tool for algorithm design. We explored both class-based and object-composition paradigms, alongside essential control structures and asynchronous primitives (*Promises*, *async/await*).

In parallel, the DRAKON language introduces a visual, structured methodology for algorithm specification, minimizing cognitive load and preventing common flow-control errors. Together with JavaScript, this hybrid approach bridges high-level algorithmic reasoning with robust, modern software implementation.

With the core language mechanics and visual modeling principles established, we are fully prepared to proceed to concrete data structure implementations, algorithmic analysis, and performance optimization in the upcoming sections.

Chapter 3. Section 3. Implementation of Basic Abstract Data Types in JavaScript

This section delves into the practical implementation of basic Abstract Data Types (ADTs) within the JavaScript programming language. ADTs, as mathematical models with defined operators, provide a foundation for algorithm development. However, translating these conceptual models into executable code requires representing ADTs using the data types and methods available in JavaScript. We will explore how to bridge the gap between abstract data structures and their concrete JavaScript implementations, demonstrating techniques to effectively utilize JavaScript's features to realize fundamental ADTs. This section will equip readers with the knowledge to translate theoretical ADT concepts into functional JavaScript code, laying the groundwork for more complex data structure and algorithm implementations.

3.1. 3.1. Data Structuring Features

In practical terms, a data structure is a collection of data structured in such a way as to ensure efficient use by users. This requires a certain ordering of data, primarily at the level of storage in computer memory. Reducing space and increasing organization in conjunction with reducing the time complexity of different tasks is the main objective of working with data structures.

Considerable experience in the development of computer technology and computing technology has made it possible to classify data structures into different categories. First, simple and integrated structures are distinguished by complexity. The criterion of simplicity is indivisibility — that is, in a computer implementation, a simple chain of bits. Simple, basic structures include variables of different types: integers, real numbers, booleans, strings.

Integrated (composite, complex) structures are data structures whose component parts are other data structures, including simple and integrated. Many basic integrated data structures are predefined by a specific programming language: arrays, slices, structures, etc. Such structures can be created by users for a specific problem, using basic integrated structures.

By way of presentation, data structures are divided into logical and physical. The logical structure is an abstract data layout that the user or programmer envisions. The physical structure is a method (schema) of specific data placement in computer memory. In general, the logical and physical structures of the same data do not coincide. In a logical (abstract) structure, data tend to be adjacent to each other, while in a computer implementation these data may be located in different memory regions.

An important feature of a data structure is the existence of links between its elements. On this basis, we distinguish disconnected and connected structures. Disconnected structures are

characterized by a lack of linkages between the elements, while connected structures are characterized by linkages. Disconnected structures include arrays, strings, stacks, queues, and linked lists. In many cases, data can be characterized by variability — that is, a change in the number of elements and/or the relationships between elements of the structure. Arrays, sets, records, tables fall under the category of static data; stacks, queues, trees fall under semi-static data; linked lists and graphs fall under dynamic data.

Linear and non-linear data structures are distinguished by structural order. Linear structures are divided into structures with sequential distribution of elements in memory (vectors, strings, arrays, stacks, queues) and structures with arbitrary connected distribution of elements in memory, depending on the nature of the reciprocal arrangement of elements in memory (singly linked and doubly linked linear lists). Nonlinear structures are multi-linked lists, trees, graphs.

One of the defining features of data structures is how to access data. In the access method, a search mechanism is important — an algorithm that determines the access path possible within a given memory structure, and the number of steps along that path to find the sought data. There are two main groups of data access methods: sequential and direct.

Sequential access means that a group of elements is accessed in a predefined ordered sequence. An example of sequential access is a singly linked list. Direct access, by contrast, means that individual elements of the data structure are accessed directly by providing a unique address for each element.

Finally, one characteristic of data structures is homogeneity. Homogeneous structures contain a set of simple data of the same type (numeric, boolean, string, etc.). Heterogeneous structures combine data of different types. Examples of homogeneous structures are arrays, slices, and stacks. Heterogeneous structures include records and sets.

The presence of a large number of data structure features has predetermined various attempts to classify them. From a programming point of view, the main features are: linearity/non-linearity, data access, and data homogeneity/heterogeneity. This section covers the JavaScript implementation of linear abstract data types such as arrays/slices, maps, linked lists, stacks, and queues.

3.2. 3.2. Linear Data Structures

Linear data structures are structures in which data elements are arranged in sequence. Linear structures can be distinguished by the way the individual elements of a data collection are accessed and by the homogeneity of the data (homogeneous and heterogeneous).

3.2.1. 3.2.1. Arrays in JavaScript

An array is a collection of data where each element is stored at a specific index. Arrays allow efficient storage and manipulation of multiple values in a single variable.

For example, the collection of integers [31, 26, 19, 24, 32, ..., 35, 41, 14, 33, 29] forms an array (Figure 3.1).

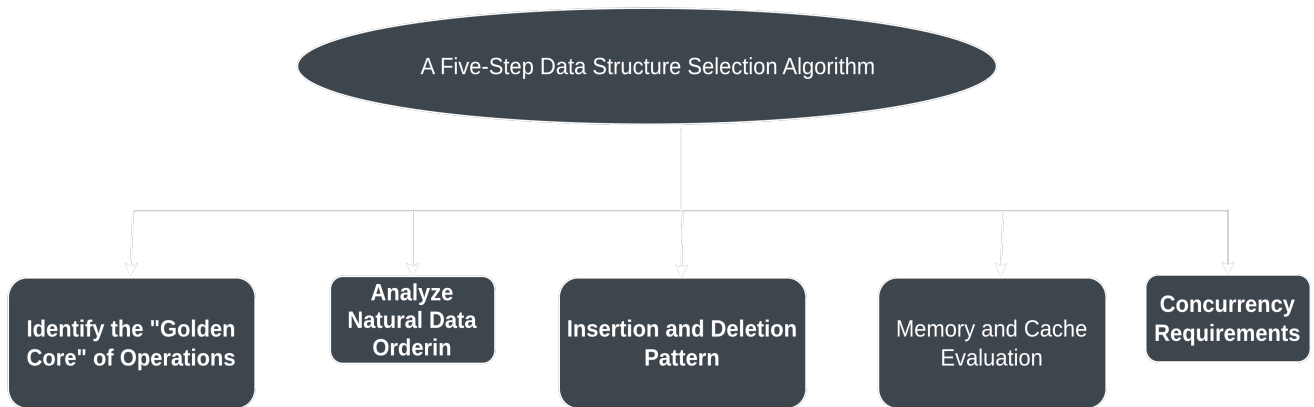


Figure 10.1. Memory hierarchy of a modern computer



Text is not SVG - cannot display

Figure 3.1. One-Dimensional Array

Declaring Arrays in JavaScript

JavaScript arrays are dynamic, meaning they can grow and shrink as needed. Unlike some other languages, JavaScript allows mixing different types within an array, but best practice is to keep arrays homogeneous (storing values of the same type).

Creating Arrays

Arrays in JavaScript can be declared in different ways:

a) Using `[]` (Recommended)

```
let numbers = [24, 12, 36, 6, 47, 11];
```

Using the Array Constructor

```
let numbers = new Array(24, 12, 36, 6, 47, 11);
```

This method is rarely used in modern JavaScript.

Multidimensional Arrays

JavaScript supports multidimensional arrays by storing arrays inside arrays.

```
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  ...
];
```

```
[7, 8, 9]
];

console.log(matrix[1][2]); // Output: 6
```

Here, `matrix[1][2]` accesses the second row, third column (6).

Summary

Feature	JavaScript Equivalent
Declaring an array	<code>let arr = [item1, item2, ...];</code>
Using the Array constructor	<code>let arr = new Array(item1, item2, ...);</code>
Accessing elements	<code>arr[index]</code>
Multidimensional arrays	<code>let matrix = [[...], [...]];</code>

3.2.2. 3.2.2. Slices

A slice is a variable-length data collection that stores elements of a homogeneous type. A slice can be thought of as a "slice" of an array. Slice syntax in JavaScript is `[]` or `[value1, value2, value3, ..., valueN]`.

In JavaScript, slices are implemented dynamically. To create a slice (array) in JavaScript, you can use an array literal or the Array constructor. For example, creating `parentSlice` would look like this:

```
let parentSlice = new Array(20).fill(0); // Creating an array of 20 elements, filled with zeros
```

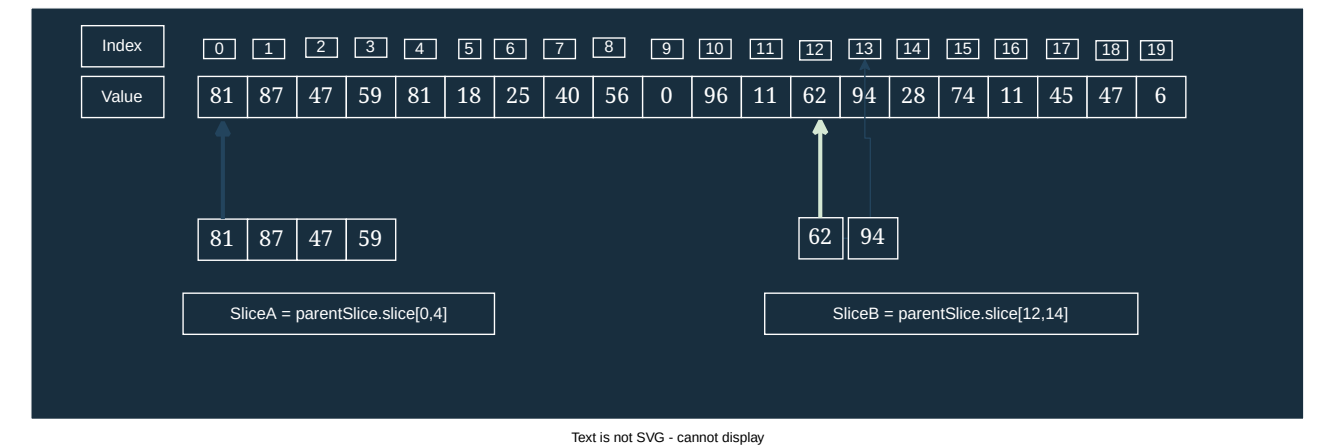


Figure 3.2. Creating Different Slices `sliceA` and `sliceB`

The `length` property indicates the number of elements in the slice (array). In JavaScript there is no explicit concept of "capacity". Multiple "slices" can be created from a single array using the `slice()` method or destructuring. For example, to create `sliceA` and `sliceB` from `parentSlice`:

```
let sliceA = parentSlice.slice(0, 4); // sliceA contains the first 4 elements (indices
```

```
0-3)
```

```
let sliceB = parentSlice.slice(12, 14); // sliceB contains 2 elements (indices 12-13)
```

In JavaScript, sliceA will contain elements from index 0 to 3, and sliceB will contain elements from index 12 to 13.

The following functions and methods are used to work with slices (arrays) in JavaScript:

- `push()`: Adds elements to the end of the array. If the array size is insufficient, it automatically increases.
- `length`: Property that returns the number of elements in the array.
- `slice()`: Creates a new array containing a "slice" of the original array.

3.2.3. 3.2.3. Objects in JavaScript

In JavaScript, objects serve the same purpose as structures in other programming languages. They allow us to store multiple related values (properties) in a single entity. Objects in JavaScript are flexible and support various ways of creation and manipulation.

JavaScript objects are defined using curly braces `{}` and consist of key-value pairs.

Creating an Object with Named Properties

```
let employee1 = {  
  firstName: "Peter",  
  lastName: "Wolf",  
  age: 35,  
  salary: 20000  
};  
console.log("Employee 1:", employee1);
```

Alternative Way: Object Constructor

Another way to create an object is by using the Object constructor.

```
let employee2 = new Object();  
employee2.firstName = "Nick";  
employee2.lastName = "Smith";  
employee2.age = 49;  
employee2.salary = 35000;  
  
console.log("Employee 2:", employee2);
```

Key Differences

- The first approach (object literal `{}`) is more common and recommended.

- The second approach (`new Object()`) is used when objects need to be created dynamically.

Creating an Object with a Factory Function

A factory function is a regular function that creates and returns an object.

```
function createEmployee(firstName, lastName, age, salary) {
  return {
    firstName,
    lastName,
    age,
    salary
  };
}

let emp1 = createEmployee("Peter", "Wolf", 35, 20000);
let emp2 = createEmployee("Nick", "Smith", 49, 35000);

console.log("Employee 1:", emp1);
console.log("Employee 2:", emp2);
```

Output:

```
Employee 1: { firstName: 'Peter', lastName: 'Wolf', age: 35, salary: 20000 }
Employee 2: { firstName: 'Nick', lastName: 'Smith', age: 49, salary: 35000 }
```

Adding Methods to a Factory Function

An object can contain methods (functions inside the object).

```
function createEmployee(firstName, lastName, age, salary) {
  return {
    getFullName: () => `${firstName} ${lastName}`,
    getSalary: () => salary, // You can read salary
    setSalary: (newSalary) => salary = newSalary // You can change salary
  };
}

let emp5 = createEmployee("Bob", "Anderson", 50, 30000);

console.log(emp5.getFullName()); // "Bob Anderson"
console.log(emp5.getSalary()); // 30000

emp5.setSalary(35000);
console.log(emp5.getSalary()); // 35000
```

Output:


```
Bob Anderson  
30000  
35000
```

3.2.4. 3.2.4. Map in JavaScript

In JavaScript, Maps are collections that store unordered key-value pairs, where each key is unique and maps to a corresponding value. Maps are particularly useful in data retrieval algorithms because they offer efficient lookups.

Unlike arrays, which use numeric indexes, Maps allow keys of any data type, making them more flexible (Figure 3.3).

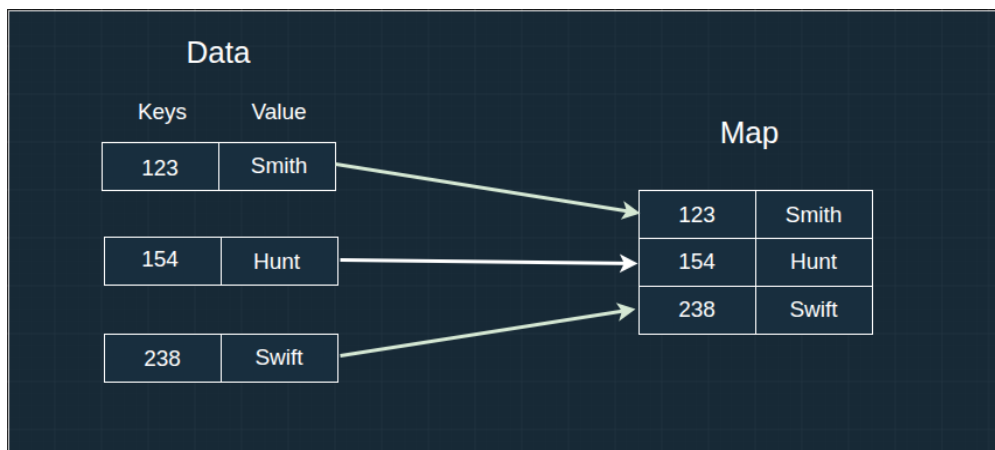


Figure 3.3. Map

In JavaScript, we use the Map object to create a map. The syntax is:

```
let myMap = new Map();
```

Here, myMap is an empty Map ready to store key-value pairs.

Declaring a Map Where Keys Are Strings and Values Are Numbers

We use the .set() method:

```
let studentScores = new Map();  
  
studentScores.set("Alice", 95);  
studentScores.set("Bob", 88);  
studentScores.set("Charlie", 92);  
  
console.log(studentScores);
```

Now, studentScores contains:

- "Alice" → 95

- "Bob" → 88
- "Charlie" → 92

Accessing Values in a Map

To retrieve a value from a map, use `.get(key)`:

```
console.log(studentScores.get("Alice")); // Output: 95
console.log(studentScores.get("Bob")); // Output: 88
```

If the key does not exist, `.get()` returns undefined.

3.2.5. 3.2.5. Linked List

A linked list is a dynamic data structure where each element, called a node, consists of two parts (Figure 3.4):

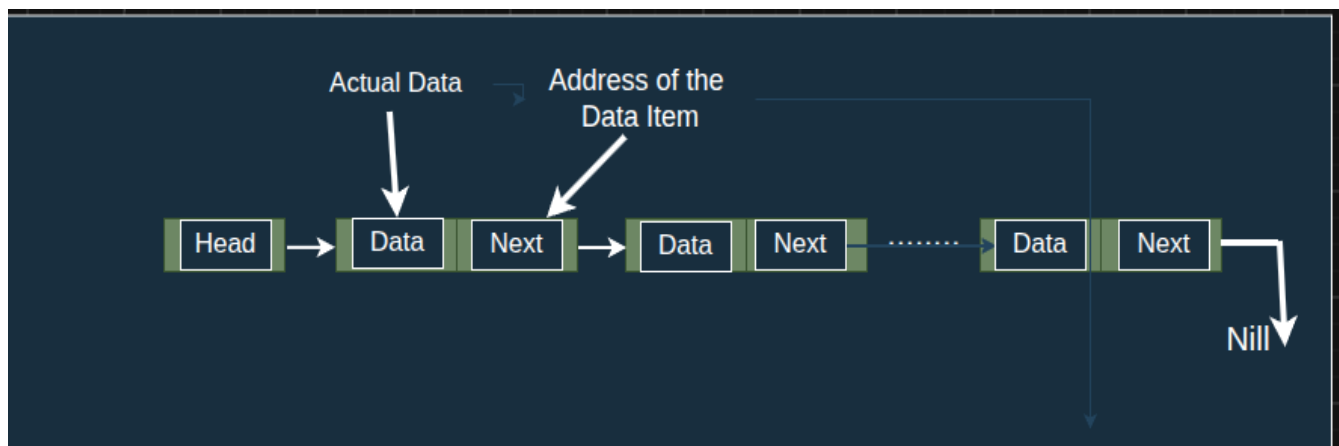


Figure 3.4. Singly Linked List Structure

a) Data (which can be any primitive or complex data type) b) A reference (pointer) to the next node in the list

Unlike arrays, where elements are stored in contiguous memory locations, linked lists store elements at different memory addresses and connect them via references.

They are widely used in memory management, file systems, and queue implementations.

In JavaScript, we represent a linked list node using an object.

Creating a Node

```
function createNode(data) {
  return {
    data,
    next: null
  };
}
```

Each node has:

- data – stores the value
- next – stores the reference to the next node (initially null)

Creating a Linked List

A linked list needs:

- a) A reference to the first node (head) b) A counter (size) to track the number of elements

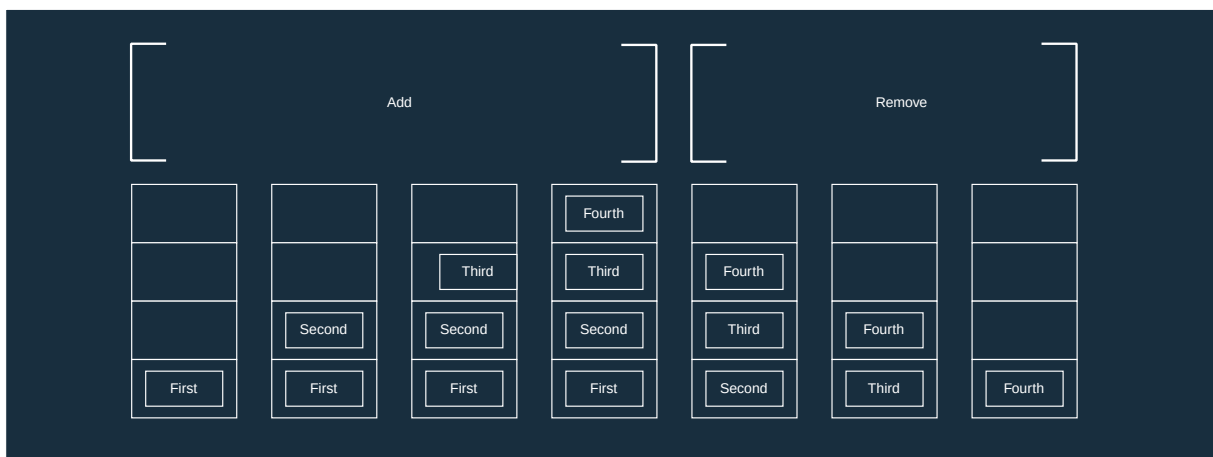
```
function createLinkedList() {  
  return {  
    head: null,  
    size: 0  
  };  
}
```

Here head points to the first node, and size stores the length of the list.

3.2.6. 3.2.6. Stack and Queue as Data Structures

- a) Stack

A stack is an abstract data type that contains elements with two basic operations: push, which adds an item to the collection, and pop, which deletes the last item added. From a technological point of view, a stack is memory where data values are loaded and retrieved according to the "Last-In-First-Out" (LIFO) strategy. Data enters the stack from only one side, called the top of the stack (Figure 3.5).



Text is not SVG - cannot display

Figure 3.5. Last-In-First-Out Stack Operation

A stack supports two primary operations:

- Push(item) – adds an element to the top of the stack

- Pop() – removes the top element from the stack

From a conceptual point of view, a stack can be compared to a stack of plates: you can only remove the top plate. To access a plate further down, you must remove all plates above it.

This structure is commonly used in:

- Undo (Ctrl+Z) functionality in text editors
- Managing function calls in recursion (function calls are stored in a stack)

Declaring a Stack in JavaScript

JavaScript does not have a built-in stack data type, but it can be implemented using arrays, as they provide push() and pop() methods.

```
function createStack() {
  let data = []; // Internal storage for stack elements

  return {
    push(item) {
      data.push(item);
    },

    pop() {
      if (data.length === 0) return undefined;
      return data.pop();
    },

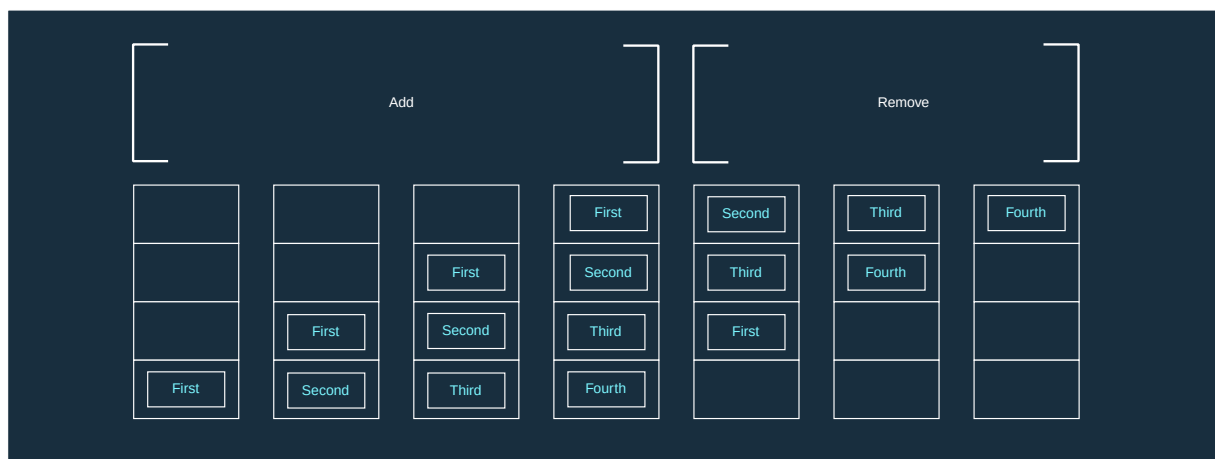
    peek() {
      return data[data.length - 1]; // View the top element
    },

    size() {
      return data.length;
    },

    isEmpty() {
      return data.length === 0;
    }
  };
}
```

b) Queue

A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle. This means that the first element added to the queue is the first one to be removed (Figure 3.6).



Text is not SVG - cannot display

Figure 3.6. First-In-First-Out Queue Operation

A queue supports two primary operations:

a) enqueue(item) – Adds an element to the back of the queue b) dequeue() – Removes an element from the front of the queue

Queues are widely used in:

- Task scheduling (CPU scheduling, print queue)
- Breadth-First Search (BFS) algorithms
- Message processing systems

3.2.7. 3.2.7. Representation of Binary Trees in JavaScript

A binary tree in JavaScript can be represented as a set of linked node objects. Each node contains a value and two references: one to the left child and one to the right child. If a child does not exist, the corresponding reference is null. The entire tree is accessed through a reference to its root node.

```
function createTree() {
  return { root: null };
}

function createNode(value) {
  return {
    value: value,
    left: null,
    right: null
  };
}
```

This factory-style representation avoids the use of class and this, while keeping the structure simple and explicit. It is convenient for describing algorithms in DRAGON diagrams: operations such as insertion, search, deletion, and traversal can be shown as manipulations of ordinary objects and

their left and right links. JavaScript code then directly follows the same logic, creating nodes with a factory function and updating references between them.

3.3. 3.3. Summary

This section covered the practical realization of fundamental Abstract Data Types (ADTs) in JavaScript, bridging theoretical concepts with concrete implementation techniques.

We examined both linear and non-linear data structures: * **Dynamic Arrays and Sets:** Leveraging JavaScript's built-in capabilities and performance tradeoffs. * **Linked Lists (Singly and Doubly):** Implementing pointer-based structures manually using factory functions and reference manipulation. * **Stacks and Queues:** Enforcing LIFO and FIFO behavior for operational constraints and algorithmic tasks such as parsing and state traversal. * **Binary Trees:** Representing hierarchical structures via linked node objects.

By relying on explicit factory functions rather than class-based syntax, these implementations maintain a direct correspondence with visual DRAGON diagrams. This hybrid methodology lays a solid foundation for evaluating memory dynamics, pointer operations, and algorithmic time complexity in subsequent sections.

Chapter 4. Section 4. DRAKON - Visual Algorithmic Language

This section explores the application of DrakonTech, a visual programming technology, to the design and implementation of data structures and algorithms. We will demonstrate how DrakonTech's graphical approach simplifies the representation of complex algorithmic logic, making it easier to understand, develop, and maintain efficient data structures and algorithms. We will cover the core principles of DrakonTech and illustrate its use in solving common algorithmic challenges, emphasizing its ability to enhance code clarity and reduce errors.

4.1. 4.1. Algorithm Representation Visualization

4.1.1. 4.1.1. Methods of Visualization of Algorithms

In theory and practice, the following forms of representation of algorithms have been formed:

- Words (written in natural language)
- Visual (images from graphic characters)
- Pseudocode (semiformalized descriptions of algorithms in conditional algorithmic language)
- Software (programming language texts)

Visual representations of algorithms were the most effective and common in terms of visualization and understanding. Conceptually, visualization is a form of presentation of information, data, and knowledge in the form of images, in order to achieve maximum convenience of perception, understanding, and analysis. Visual representation allows a better view and understanding of the structural elements used in the design of the algorithm, including the logic of their interaction.

The most common means of visualizing algorithmic constructions are flowcharts and Drakon-diagrams. The advantage of flowcharts is the availability of a corresponding State standard. However, flowcharts have recently been criticized: they are unsuitable for structural programming, difficult to formalize, and cannot be used to generate programming code.

An alternative to flowchart imaging is Drakon-based algorithm engineering, which results in Drakon-diagrams. Drakon-diagrams are suitable for formalized recording, automatic code retrieval, and execution on a computer. However, a more important aspect of Drakon-technology is its cognitive difference from flowchart technology. While flowcharts do improve program clarity, this is not always the case, and improvements are limited. In addition, there are many cases where unsuccessful flowcharts confuse the case and make understanding difficult. In contrast, Drakon-diagrams are much better understood (Figure 4.1).

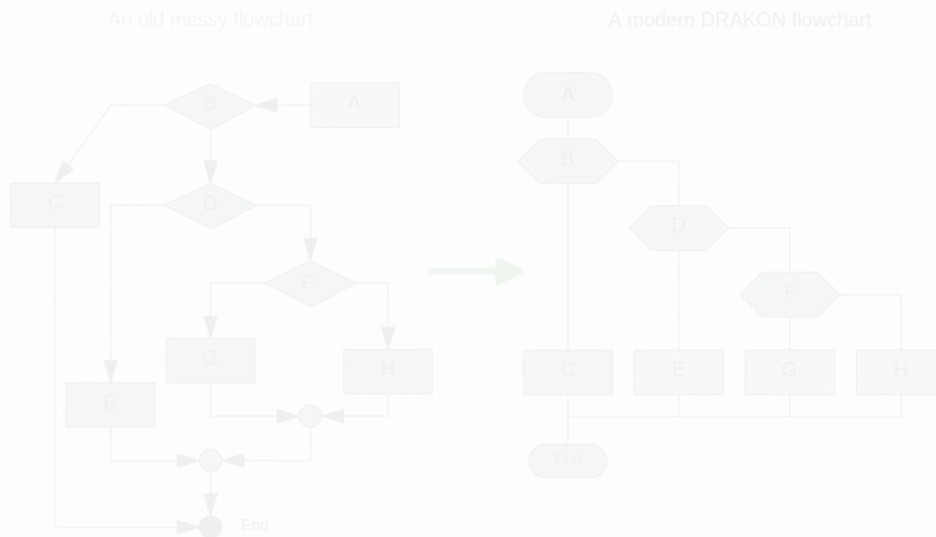


Figure 4.1. Comparison of a Visual Representation of an Algorithm in the Form of a Flowchart and a Drakon-Diagram

4.1.2. 4.1.2. Algorithmic Language Drakon

DrakonTech is a visual programming language with a strong emphasis on clarity and understandability, which makes it particularly well-suited for developing algorithmic thinking. Here is a breakdown of its purpose and potential.

Purpose:

- **Improving Algorithm Comprehension:** Drakon technology uses visual diagrams to represent algorithms, making them easier to grasp than traditional text-based code. This visual approach helps to clarify the flow of logic and the relationships between different parts of an algorithm.
- **Enhancing Communication:** The standardized visual language facilitates communication between developers, as well as between developers and non-technical stakeholders. This is especially valuable in complex projects where clear communication is essential.
- **Increasing Software Reliability:** By promoting clear and structured algorithms, Drakon technology helps to reduce errors and improve the reliability of software. This is particularly important in critical applications, such as those in aerospace or medical fields.
- **Facilitating Learning:** Drakon technology serves as an excellent tool for teaching and learning programming concepts, as it helps to visualize abstract ideas.

Potential for Developing Algorithmic Thinking:

- **Visualizing Logic:** The visual nature of DrakonTech allows users to see the logical flow of an algorithm, which helps to develop their ability to think logically and systematically.
- **Structuring Problems:** Drakon-technology encourages users to break down complex problems into smaller, more manageable steps, which is a fundamental aspect of algorithmic thinking.
- **Identifying Patterns:** By visualizing algorithms, users can more easily identify patterns and recurring structures, which can lead to more efficient and elegant solutions.
- **Debugging and Problem-Solving:** The visual representation of algorithms makes it easier to identify and correct errors, which helps to develop problem-solving skills.

- Abstraction: Drakon helps to teach abstraction by allowing people to look at the overall flow of a process without necessarily having to focus on the small granular details of every step.

In essence, Drakon-technology's strength lies in its ability to bridge the gap between abstract algorithmic concepts and concrete visual representations. This makes it a valuable tool for anyone seeking to develop their algorithmic thinking skills.

4.1.3. 4.1.3. DrakonTech - JavaScript Implementation of Drakon Technology

Visual Development with Flowcharts: DrakonTech allows you to build programs by creating flowcharts. Each flowchart becomes a JavaScript function. This visual approach ensures you can still understand your program even years later.

Readable and Standardized Diagrams: Drakon flowcharts are based on a strict aerospace-industry standard that ensures your diagrams are clean, consistent, and free of visual noise. You do not need to know the standard—DrakonTech automatically ensures your diagrams adhere to it.

Intuitive and Efficient Editing: Forget traditional, cumbersome diagramming software. With DrakonTech, creating and modifying flowcharts is quick and easy. A few clicks are all it takes to build even the most advanced algorithms.

Advanced Features for Modern Development: DrakonTech's code generator supports:

- Modules
- Functions
- Object-oriented programming
- Finite automata (finite-state machines)
- Backward chaining

4.2. 4.2. Interface of DrakonTech

The interface of DrakonTech is designed with a focus on visual clarity and ease of use. Here is a general overview (Figure 4.2):

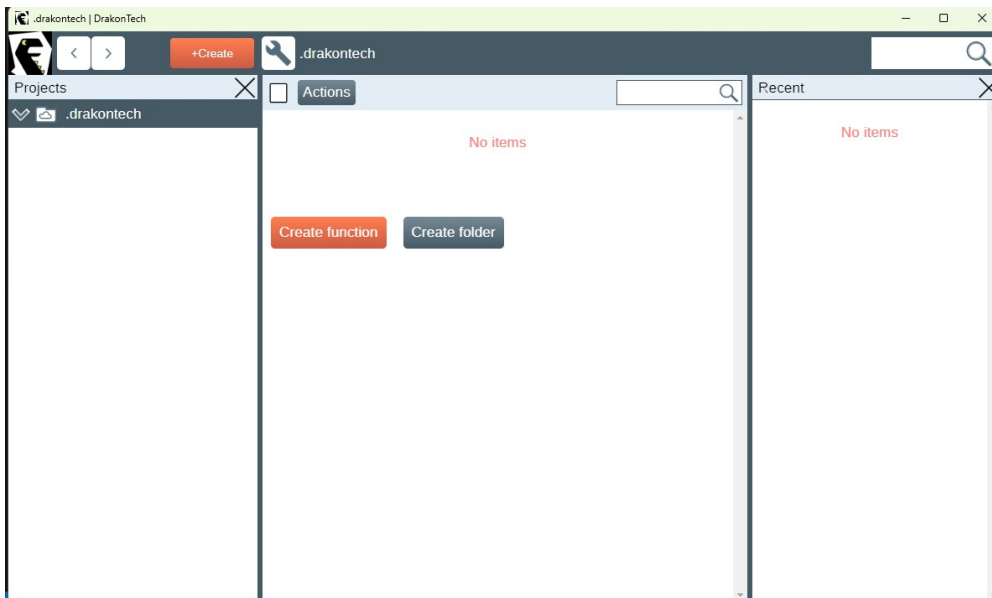


Figure 4.2. DrakonTech Interface

4.2.1. 4.2.1. Core Principles

- **Visual Flow:** The primary focus is on a graphical workspace where users construct algorithms using visual icons and connecting lines. This emphasizes the flow of control and data.
- **Simplified Icons:** Drakon-technology uses a limited set of standardized icons, each representing a specific action or decision. This keeps the diagrams clean and easy to interpret.
- **Structured Layout:** The language enforces a structured layout, which helps to maintain consistency and readability. This is a key feature that distinguishes Drakon-technology from free-form flowcharting tools.

4.2.2. 4.2.2. Typical Interface Elements

- **Canvas/Workspace:** This is the central area where users create and edit Drakon diagrams. Users can drag and drop icons, connect them with lines, and add text labels.
- **Icon Palette/Toolbar:** This provides access to the various Drakon icons, such as:
 - Start and end points
 - Action boxes
 - Decision diamonds
 - Loop structures
- **Properties/Inspector Panel:** This panel allows users to modify the properties of selected icons, such as:
 - Text labels
 - Conditions for decision points
 - Data values
- **Code Generation/Output Panel:** In DrakonTech, which is designed for code generation, this area displays the generated code in the target programming language (e.g., JavaScript).
- **Navigation and Zoom Controls:** These allow users to navigate large diagrams and zoom in or

out for detailed editing.

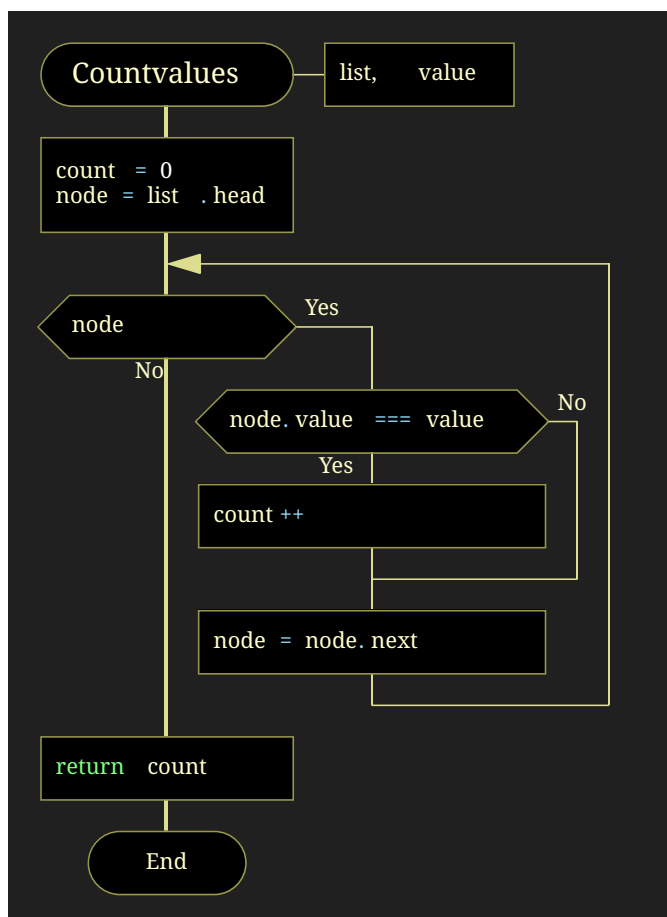
Key Characteristics:

- **Clarity:** The interface is designed to minimize visual clutter and maximize the clarity of the algorithm.
- **Efficiency:** The tools and layout are optimized for efficient diagram creation and editing.
- **Standardization:** The interface adheres to the Drakon language's standards, ensuring consistent and understandable diagrams.

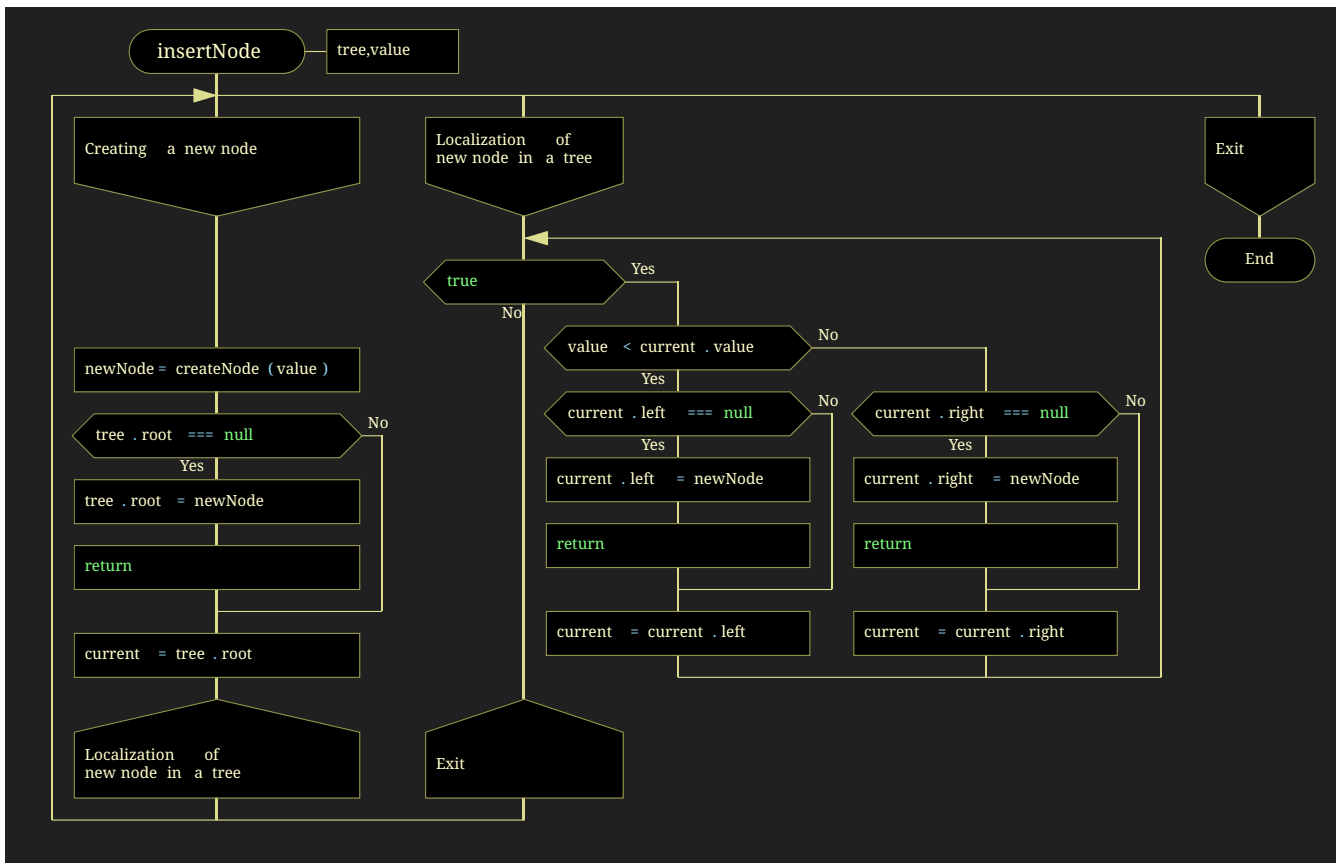
In summary, the DrakonTech interface prioritizes visual communication and structured algorithm representation, making it a valuable tool for both programming and conceptual understanding of complex logical workflows.

4.3. 4.3. Creating a Drakon-Diagram

The web application DrakonTech provides for the creation of two types of Drakon-diagrams; the choice is determined by the complexity of the algorithm (Figure 4.3):



a) Primitive



b) Silhouette

Figure 4.3. Two Types of Drakon-Diagrams

To create a Drakon-diagram, the user chooses the necessary icons, filling them with the corresponding operators of the JavaScript programming language (Figure 4.4).

Image	Meaning of the icon	Image	Meaning of the icon
	Question		Action
	Case		Choice
	Insertion		Loop
	Primitive		Sihlouette
	Simple input		Simple output
	Pause		

Figure 4.4. Set of DrakonTech Icons

Creating a Drakon-diagram must follow certain rules. The selected icon is moved by the mouse to

the vertical line (skewer in Drakon technology terms) in the right place. Creating a Drakon-diagram must follow certain rules:

- Creating a Drakon-diagram begins with a name that should reflect the purpose of the algorithm (function) and be located at the top.
- The diagram should have only one beginning and one end. The "end" graph is placed at the bottom of the diagram (Figure 4.5):

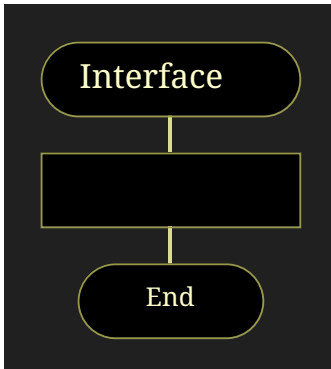


Figure 4.5. Drakon-Diagram Begin/End

The action flow represented in the diagram should only go from top to bottom. This approach is more convenient because in our cultural area texts are read in this way: from top to bottom. The top-down flow of actions is a fundamental principle of Drakon technology, which ensures that the algorithm's logic is easy to follow and understand.

- Avoid turning. The only case where lines have to change direction is where decisions are made. Turns are needed only when the algorithm requires making a choice between different actions. If there are no decisions, you need to go down. In any case, it is necessary to minimize the number of turns.
- Crossings of lines are absolutely not allowed. All attempts to apply crossings must be prevented. However, in case of an intersection, the editor will give an error.
- A top-down action prevents the use of arrows. The only exception is a while loop (Figure 4.6):

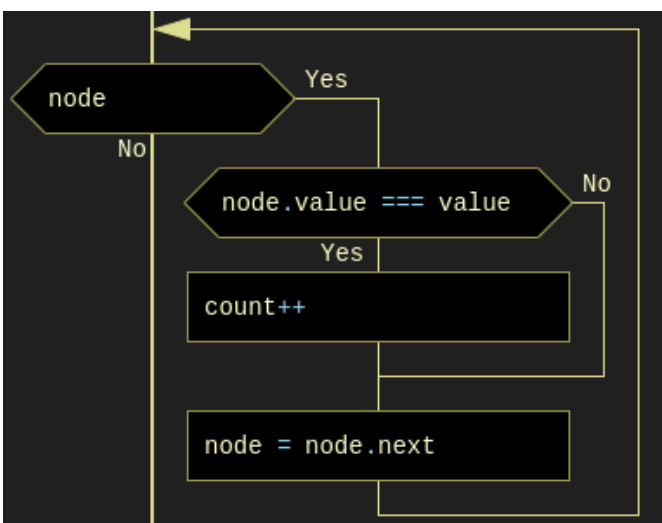


Figure 4.6. While-Loop Type Option

When creating a Drakon-diagram, only straight vertical and horizontal lines should be used, as

straight lines are easier to understand than curves.

The distance between icons is set automatically.

Branching is done only to the right. Branching to the left should be excluded. Following this rule significantly increases the predictability and uniformity of charts.

For convenience, the Drakon diagram can be moved around the work area using Shift + mouse wheel (horizontally) and using the mouse wheel (vertically).

The reader may get the impression that creating Drakon-diagrams is a very complex process. It should be recalled once again that in the DrakonTech editor everything is quite simple and clear. The editor will not allow the violation of the stated rules, which is checked by the verify option.

4.4. 4.4. Basic Structures of Drakon Algorithms

4.4.1. 4.4.1. Linear Structure

The linear structure of an algorithmic process implements operations that are performed sequentially in order of writing. A typical example of such a process is a standard three-step computing scheme:

a) Input data b) Formula calculation c) Output result

The graphical representation of the basic element of the linear structure in JavaScript is in the form of simple rectangles (Figure 4.7).

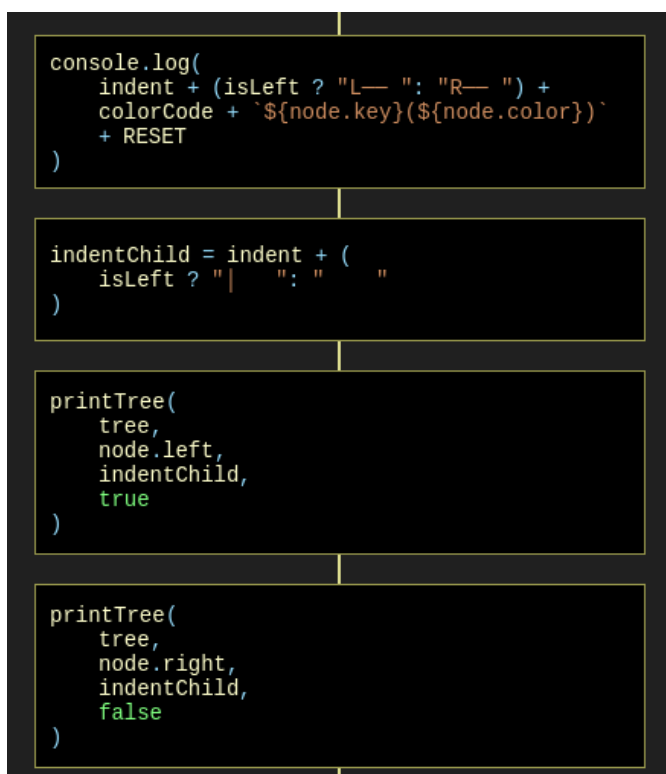


Figure 4.7. Linear Structure of a Drakon-Diagram

4.4.2. 4.4.2. Basic Structures of Branched Structure

The branched structure contains at least one condition check, which ensures the transition to one of the possible solutions. Each option leads to a common output; that is, the algorithm will continue regardless of which path is chosen. The branching structure exists in two main options:

a) if-else Construction

The Drakon snippet of the construction "if (condition), then (action) otherwise (action)" corresponds in programming languages to the if...else statement. An example of an if-else construction in a Drakon-diagram in an algorithm for finding minimum and maximum array values (Figure 4.8):

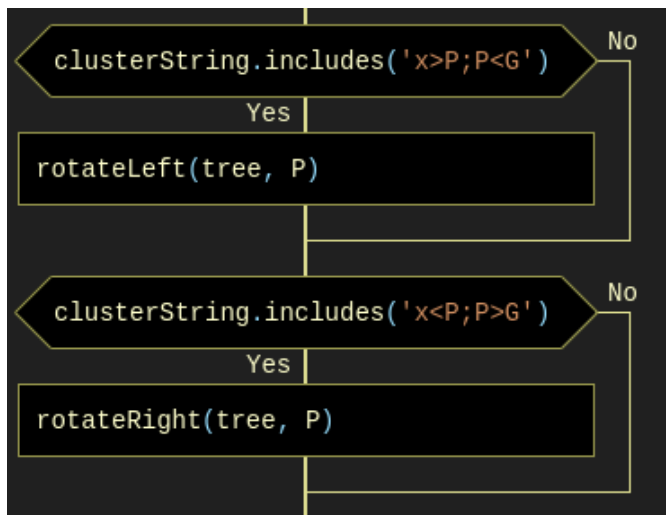


Figure 4.8. Diagram Snippet with if-else Construction

b) Select Construction (Switch Statement)

The values to which the expression in the "Choose" graph will be compared are covered in the "Option" graphs. If there is no text in the extreme right, it means "all other values". Below is an example of how to branch a sorting algorithm (merge sort) (Figure 4.9).

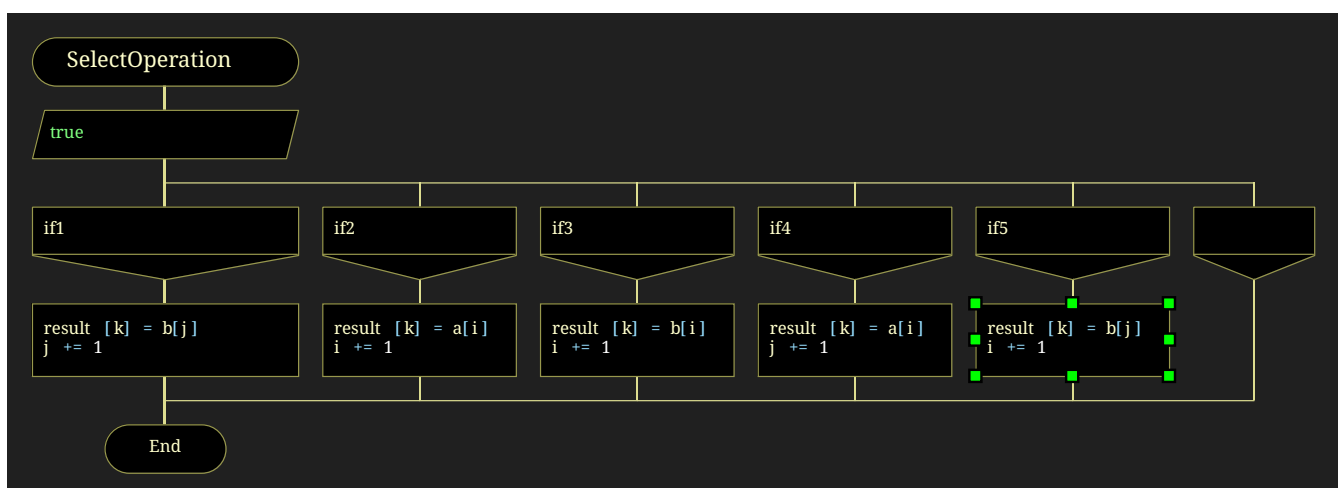


Figure 4.9. Branch Selection Snippet (Select Statement)

4.4.3. 4.4.3. Basic Loop Constructions

The loop structure involves repeating the same sequence of actions repeatedly. The number of

repetitions is determined by the input data or task conditions. Loop structures include, first of all, the construction "Loop-For" (C-style loop), "Loop for each", composite constructions "Loop-With-Arrow" ("Do-Check loop"), and "Do-Check-Do".

a) Loop-For Construction

"Loop-For" consists of three parts. In the first part, the loop initialization is fixed. In the second, the loop completion condition is checked. If true, the body operators of the loop are executed until the expression becomes false. If false, the loop ends and control is passed to the next operator. In the third part, the loop parameter increases. The snippet of the Drakon-diagram with the "Loop-For" design has the appearance shown in Figure 4.10:

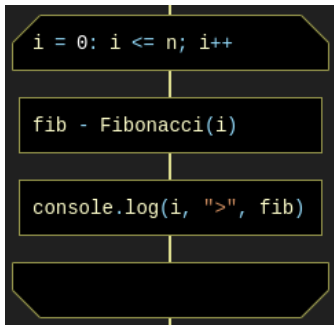


Figure 4.10. Drakon Construction "Loop-For"

b) Loop for each Construction

The "foreach" loop executes the operator or block of operators for each array element or data list (Figure 4.11):

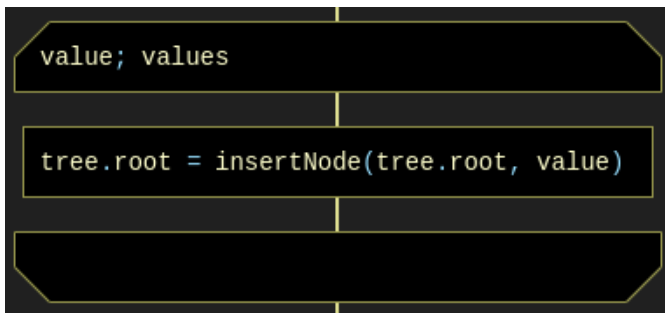


Figure 4.11. Drakon Construction "Loop for Each"

c) Loop With Arrow Construction

An example of a composite construction "Loop With Arrow" (Figure 4.12):

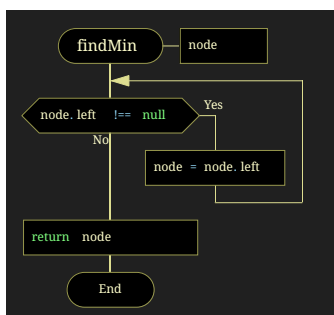


Figure 4.12. Drakon Construction "Do-Check Loop"

4.5. 4.5. From Drakon-Diagram to Program Code

Each Drakon-diagram corresponds to a program module. Figure 4.13 shows the structure of the array sorting algorithm consisting of a series of separate modules (Drakon-diagrams) and the main Drakon-diagram containing the main program's main design.

Let us create a Drakon-diagram using the example of a program for determining the digital root. The digital root of a non-negative integer is the single-digit value obtained by repeatedly summing the digits of the number. This process continues until the result is a single digit.

For example, let us take the number 12345:

- First sum = $1 + 2 + 3 + 4 + 5 = 15$
- Second sum = $1 + 5 = 6$
- Therefore, the digital root of 12345 is 6.

To solve the problem of finding the digital root of an integer, we will create four Drakon-diagrams: module, main, digRoot, and sumDigits (Figure 4.13).

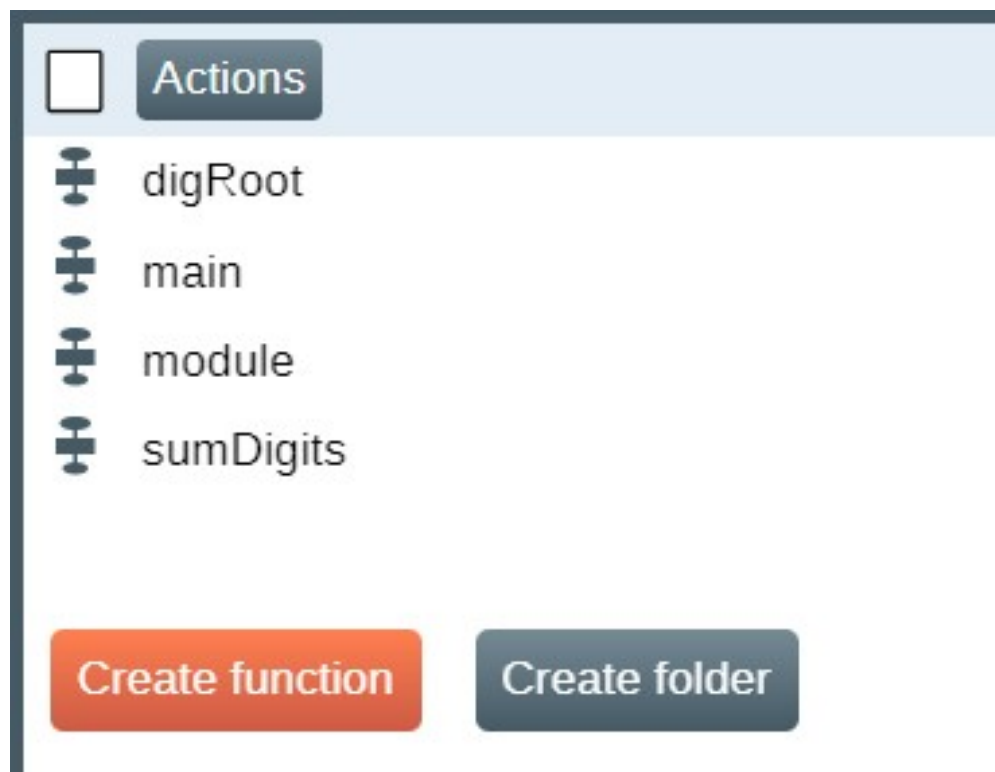


Figure 4.13. Modules: *module*, *main*, *digRoot*, and *sumDigits*

The function module contains a call to the `main()` function. The `main()` function contains a call to the `digroot` function with an integer specified (Figure 4.14):

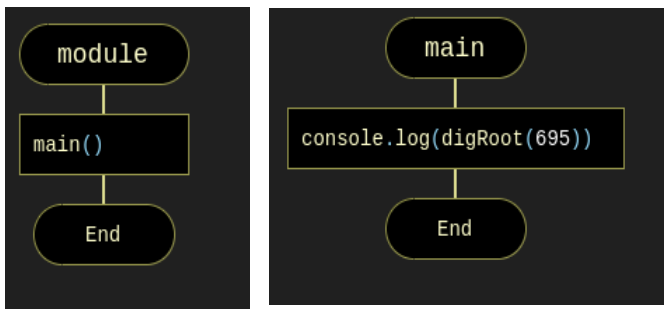


Figure 4.14. Function `module` and `main`

The purpose of the function `digRoot(n)` is to organize a call to the function `sumDigits(n)` that calculates the sum of the digits of a given number. In turn, the `sumDigits(n)` function splits the string "12345" into individual characters, after which the sum of the corresponding integers converted by the `parseInt(digit, 10)` function is calculated in a loop (Figure 4.15).

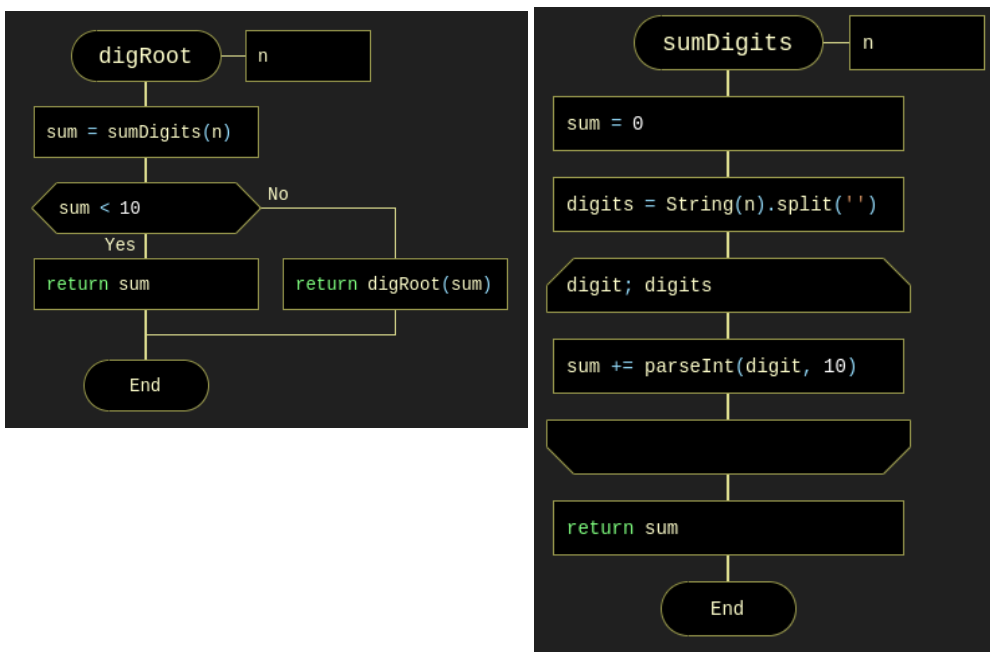


Figure 4.15. Functions `digRoot(n)` and `sumDigits(n)`

After creating the Drakon diagrams, the program code for determining the digital root in JavaScript is automatically generated by activating the "wrench" icon (Figure 4.16).

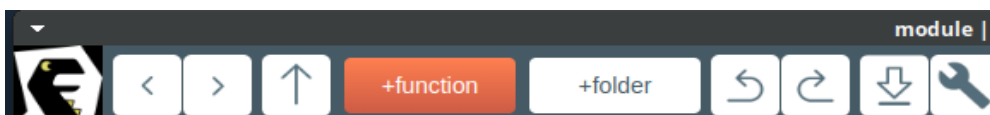


Figure 4.16. "Wrench Icon" for Generating Program Code

The result of automatic code generation in JavaScript:

```

main();
function digRoot(n) {
  var sum;
  sum = sumDigits(n);
  if (sum < 10) {
    return sum;
  } else {

```

```

return digRoot(sum);
}
}
function main() {
console.log(digRoot(695));
}
function sumDigits(n) {
var digit, digits, sum;
sum = 0;
digits = String(n).split('');
for (digit of digits) {
sum += parseInt(digit, 10);
}
return sum;
}
}

```

4.6. 4.6. Execution of the Generated Code

The generated code can be opened in one of the integrated development environments (IDE): Visual Studio Code, VIM, Eclipse, Atom, Sublime Text, and a number of others. This book uses the IDE Visual Studio Code (VSC), the description of which is not included here.

The execution of the generated program in the VS Code environment is carried out using the command: `node ProgramName.js`. Node.js is a JavaScript runtime environment that allows you to run JavaScript code outside of the browser. Node.js extends the capabilities of JavaScript, allowing it to create various tools and applications. The installer of Node.js is on the site nodejs.org. The generated program is launched via the terminal (Ctrl + `) or View → Terminal.

As part of the hybrid approach, debugging a program often has to be carried out in parallel in two environments: DrakonTech and Visual Studio Code. Naturally, in case of graphical syntax errors, Drakon-diagram code generation will not be performed.

The debugging process of the generated code is recommended to be carried out with the help of the appropriate programming environment toolkit. After debugging is complete, all corrections must be made to the corresponding graphs in the Drakon-diagram. In any case, it is always necessary to ensure that the contents of the diagrams correspond to the program code in the relevant IDE.

4.7. 4.7. Summary

This section explored the fundamentals and practical applications of the DRAKON visual algorithmic language and the DrakonTech environment within our hybrid development workflow.

By translating complex algorithmic logic into structured visual diagrams, DRAKON eliminates ambiguity and significantly reduces cognitive overhead during algorithm design. We examined the key visual primitives, syntax rules, and architectural patterns (such as primitive and silhouette algorithms) that guarantee deterministic flow control. Furthermore, we demonstrated how DrakonTech automates the generation of clean, executable JavaScript code, seamlessly bridging high-level algorithmic concepts with runtime execution in Node.js and Visual Studio Code.

Mastering this visual approach equips developers and students with a disciplined method for designing, verifying, and debugging algorithms visually before and alongside traditional code editing.

Chapter 5. Section 5. Complexity and Stability Analysis of Algorithms

This section introduces the analysis of algorithm complexity and stability as essential tools for evaluating algorithm quality. It explains that a correct algorithm must not only produce the expected result, but also terminate in a finite number of steps and use time and memory resources efficiently. The section presents theoretical complexity analysis through an abstract machine model, allowing algorithms to be compared independently of a specific computer or JavaScript runtime. Special attention is given to asymptotic notation, including Big-O, Omega, and Theta, and to common complexity classes such as constant, logarithmic, linear, and quadratic complexity. JavaScript examples are used to connect these mathematical ideas with practical code behavior, while DRAKON diagrams can help visualize control flow and repeated operations that influence complexity. The section also introduces space complexity and algorithm stability, preparing the reader to evaluate algorithms by correctness, performance, memory usage, and preservation of data order.

5.1. 5.1. Requirements for Algorithms

The previous sections focused on the description of the most common data structures, on the one hand as abstract entities, and on the other — their concrete implementation in the JavaScript programming language. This section will deal with the theoretical analysis of algorithms for processing data structures from the perspective of estimating the time spent on program code execution and computer memory usage.

In theory and in practice, data structure processing algorithms are defined as a set of sequentially performed, well-defined procedures that provide a final computational result based on a fixed and limited set of source data. Most data processing algorithms include basic data processing procedures such as searching, sorting, adding, updating, deleting, etc.

Foremost, it should be noted that a "good algorithm" must have two characteristics: a) correctness and b) finiteness. Correctness means that if an algorithm is designed to solve a particular problem, it must always produce the correct result for all source data, and no input data will yield the wrong result. The finiteness of an algorithm means that a finite number of computational operations must be performed to obtain the result — that is, the execution of the program code must stop at some point in time.

Once the algorithm is correct and finite, the most important characteristic of the algorithm is efficiency, which is achieved by meeting the following requirements:

- The algorithm should effectively use the resources available to the system.
- The computational time (time taken to generate output data corresponding to particular input data) should be minimal.
- The memory used by the algorithm should also be as small as possible.

In most computational tasks, it is necessary to strive for a compromise between computation time and occupied memory. In other words, when choosing a method for solving a computational problem, you need to decide on priorities: what is more important — computation time or amount of memory occupied. It may seem that this is not so important for modern computers. This becomes critical for large-scale problems — for example, in artificial intelligence or bioinformatics, such as genome decoding. Here, finding the right balance between time and memory is often the deciding factor. In this regard, we will define the term "algorithm complexity" as a measure of the amount of time and/or space required to solve a problem, depending on the size of the data.

Such analysis involves determining a function that links the length of the input data of the algorithm to the time it takes (its time complexity) or the number of storage locations it uses (space complexity). An algorithm is considered effective when the values of this function are either negligible or slowly increasing relative to the size of the input data.

5.2. Theoretical Analysis of Algorithm Complexity

Ultimately, the created algorithms must be implemented on some computing device using the generated program code. Therefore, one should distinguish between the complexity of the algorithm itself and the complexity of its computer implementation. The fastest algorithm implemented on a slow computing device may be less efficient than a less successful algorithm implemented on a computer with more processing power or a programming language capable of parallel computing.

Thus, when assessing the complexity of algorithms, they are evaluated on an abstract machine with random access to memory, which makes it possible to ignore the low-level parameters of the computing device (processor memory size, multitasking, etc.). The model of such a machine consists of memory and a processor, which work as follows:

- Memory consists of cells, each of which has an address and can store one data element.
- Each memory call takes one unit of time, regardless of the number of the addressable cell.
- The amount of memory is sufficient to execute any algorithm.
- The processor performs any elementary operation in one time step.
- Loops and functions are not considered elementary operations.

The complexity $T(n)$ of an algorithm associated with estimates of its complexity is determined by counting the number of operations performed. For example, consider the algorithm for finding the maximum element of an array:

```
const array = [11, 9, 17, 45, 411];
let maxNumber = array[0];
```

```
for (let index = 1; index < array.length; index++) {  
  if (array[index] > maxNumber) {  
    maxNumber = array[index];  
  }  
}  
console.log("The largest value of an array element =", maxNumber);
```

When implementing this algorithm, the following will be performed:

- (N - 1) operations of assigning a new value to the loop counter i;
- (N - 1) operations of comparing the counter with the value N;
- (N - 1) operations of comparing an array element with the value of maxNumber;
- (From 1 to N) operations of assigning a value to the variable maxNumber.

It is obvious that such a calculation for determining the complexity of an algorithm and estimating its complexity is individual and depends on the size of the input data. Therefore, it was quite natural to propose a qualitative assessment of complexity depending on the "quality" of the initial data: the best case, the worst case, and the average case.

5.3. 5.3. Qualitative Assessment of Complexity

Asymptotic notations such as Omega notation (Ω), Theta notation (Θ), and Big-O notation (O) that reflect the three time bounds of an algorithm (lower, middle, and upper) are used to quantify the complexity of algorithms.

- Omega notation (Ω) represents the lower bound of the algorithm's running time, which characterizes the best algorithm complexity in terms of achieving efficiency. For any value of n of the source data, the minimum time required by the algorithm is set to $\Omega(f(n))$.
- Big-O notation (O) represents the upper bound on the execution time of the algorithm, which characterizes the worst complexity of the algorithm in terms of efficiency. In most practical cases, this notation is used to evaluate the effectiveness of algorithms, since it is the worst-case scenario of the computational process that the software developer must evaluate.
- Theta notation (Θ) represents both an upper and lower bound on the execution time of algorithms and is used to analyze their average complexity.

Consider in more detail the Big-O notation that determines the upper bound of any algorithm — that is, the algorithm cannot take longer than the upper bound.

5.3.1. Constant Time Complexity — $O(1)$

An algorithm with constant time complexity requires the same amount of time regardless of its input data. For example, given two numbers, you need to calculate their sum.

Logarithmic Time Complexity — $O(\log n)$

When the time taken by an algorithm is proportional to the logarithm of the input size n, it is said to have logarithmic time complexity. An example is the simple calculation of the number of print

operations of the index in a loop:

```
let n = 16;
for (let i = 1; i < n; i *= 2) {
  console.log("i = ", i);
}
```

Recall that $\log_2 16 = \log_2 2^4 = 4 \log_2 2 = 4$, since $\log_2 2 = 1$. Algorithms with logarithmic complexity are considered highly efficient because the ratio of execution time to the size of the data array decreases as the data size increases. Algorithms that work in logarithmic time are usually found in binary tree operations or binary search operations. Such algorithms will be discussed in the next section.

5.3.2. Linear Time Complexity — $O(n)$

Linear complexity characterizes algorithms whose execution time linearly depends on the size of the input data (n). The simplest example is the problem of computing the sum of n numbers:

```
let n = 16;
let sum = 0;

for (let i = 0; i < n; i++) {
  sum += i;
}
console.log("sum =", sum);
```

Typical examples of linear time complexity are algorithms associated with array sorting, which will be shown in the next section. It is important to note that linear complexity algorithms do not require additional memory.

5.3.3. Quadratic Time Complexity — $O(n^2)$

In quadratic complexity algorithms, the number of operations is quadratic depending on the size of the input data. As an example, we can give the algorithm of multiplication of two numbers in the interval $[1:n; 1:m]$:

```
let n = 3;
let m = 5;
let product = 1;

for (let i = 1; i < n; i++) {
  for (let j = 1; j < m; j++) {
    product = product * i * j;
  }
}
console.log("product =", product);
```


Comparison of Complexities

The comparison of the discussed complexities is presented in the form of a table.

Table 5.1. Comparison of General Complexities

n	Constant $O(1)$	Logarithmic $O(\log n)$	Linear $O(n)$	Linear Logarithmic $O(n \log n)$	Quadratic $O(n^2)$
1	1	1	1	1	1
2	1	1	2	2	4
4	1	2	4	8	16
8	1	3	8	24	64
16	1	4	16	64	256

Or in graphic form (Figure 5.1):

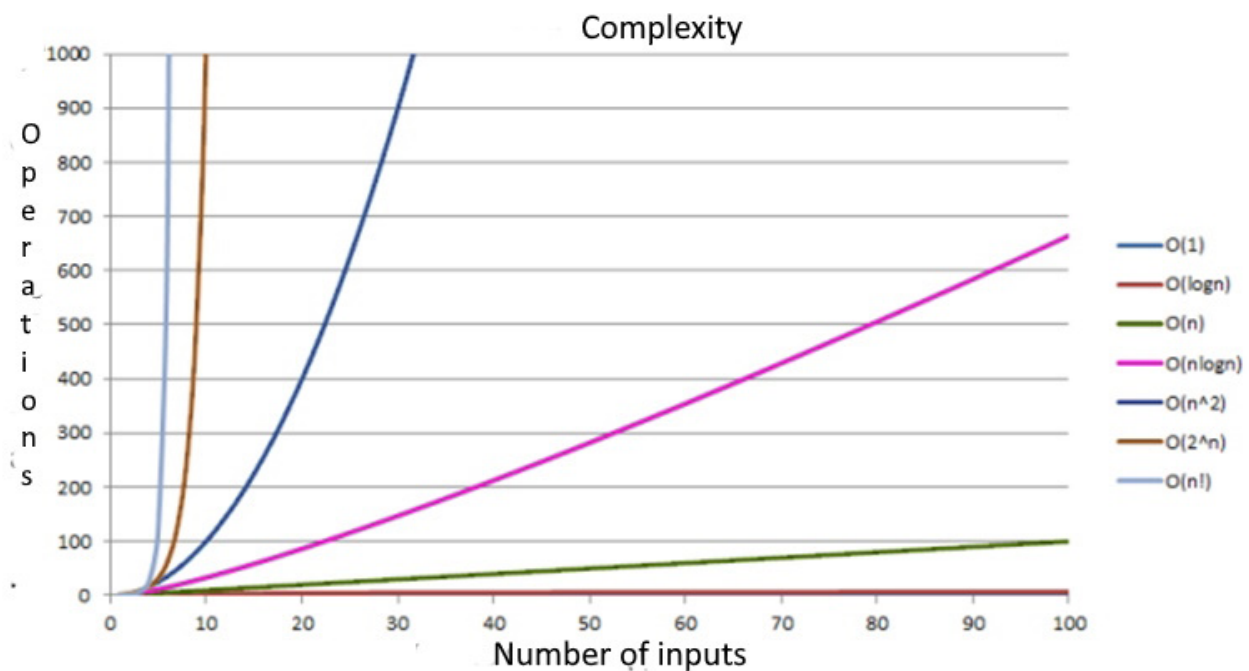


Figure 5.1. Comparison of General Complexities

As shown in this table and graph, as the complexity of the function increases, the number of computations or the time required to perform the function can increase significantly. Next, the algorithms will be described with estimates of their complexity.

5.4. Space Complexity

Another characteristic of the computational complexity of algorithms is space complexity, estimated by the amount of memory required by the algorithm in its life cycle. This space consists of a fixed part and a variable part.

- The fixed part is the space required to store simple variables and constants, as well as the program size, which is independent of the complexity of the task.
- The variable part is the space required by variables whose size depends entirely on the size of the task. For example, recursion stack space, dynamic memory allocation, etc.

Space complexity is evaluated as $O(1)$ if only the input data set is used. If the problem requires auxiliary memory of the same size as the input memory (memory to store the original array), then the space complexity is evaluated as $O(n)$. In the case of a two-dimensional array or matrix, space complexity may be $O(n^2)$.

5.5.5. Algorithm Stability

It is especially important to know whether the algorithm used is stable, especially in situations where your data already has some order that you would like to preserve when sorting with a different key. For example, a spreadsheet has rows containing student data sorted by name by default. You would also like to sort it by class while maintaining the orderly order of names.

On the other hand, the stability of sorting does not matter when the sorting keys of objects in a collection are the objects themselves — arrays of integers or strings, for example — because we cannot determine the difference between duplicate keys.

5.6. Summary

This section established the theoretical and analytical foundations for evaluating algorithm performance and correctness, independently of specific hardware or JavaScript execution environments.

We explored the core principles of algorithmic efficiency:

- * **Time and Space Complexity:** Evaluating how execution time and auxiliary memory usage scale relative to the input size n .
- * **Asymptotic Notations:** Utilizing $\mathcal{O}$ (upper bound), Ω (lower bound), and Θ (tight bound) to classify algorithm growth rates (e.g., constant, logarithmic, linear, quadratic).
- * **Abstract Machine Models:** Analyzing algorithms via the RAM model to decouple performance metrics from compiler optimizations and environment overhead.
- * **Algorithm Stability:** Assessing whether an algorithm preserves the relative order of elements with equal keys, a critical property in multi-key sorting scenarios.

Understanding these analytical tools enables rigorous comparison between competing algorithmic solutions, ensuring optimal resource utilization in practical applications.

Chapter 6. Section 6. Basic Search Algorithms

This section examines the primary algorithms used for searching data structures. To enhance the understanding of the practical applications of these algorithms, DRAKON-diagrams are provided along with assessments of their time and space complexity.

Searching is one of the most important algorithms for processing data structures. Search algorithms are designed to check the presence of an element or extract an element from any data structure in which it is stored. More strictly, the search problem can be formulated as follows: find one or more elements in a set, where the elements sought must have a certain property. This property can be absolute or relative. A relative property characterizes an element with respect to other elements: for example, a minimum element in a set of numbers.

There are many search algorithms. Their complexity varies from simple sequential search algorithms to highly efficient binary search algorithms, which, however, require sorted datasets. Of particular note are algorithms based on presenting the core set of data in a different, more searchable form, which are used in real-world applications for processing data sets in huge databases.

There is no single algorithm to solve all search problems. Some algorithms run faster than others but require additional RAM to run. Others run very quickly but can only be used for pre-sorted arrays. At the same time, the analysis of search algorithms is somewhat different from sorting algorithms. In particular, there is no stability problem for them. However, there may be situations that require the introduction of new criteria of complexity and its assessment. In general, all algorithms are reduced to the following steps:

- Establishing the property of the elements of the source set; in most cases, these are the values of the elements.
- Matching the value of the element with the reference property (for absolute properties) or comparing the properties of two elements (for relative properties).
- Traversing the elements of the set.

Fundamentally, search algorithms are categorized by their underlying search paradigms and strategies. Based on the type of search operation, these algorithms are usually classified into two categories:

- Sequential Search: The list or array is traversed sequentially and each element is checked.
- Interval Search: These algorithms are specifically designed for searching in sorted data structures. These types of search algorithms are much more efficient than linear search because they repeatedly target the center of the search structure and divide the search space in half.

6.1. 6.1. Linear Data Searching

6.1.1. a) Linear Search of Unsorted Data

In the case of a linear search for an element in an unordered dataset, for example, in an array or in a slice, the simplest algorithm is to examine all elements until the desired value is found (Figure 6.1).

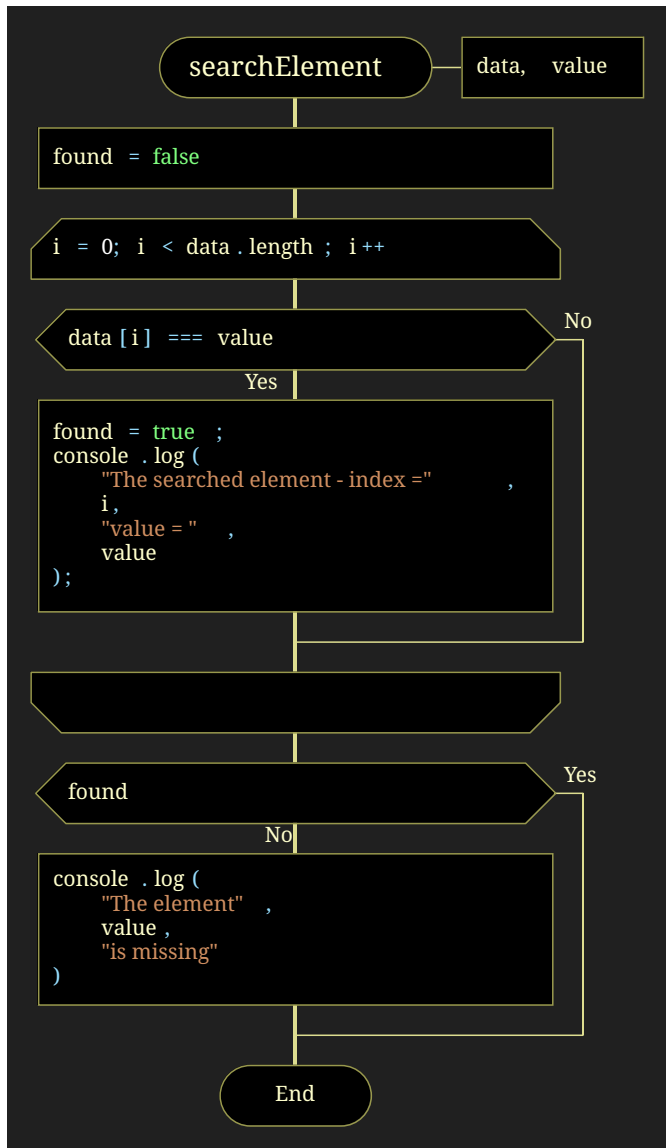


Figure 6.1. DRAKON-diagram of the Linear Search Algorithm

This algorithm is not very efficient, but it works on arbitrary collections. Time complexity: $O(n)$. In the worst case (the desired element is in the last position), to find the element you need to traverse all elements. Here n is the size of the slice. Space complexity: $O(1)$. No additional memory is required to accommodate the slice. Another worst-case scenario is the absence of the required element.

6.1.2. b) Linear Search of Sorted Data

If the elements of the dataset are sorted in ascending or descending order, finding the desired element is much more efficient than in an unordered linear search. In many cases, you do not have

to traverse the whole list. For example, when traversing an ascending sorted list, if an element with a higher value than the search key is encountered, the search is stopped. This approach saves time and increases productivity. Figure 6.2 shows the DRAKON-diagram of this algorithm.

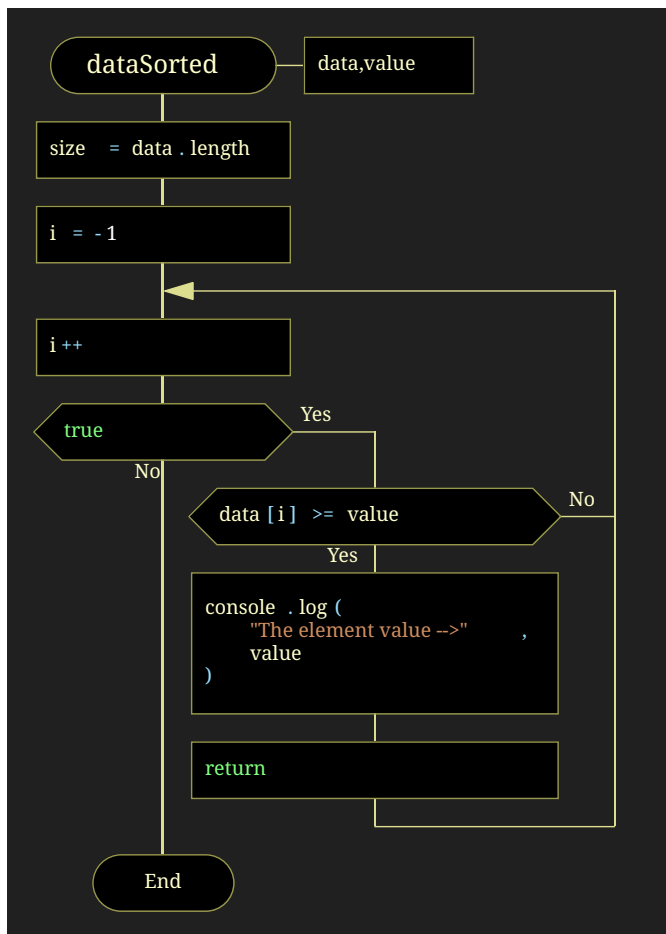


Figure 6.2. DRAKON-diagram of the Search in a Sorted Array

6.2. 6.2. Binary Search for Data in a Sorted Structure

Binary search is performed as follows:

- Determine the value of the element in the middle of the data structure. The resulting value is compared to the value being searched for.
- If the search value is less than the middle value, the search is carried out in the first half of the elements; otherwise, in the second half.
- The search continues by determining the value of the middle element in the selected half and comparing it to the key.
- The process continues until an element with the search value is found or the search interval is empty.

The DRAKON-diagram of the binary search algorithm is represented in Figure 6.3 (the main() module is similar to the previous algorithm):

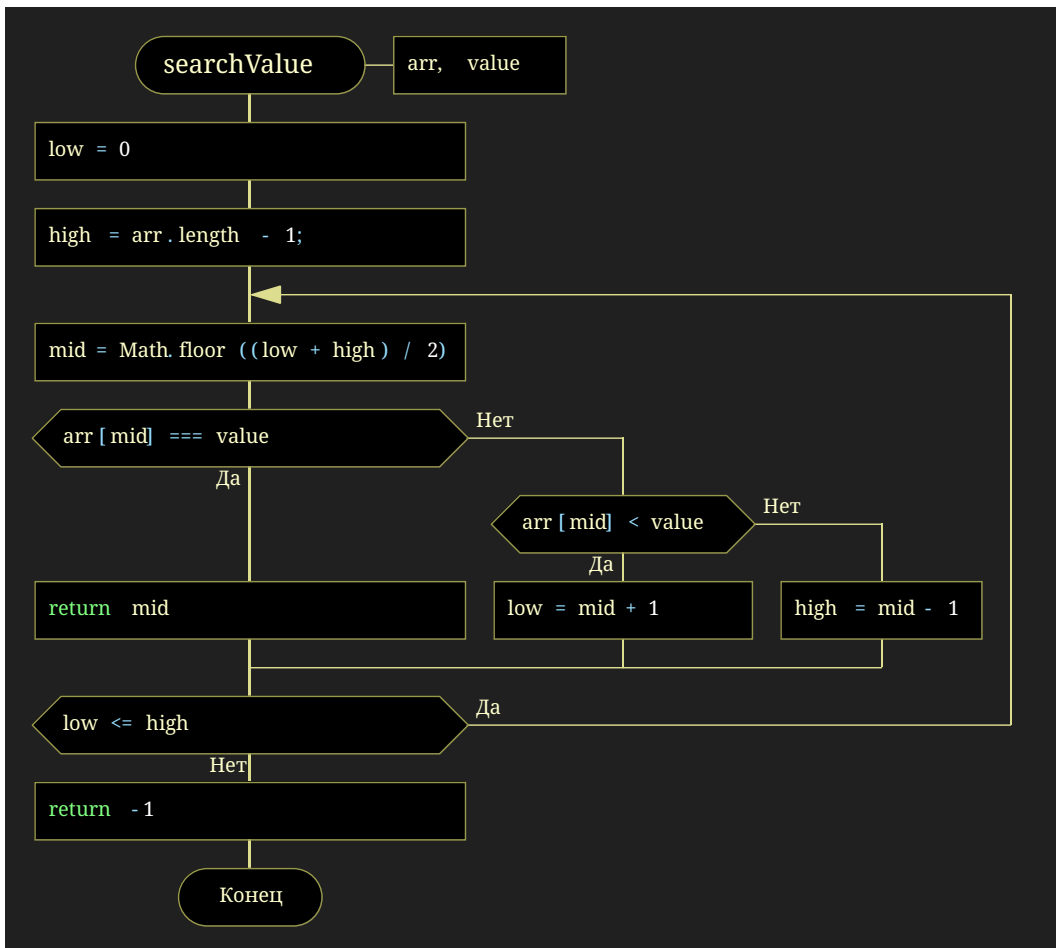
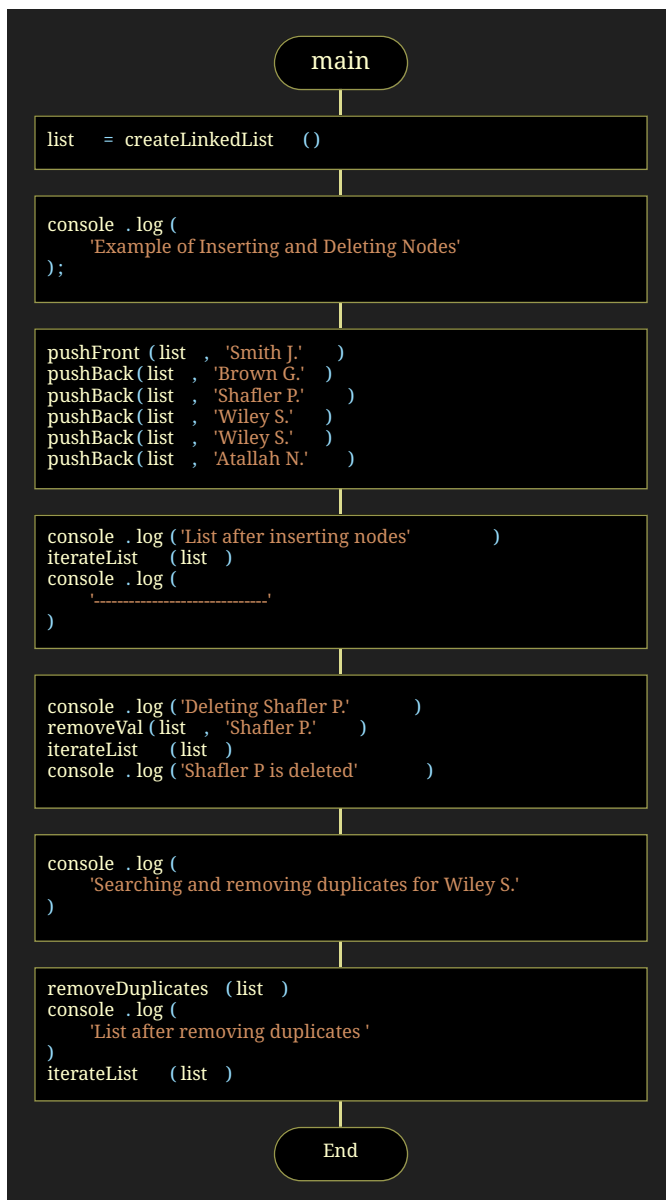


Figure 6.3. DRAGON-diagram of the Binary Search Algorithm

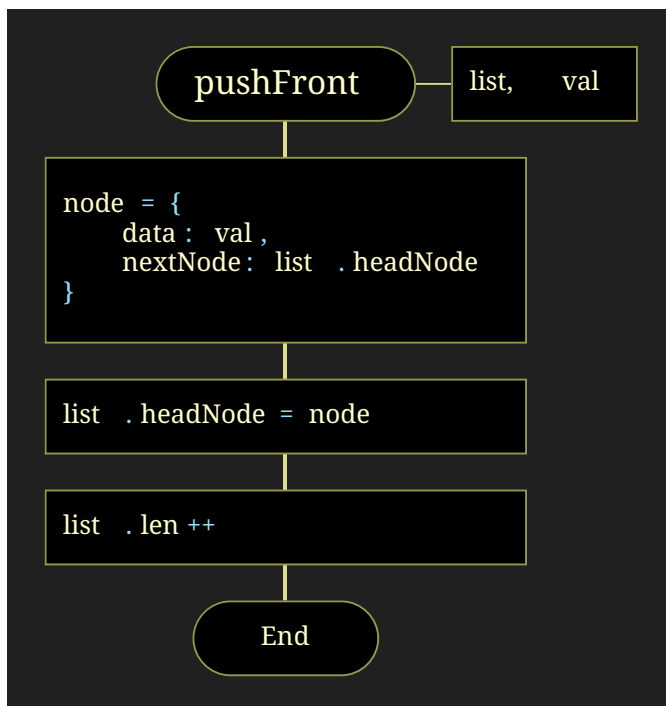
Time complexity of the binary search algorithm belongs to class $O(\log n)$. This means that the asymptotic increase in the time taken by the function to process an input set of size n will not exceed $\log n$. Space complexity: $O(1)$ for the iterative implementation.

6.3. 6.3. Searching and Related Operations in a Singly Linked List

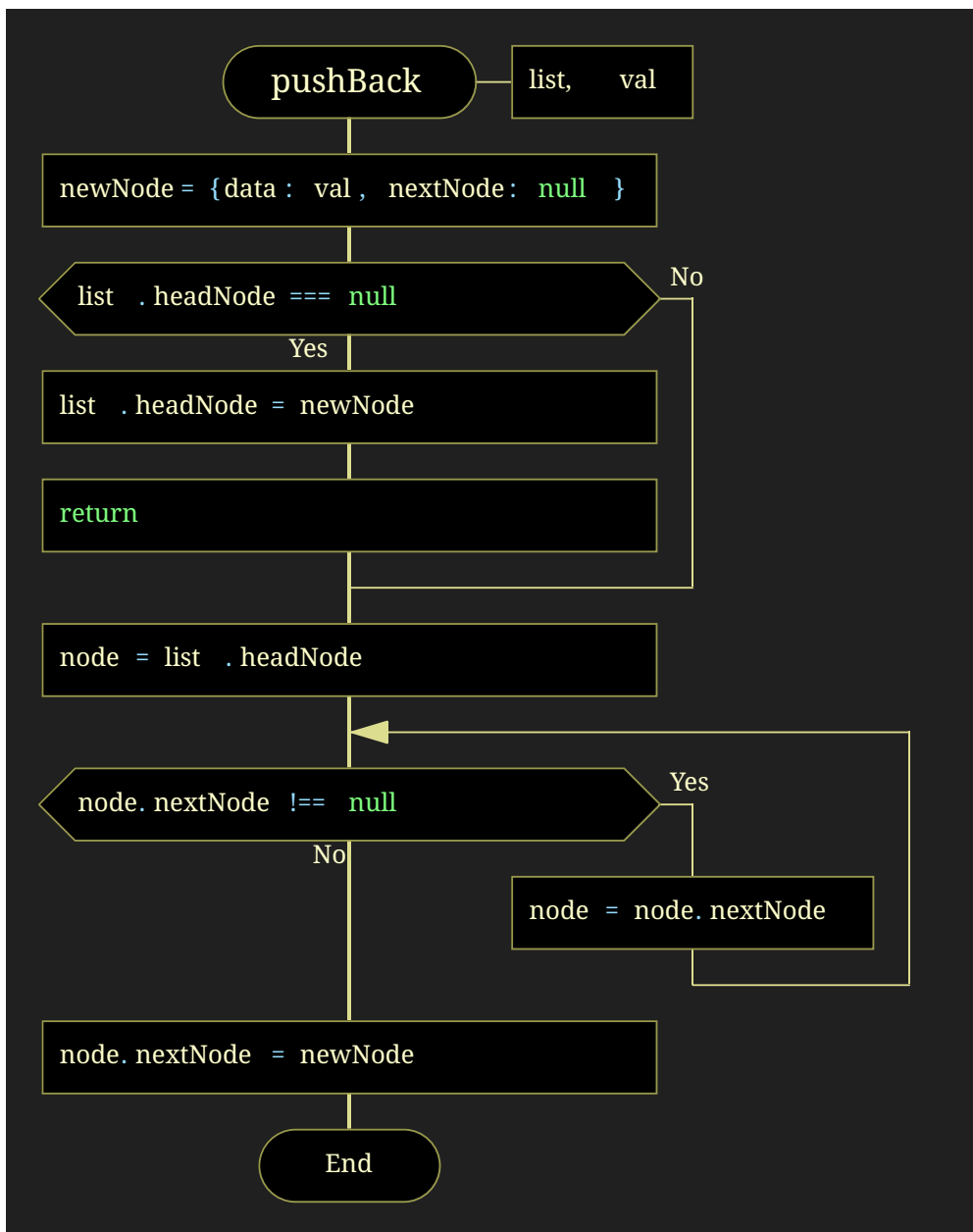
There are three possibilities for a singly linked list. First, the desired value is missing from the list. Second, the desired value appears once. Third, the desired value appears multiple times. You can also set the task of removing duplicates — nodes that are redundant. To solve these problems, it is necessary to create a singly linked list whose items contain values "Smith J.", "Brown G.", "Shafler P.", "Wiley S.", "Wiley S.", "Atallah N.". In this list you should delete the entry "Brown G." and then add a new entry "Singer B." placing it after the entry "Wiley S.". After that, you should delete duplicates of the entry "Wiley S." leaving only one. The linked list operations below demonstrate search as a subroutine in deletion, insertion and duplicate removal — all of which rely on locating an element by value. The corresponding DRAGON-diagrams are presented in Figure 6.4.



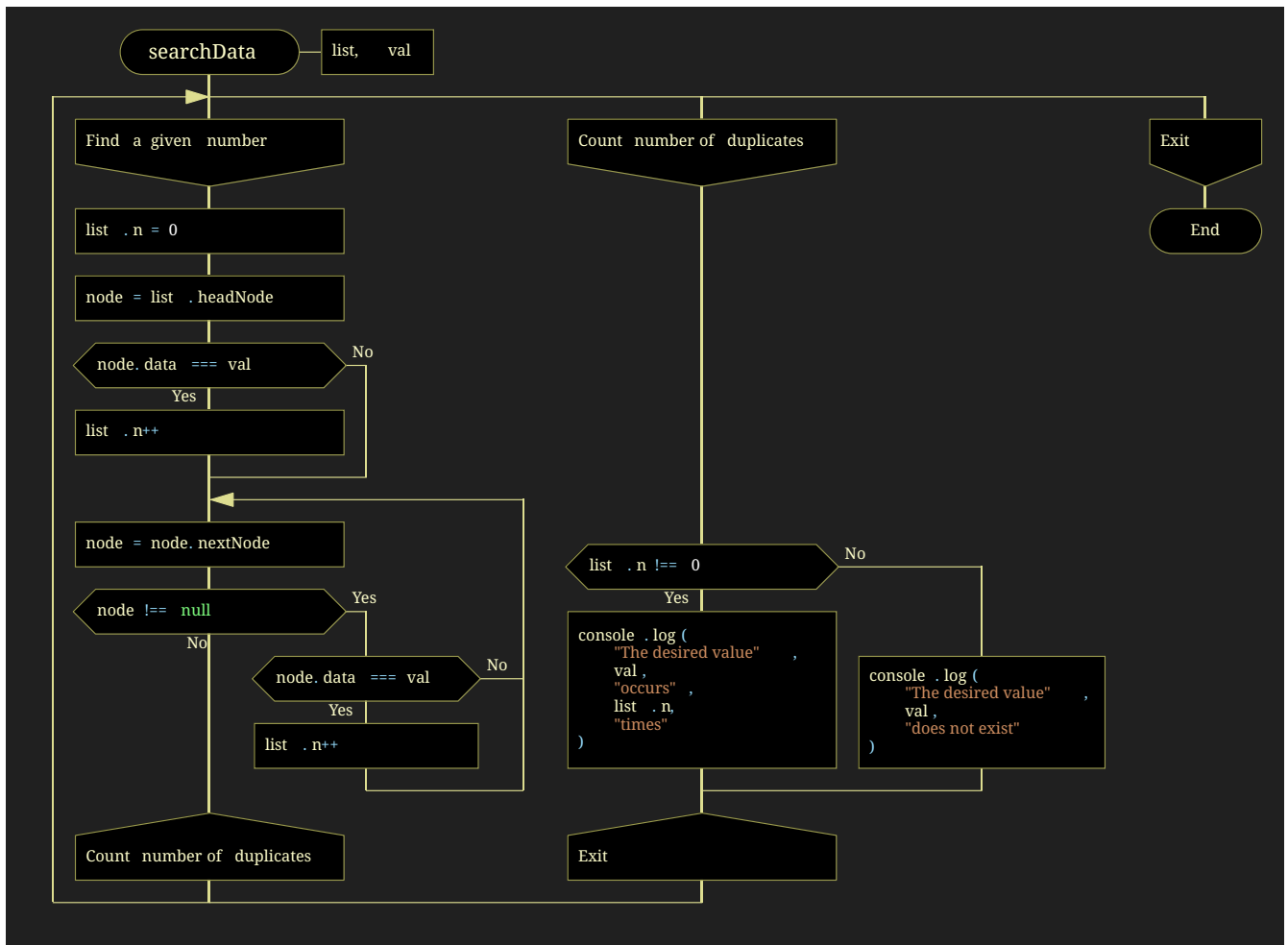
a) Function main() as the Entry Point for Linked List Operations



b) Functions for Pushing New Items (pushFront(list, val))



c) Functions for Pushing New Items *pushBack(list, val)*



d) Function for Searching (searchData(val))

Smith J.
Brown G.
Shafler P.
Wiley S.
Wiley S.
Atallah N.

Deleting 'Shafler P.'
'Shafler P.' is deleted
List after inserting nodes
Smith J.
Brown G.
Shafler P.
Wiley S.
Wiley S.
Atallah N.

Deleting 'Shafler P.'
'Shafler P.' is deleted
List after deletion
Smith J.
Brown G.
Wiley S.
Wiley S.
Atallah N.

Searching and removing duplicates for 'Wiley S.'
The desired value Wiley S. occurs 2 times
List after removing duplicates
Smith J.
Brown G.
Wiley S.
Atallah N.

List after inserting nodes
Smith J.
Brown G.
Shafler P.
Wiley S.
Wiley S.
Atallah N.

Deleting 'Shafler P.'
Shafler P. is deleted
List after deletion
Smith J.
Brown G.
Wiley S.
Wiley S.
Atallah N.

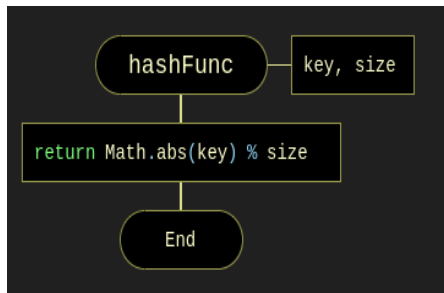
```
Searching and removing duplicates for `Wiley S.`  
The desired value Wiley S. occurs 2 times  
List after removing duplicates  
Smith J.  
Brown G.  
Wiley S.  
Atallah N.
```

6.4. 6.4. Hashing

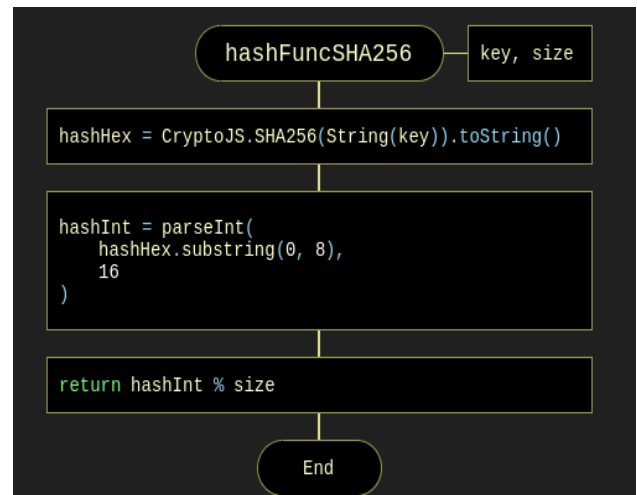
The time to find an item in a data set often depends on how many comparisons are made. To improve search efficiency, the number of key comparisons must be minimized. One way is to map keys to a smaller range of indices – this process is called hashing, and the resulting structure is a hash table.

In terms of abstract data type (ADT) theory, a hash table implements an associative array. It stores key-value pairs and supports three basic operations: insert (add a pair), lookup (find a value by key), and delete (remove a pair).

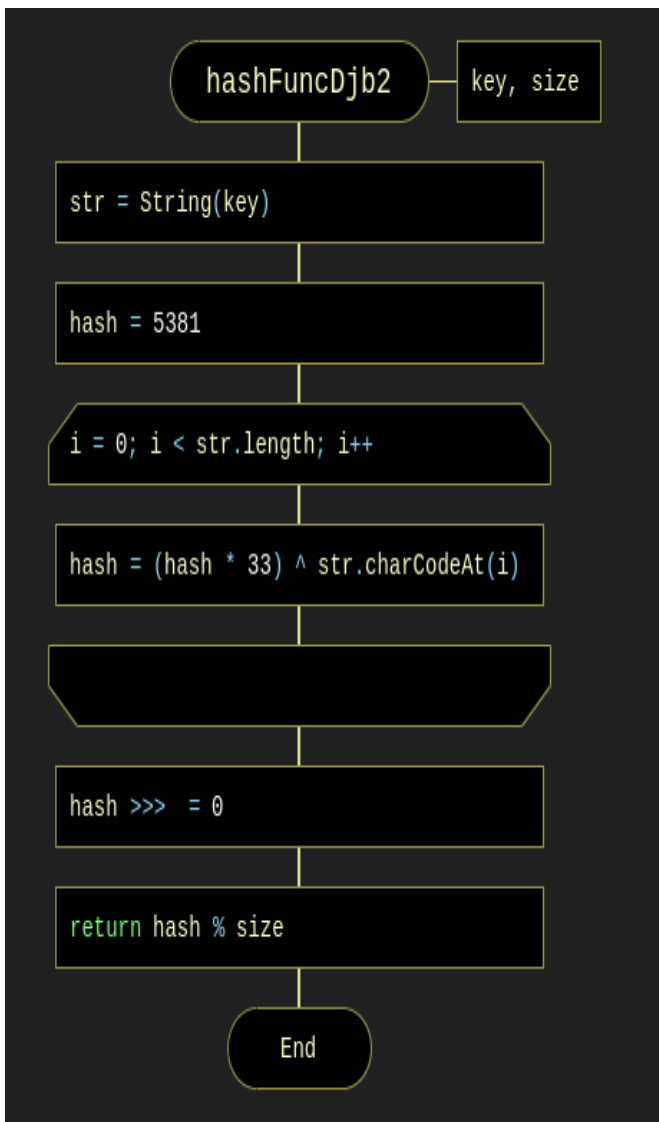
From a programming perspective, a hash table consists of buckets — groups of elements mapped to the same hash index. The hash function transforms a given key into an integer index (the bucket number). Many methods exist to construct a hash function, some of which are shown in Figure 6.5.



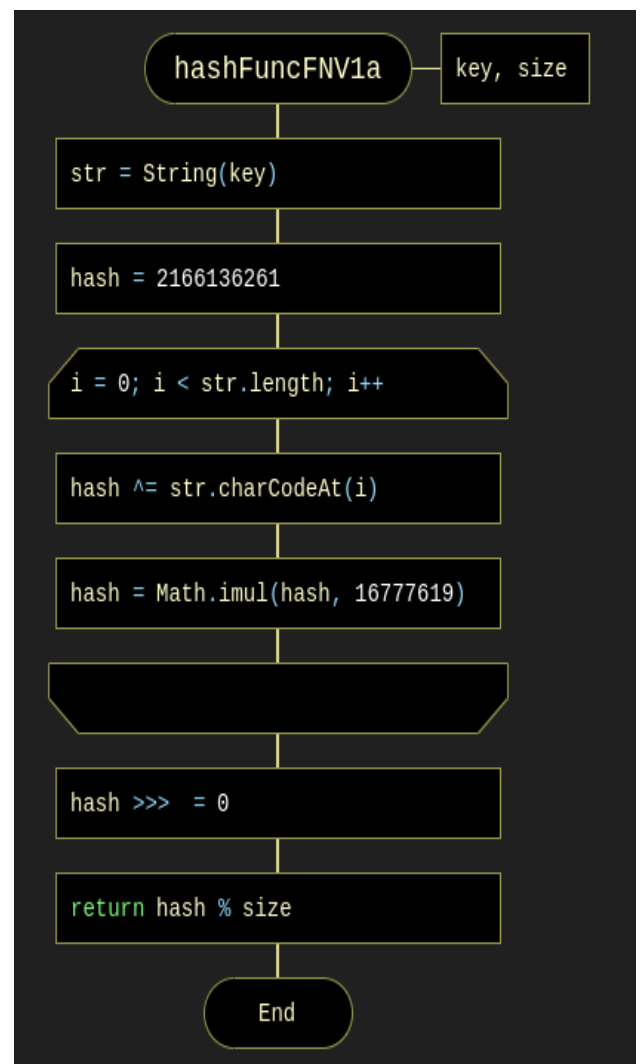
a)



b)



c)



d)

Figure 6.5. DRAKON-diagram of Hash Function Construction Methods

The simplest is the division method, where the hash is the remainder of the key divided by the number of buckets:

$$h(\text{key}) = \text{key} \bmod m \text{ (assuming the key is an integer).}$$

In JavaScript, for numeric keys, this becomes $h = x \% m$.

6.4.1. 6.4.1. Creating a Hash Table and Chain Nodes

To implement a hash table using chaining, we need two helper structures: the table itself (an array of buckets) and a node for the linked list.

The function `createHashTable(size)` initializes a new hash table instance. It takes one argument – the desired number of buckets. Internally, it does the following (Figure 6.6):

- Creates an array table of the given length.
- Fills each element of the array with null. This means all buckets are initially empty – no key has been added yet.
- Returns an object containing table and size. This object will be used by all subsequent operations (insert, traverse, lookup, remove).

In practice, the table object often also includes a count field (current number of elements) and a loadFactor (threshold for resizing). However, we start with a minimal version for simplicity.

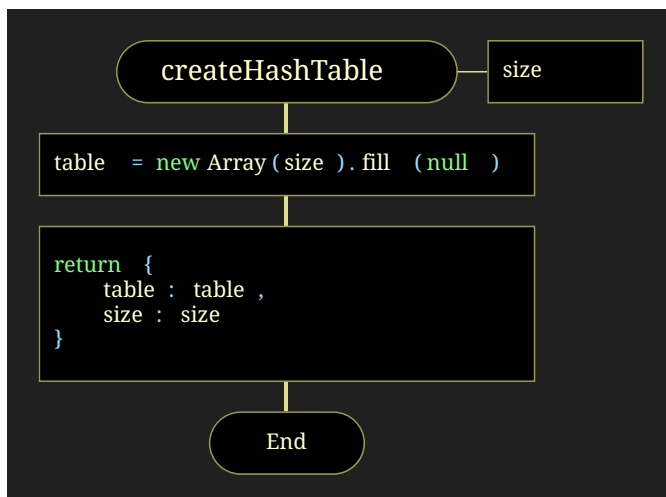


Figure 6.6. Function for Creating a Hash Table (`createHashTable`)

Because collisions are resolved with linked lists, each element in a bucket must store not only a key-value pair but also a reference to the next element in the same chain.

The function `createNode(key, value, next = null)` creates such a node (Figure 6.7). It:

- Accepts a key, a value, and an optional reference to the next node (default null).
- For debugging, prints a message to the console indicating that a node has been created. In production code this log is usually removed, but in educational examples it helps trace the algorithm's behavior.
- Returns an object with three fields: key, value, and next.

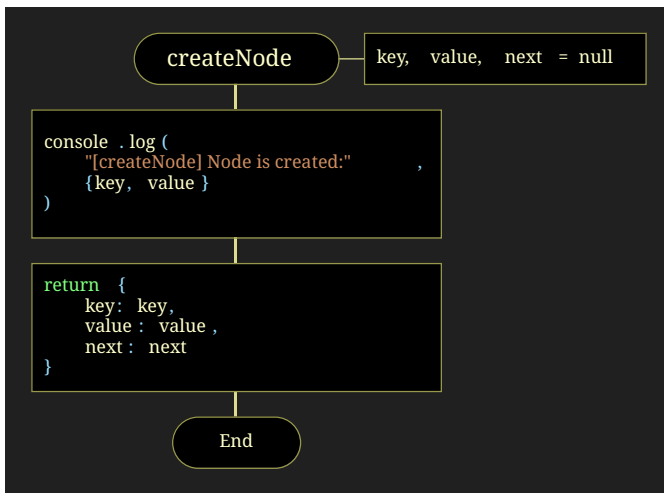


Figure 6.7. Function for Creating a Node (createNode)

Note that the next reference allows us to build singly linked lists inside a bucket. If only one element exists in a bucket, its next is null. When we add a new element that hashes to the same index, we place it at the beginning (or end – depending on the strategy; our algorithm will add to the head for simplicity).

As a result, this hash table would have as many singly linked lists (buckets) as specified by the Size constant. In the example below, the number of slots is 15.

Consider constructing a hash table of size $m = 15$ for a collection of integers from 0 to 119. The hash table slots are initially empty:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----

A hash function that establishes a relationship between an element and its corresponding slot must accept any element from the dataset (ranging from 0 to 119) and return an integer that corresponds to a slot number between 0 and 14. The algorithm implementing the remainder method sequentially processes each element from the original set, dividing it by 15 and returning the remainder as the hash value. This can be expressed as $h(\text{item}) = \text{item} \bmod 15$. For instance, the hash code for the element 119 is calculated as $119 \bmod 15 = 14$ (since $119 - 15 \times 7 = 14$), thus placing the value 119 into the corresponding slot.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
-	-	-	-	-	-	-	-	-	-	-	-	-	-	119

The algorithm then identifies slots for other elements, gradually filling them. When the algorithm encounters element 104 in the loop, the remainder of the division is $104 \% 15 = 14$, so this element will also be included in the 14th slot. In this way, each slot will accumulate corresponding elements with the same hash index. For example, for a hash index of 8, the slot will consist of these elements: 113, 98, 83, 68, 53, 38, 23, 8. The entire hash table will be as follows (Table 6.1):

Table 6.1: Hash Table of 15 Slots

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
105	106	107	108	109	110	111	112	113	114	115	116	117	118	119

90	91	92	93	94	95	96	97	98	99	100	101	102	103	104
75	76	77	78	79	80	81	82	83	84	85	86	87	88	89
60	61	62	63	64	65	66	67	68	69	70	71	72	73	74
45	46	47	48	49	50	51	52	53	54	55	56	57	58	59
30	31	32	33	34	35	36	37	38	39	40	41	42	43	44
15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Thus, the remainder method converts a collection of 120 integers into a hash table of 15 slots. The search for an element is now greatly accelerated as it takes two steps: first, the hash function $h = (x \% m)$ computes the hash index, and then the search is performed within a bucket (exactly 8 elements in each bucket for this example).

The implementation of the basic hash table functions using the hash function is as follows:

- Create a HashTable structure of size m to store objects.
- Compute the object's hash code by passing it through the hash function.
- Obtain the bucket hash indices where the objects will be stored.
- Store these objects in the designated bucket.

The general description of algorithms for working with hash tables is presented as a table of basic parameters of applied functions (Table 6.2) and corresponding DRAKON diagrams (Figure 6.8a - 6.8f).

Table 6.2: Functions Overview (Chaining Implementation)

Function	Purpose	Parameters	Return value	Main steps	Notes / Complexity
<code>hashGet(hash, key)</code> (Figure 6.8a)	Find a value by key	<code>hash</code> – table object, <code>key</code> – search key	Value or <code>undefined</code> if not found	1. Compute index via <code>hashFunc</code>. 2. Traverse the chain at that bucket. 3. If <code>node.key === key</code> found, return <code>node.value</code>. 4. Otherwise return <code>undefined</code>.	Average-case (expected) $O(1)$ complexity.

Function	Purpose	Parameters	Return value	Main steps	Notes / Complexity
<code>hashInsert(hash, key, value)</code> (Figure 6.8b)	Insert or update a key-value pair	<code>hash, key, value</code>	Bucket index where insertion/update occurred	1. Compute index. 2. Traverse the chain: if key exists, update its value and return index. 3. Otherwise, create a new node and insert at the head: <code>hash.table[index] = createNode(key, value, hash.table[index]).</code>	Insertion at the head is simpler and faster than at the tail.
<code>hashLookupByValue(hash, value)</code> (Figure 6.8c)	Find a key by value (reverse lookup)	<code>hash, value</code> – search value	Key or <code>undefined</code> if not found	1. Iterate over all buckets (0 to <code>size-1</code>). 2. In each bucket, traverse the chain. 3. If <code>node.value === value</code> , return <code>node.key</code> . 4. If not found, return <code>undefined</code> .	O(n) complexity – inefficient; use only for debugging or rare cases.
<code>hashRemove(hash, key)</code> (Figure 6.8d)	Delete an entry by key	<code>hash, key</code> – key to delete	<code>true</code> if deleted, <code>false</code> if key not found	1. Compute index. 2. Traverse the chain with a <code>prev</code> pointer. 3. If <code>node.key === key</code> : if <code>prev === null</code> (head) – <code>hash.table[index] = node.next</code> , else <code>prev.next = node.next</code> . 4. Return <code>true</code> . 5. If not found, return <code>false</code> .	The parameter must be <code>key</code> , not <code>value</code> .
<code>hashTraverse(hash)</code> (Figure 6.8e)	Print table contents (values only)	<code>hash</code>	– (console output)	For each bucket, print "Slot i:" followed by all values (<code>node.value</code>) separated by " → ", or "Empty" if the bucket is empty.	Does not show keys – <code>printHashTable</code> is more informative.
<code>printHashTable(hash)</code> (Figure 6.8f)	Print table in <code>key → value</code> format	<code>hash</code>	– (console output)	For each bucket, build an array of strings " <code>key → value</code> " for all nodes in the chain, then print joined by " → " or "Empty".	Better than <code>hashTraverse</code> for debugging.

DRAKON-diagrams of hash table algorithms: find by key, insert, reverse lookup, delete, traverse,

and print are shown in Figure 6.8a–f.

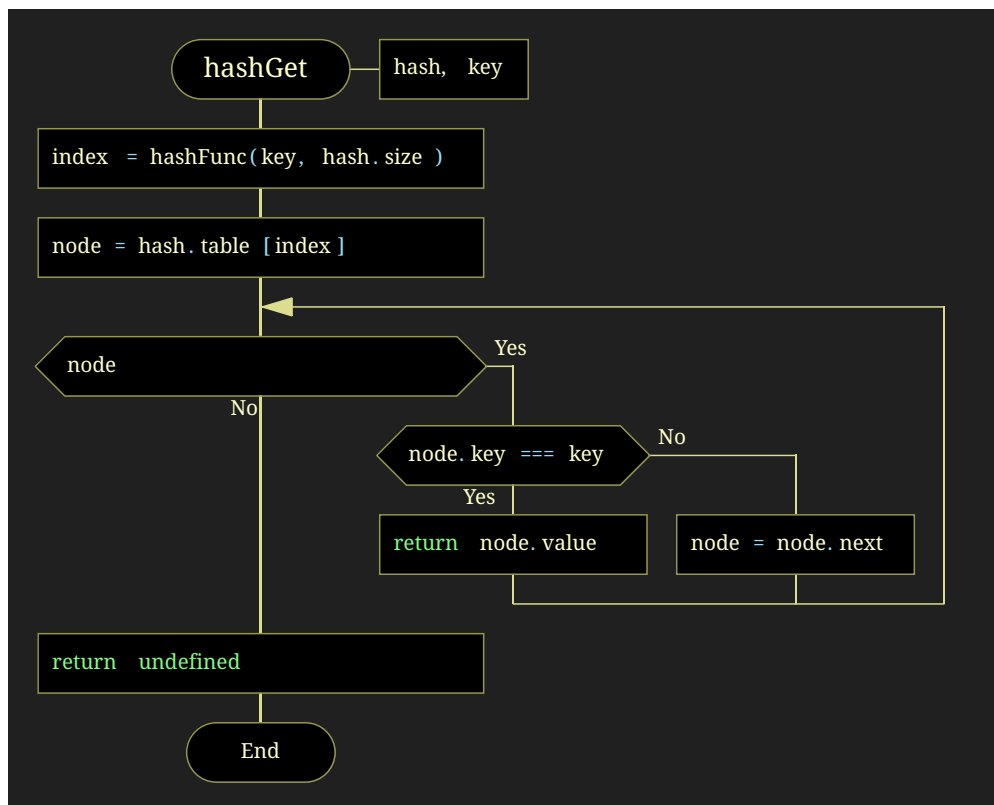


Figure 6.8a. Hash Table Search by Key (`hashGet`)

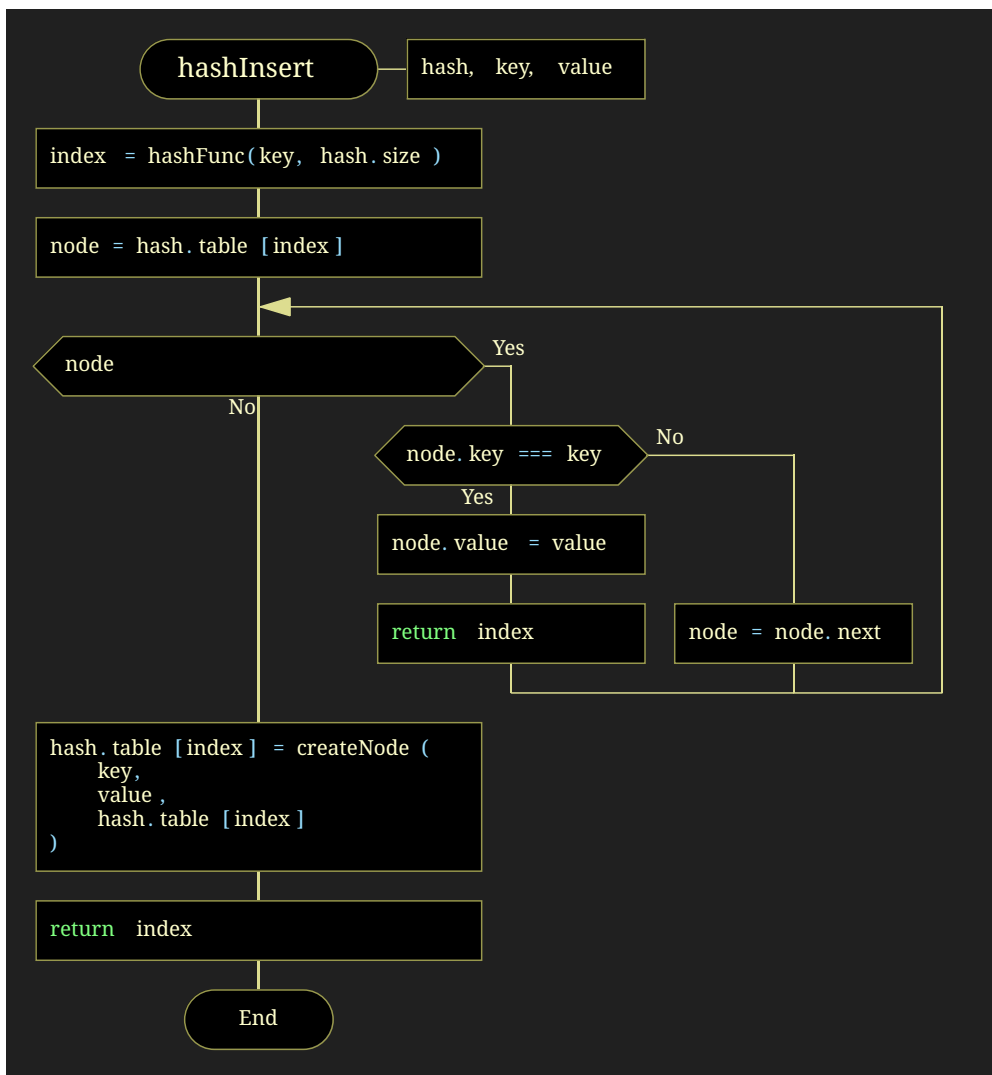


Figure 6.8b. Hash Table Insertion or Update (`hashInsert`)

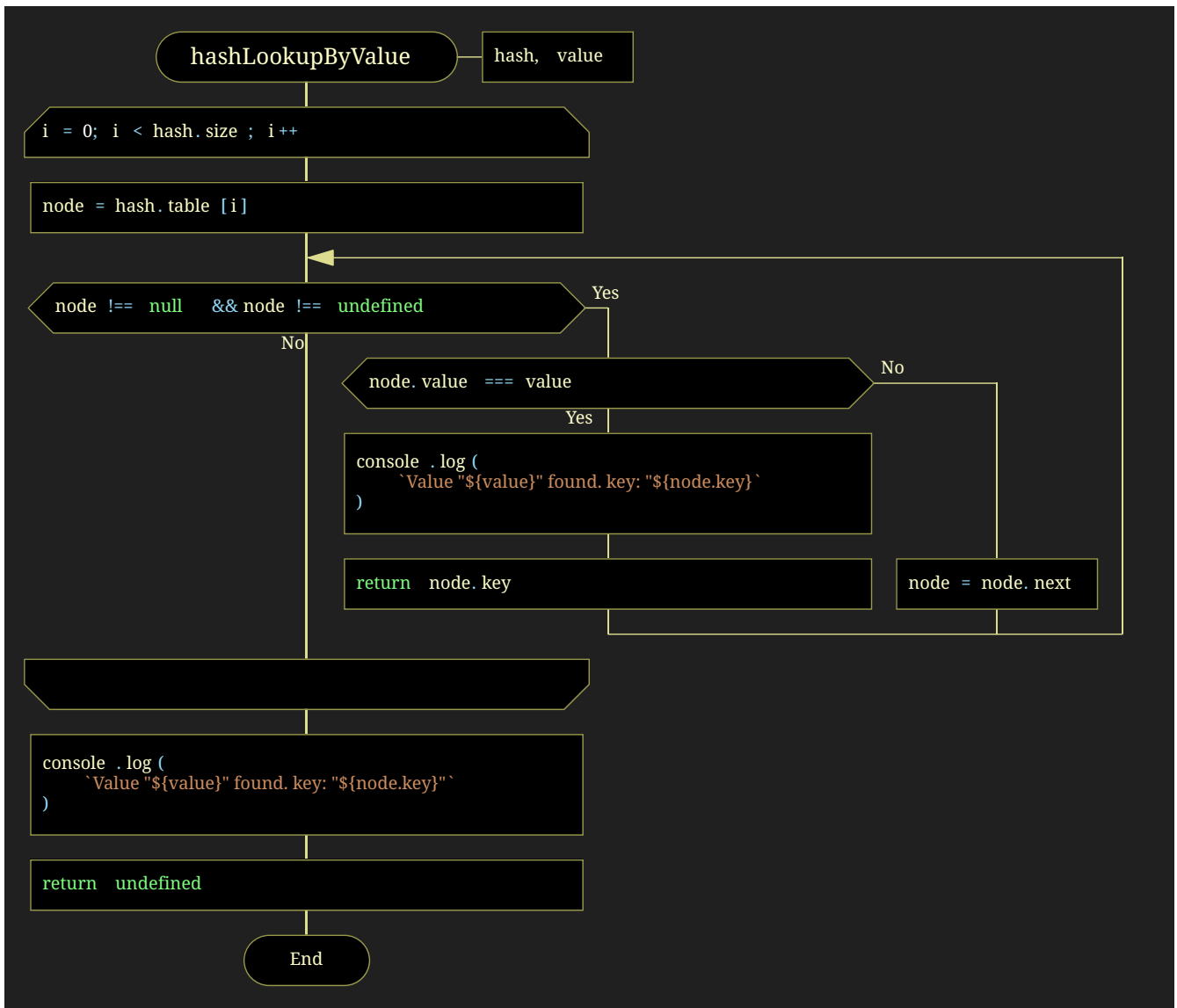


Figure 6.8c. Hash Table Reverse Lookup by Value (`hashLookupByValue`)

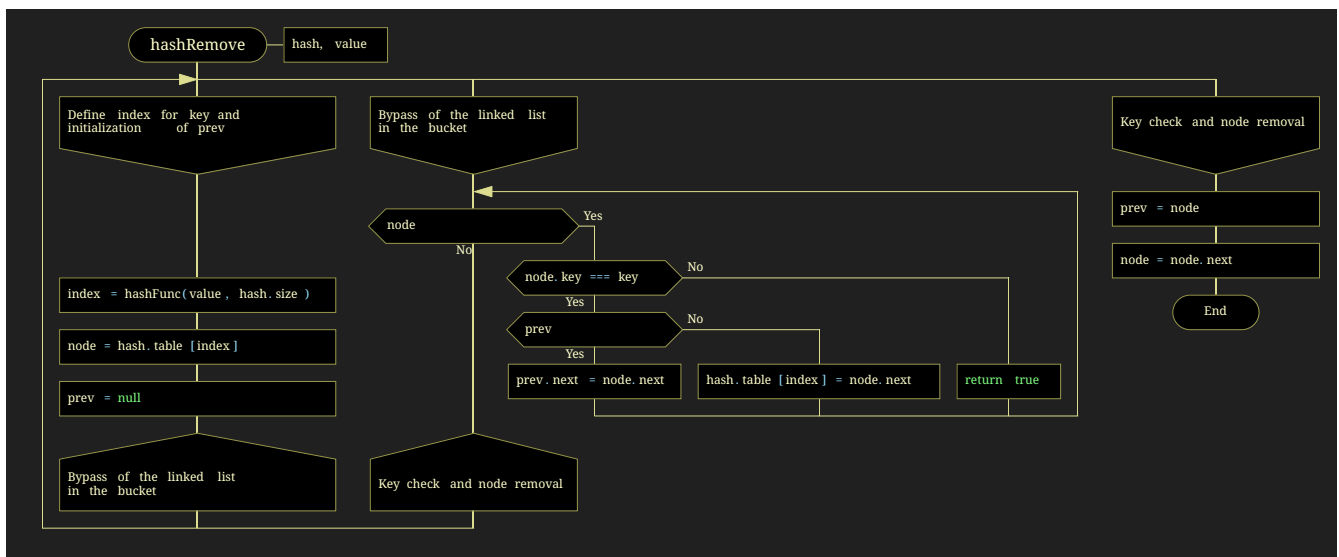


Figure 6.8d. Hash Table Deletion by Key (`hashRemove`)

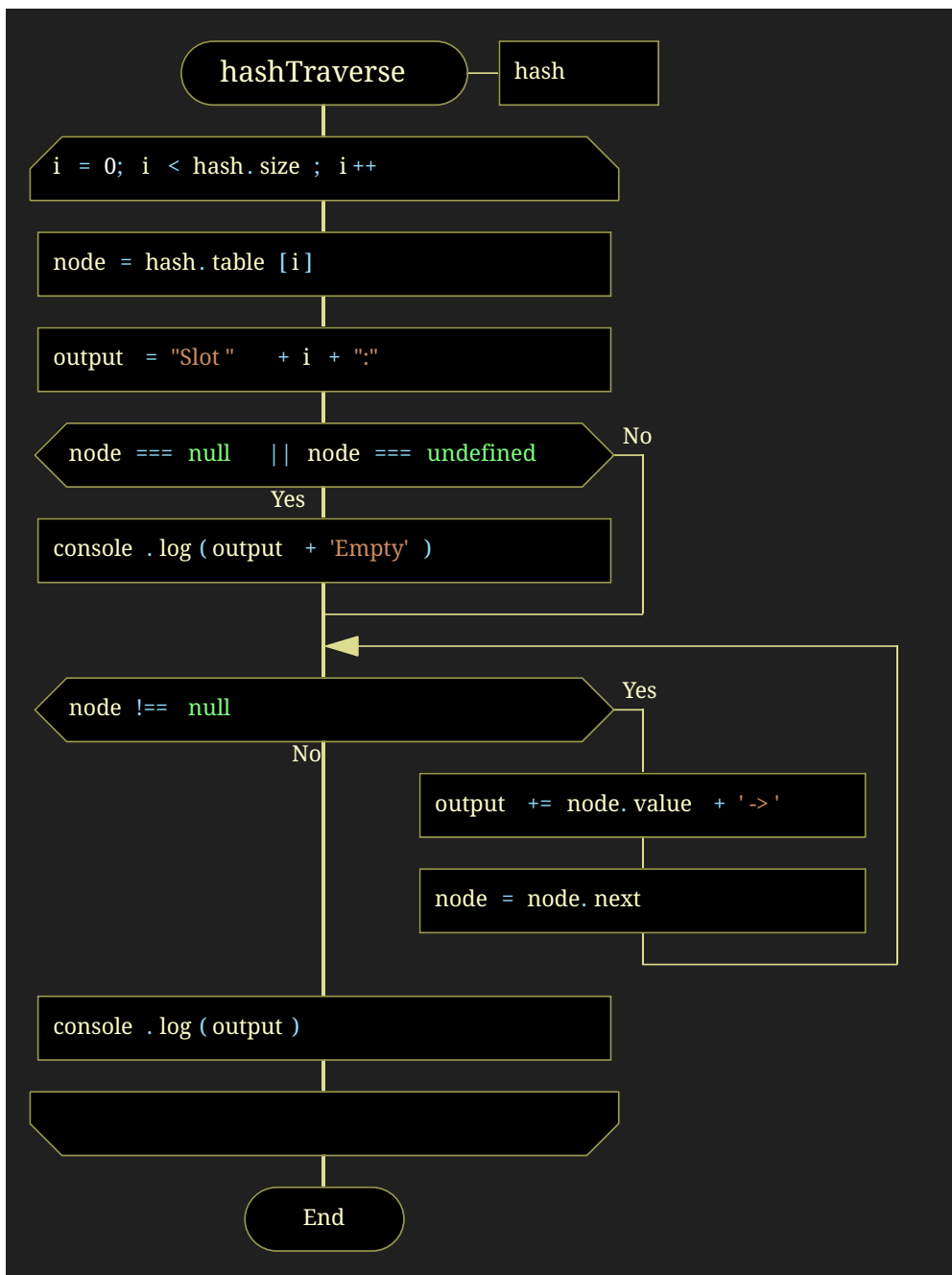


Figure 6.8e. Hash Table Traversal Function (hashTraverse)

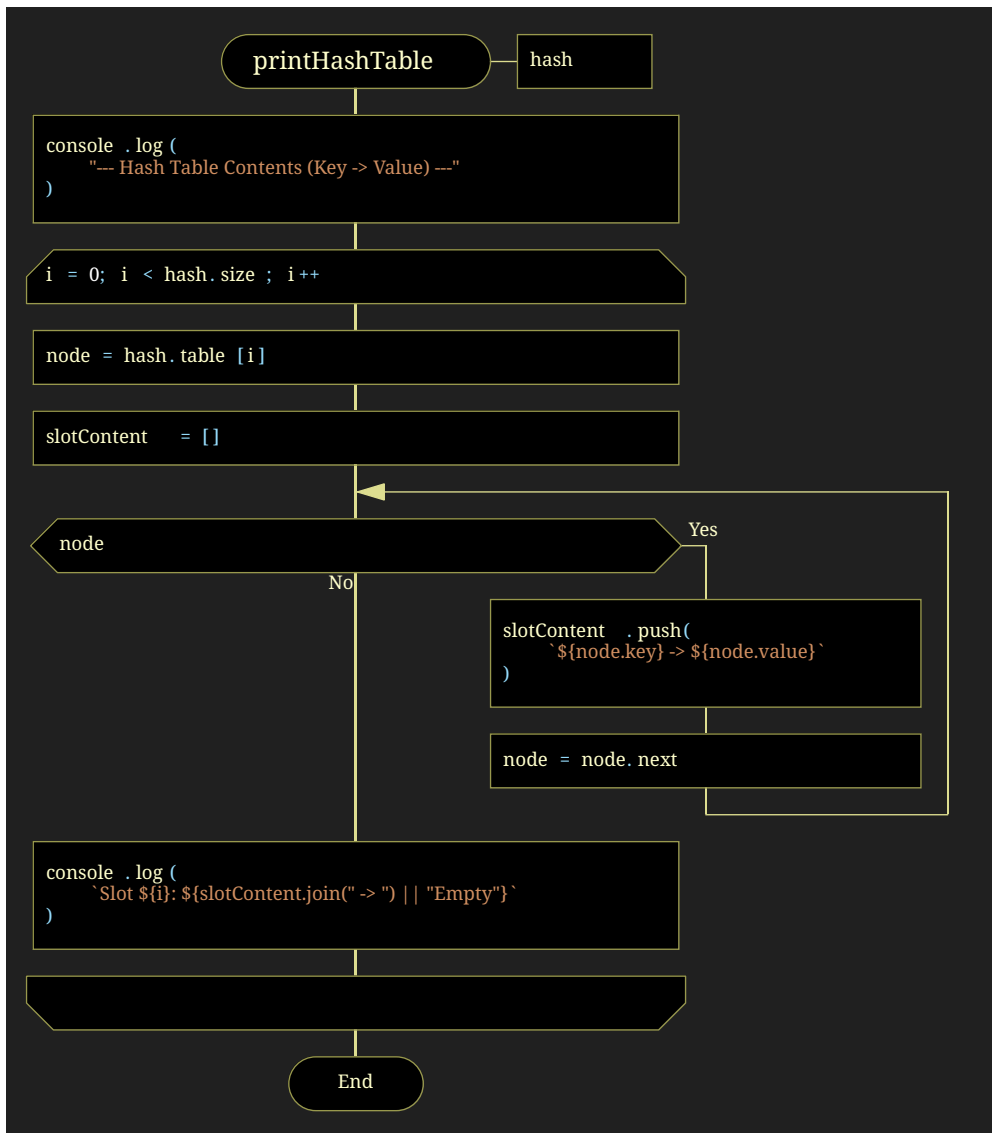


Figure 6.8f. Hash Table Printing (`printHashTable`)

6.4.2. Concrete Example

Consider a concrete example. A hash table of size $m = 16$ is populated with 16 records, each keyed by a six-digit identifier and holding a person's name as the value. Calling `printHashTable(hash)` after all insertions produces the following snapshot of the table (Table 6.3):

Table 6.3: Hash Table Contents Before Removal (`printHashTable`, $m = 16$)

Slot	Chain (key → value)
0	Empty
1	625073 → Olivia J. Watson
2	Empty
3	519283 → Alice R. Cooper
4	476820 → Ivy N. Butler → 357908 → Carol A. Foster
5	807541 → Mia G. Richardson → 392645 → Jack K. Barnes → 430165 → Emma L. Reed

Slot	Chain (key → value)
6	941206 → Paul V. Steven → 746102 → Bob M. Bennett
7	Empty
8	679832 → Frank W. Brooks
9	Empty
10	169834 → Noah D. Patterson → 734218 → Leo C. Coleman → 821674 → David S. Hughes
11	213547 → Grace E. Simmons
12	Empty
13	958301 → Henry T. Peterson
14	Empty
15	568079 → Kelly P. Ross

Suppose we now call `hashRemove(hash, 821674)`. The algorithm proceeds as follows:

- Compute the bucket index: `hashFunc(821674) = 821674 mod 16 = 10`.
- Traverse the chain stored in slot 10. The chain currently holds three nodes: `169834 → Noah D. Patterson, 734218 → Leo C. Coleman, 821674 → David S. Hughes`.
- After each `node.key === key` check, the previously visited node is kept in the `prev` variable. The first two nodes (`169834, 734218`) do not match and are skipped in turn.
- The third node matches (`node.key === 821674`). Since it is not the head of the chain, the link is repaired by setting `prev.next = node.next` — the node holding `David S. Hughes` is detached from the chain, and `hashRemove` returns `true`.

Only slot 10 is affected by this operation; every other slot remains exactly as in Table 6.4:

Table 6.4: Slot 10 Before and After `hashRemove(hash, 821674)`

State	Chain (key → value)
Before	169834 → Noah D. Patterson → 734218 → Leo C. Coleman → 821674 → David S. Hughes
After	169834 → Noah D. Patterson → 734218 → Leo C. Coleman

Finally, consider a call to `hashLookupByValue(hash, "Olivia J. Watson")` — a reverse lookup, illustrating the $O(n)$ traversal described in Table 6.2. The algorithm scans every bucket in order; the very first non-empty bucket it inspects, slot 1, already contains the matching node, and the function returns key `625073`. Unlike `hashGet`, which locates a value in average-case $O(1)$ time by jumping straight to the correct bucket, `hashLookupByValue` has no such shortcut: in the worst case it must inspect every node in every bucket before concluding that no match exists.

6.4.3. Topics Beyond the Scope of This Book

This section has focused on the fundamental principles of hashing and the implementation of a hash table using separate chaining. However, several important and advanced aspects of modern hashing remain outside the scope of this book:

- Open addressing techniques — linear probing, quadratic probing, and double hashing.
- Dynamic resizing of hash tables when the load factor exceeds a threshold.
- Cryptographic hash functions (MD5, SHA family, etc.) used for data integrity and security rather than indexing.
- Perfect hashing and minimal perfect hashing, which eliminate collisions entirely but require complex precomputation.
- Distributed hash tables (DHTs) — used in peer-to-peer networks and blockchain architectures.

These topics will be explored in future works devoted to large-scale data management, high-performance systems, and secure information storage.

Chapter 7. Section 7. Basic Sorting Algorithms

This section provides a comprehensive analysis of fundamental sorting algorithms for contiguous memory segments (array slices). It explores core concepts, algorithmic mechanics, and performance trade-offs across classic comparison-based methods: Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, Merge Sort, Shell Sort, and Heap Sort. Each algorithm is evaluated in terms of best, average, and worst-case time complexity, auxiliary space requirements, and numerical stability. To enhance pedagogical clarity and structural logic, visual workflow formalisms—including DRAKON diagrams, step-by-step state transition tables, and heap tree visualizations—are systematically integrated. Finally, a comparative framework is established to guide optimal algorithm selection based on key data characteristics and constraints.

7.1. 7.1. General Characteristics of Sorting Algorithms

Sorting is one of the most common operations in data processing — not only in computer science, but in many areas of human activity. In computing, sorting algorithms are used everywhere: in databases, search engines, operating systems, and beyond.

At its core, the goal of sorting is simple: given a list of records (each with a key value), we want to reorder the list so that the keys are arranged in increasing or decreasing order. This makes it easier to work with the data — especially when we need to search or analyze it later.

Over time, many sorting algorithms have been developed. Some are easy to implement and work well for small datasets, but become inefficient as the dataset grows. Others are designed for large collections and offer better performance, though they may be more complex to understand or implement.

To help understand how these algorithms work, especially the more complex ones, we will use visual explanations. In this book, we include DRAKON-diagrams and illustrations to make the logic clear and accessible.

Since most sorting tasks in programming involve arrays or similar structures, this section focuses on sorting slices (contiguous segments of arrays).

7.2. 7.2. Bubble Sort

Bubble Sort is one of the simplest sorting algorithms, especially suitable for small datasets. The algorithm gets its name from the way larger values "bubble up" to the end of the array with each

pass. The DRAKON-diagram of the algorithm is shown in Figure 7.1.

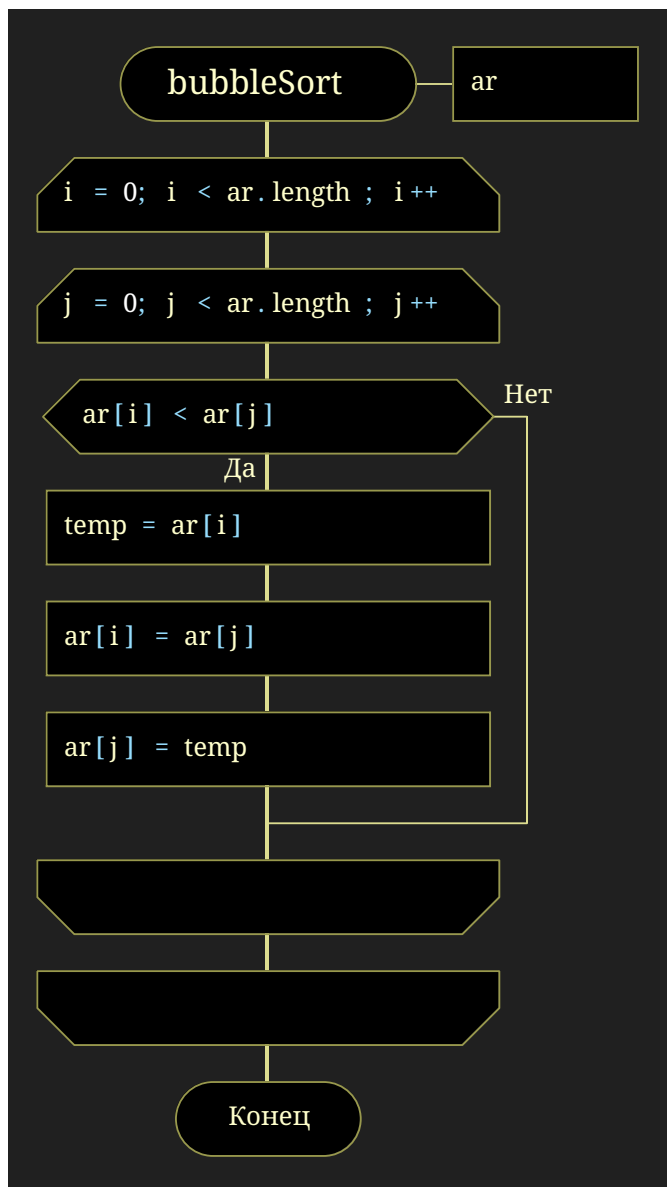


Figure 7.1. DRAKON-diagram of the Bubble Sort algorithm

The algorithm uses two nested loops. For each index i , it iterates over the remaining elements with index j . At each step, the values $arr[i]$ and $arr[j]$ are compared. If $arr[j]$ is smaller than $arr[i]$, the two elements are swapped. This process pushes the largest unsorted value toward the end of the array during each outer loop iteration.

The following table shows the results of sorting a series of values for each outer loop parameter (i).

$j=0$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$	$j=8$	$j=9$
90	11	81	21	71	31	61	41	51	10
11	90	81	21	71	31	61	41	51	10
11	81	90	21	71	31	61	41	51	10
11	21	81	90	71	31	61	41	51	10
11	21	71	81	90	31	61	41	51	10
11	21	31	71	81	90	61	41	51	10

j=0	j=1	j=2	j=3	j=4	j=5	j=6	j=7	j=8	j=9
11	21	31	61	71	81	90	41	51	10
11	21	31	51	61	71	81	90	51	10
11	21	31	41	51	61	71	81	90	10
10	11	21	31	41	51	61	71	81	90

The time complexity of Bubble Sort is quite high — $O(n^2)$. This is due to the large number of comparisons ($\text{arr}[j] < \text{arr}[i]$) and possible swaps. However, the space complexity is $O(1)$, since no additional memory is needed beyond the original array.

The algorithm is stable, meaning that equal elements retain their original order after sorting.

Because of its simplicity, Bubble Sort is still used in some applications — for example, in computer graphics. There, it is helpful for correcting small errors in nearly sorted arrays, often completing in linear time (about $2n$ steps). However, it is highly inefficient for large datasets.

7.3. 7.3. Selection Sort

The Selection Sort algorithm is based on comparison operations. It works by dividing the dataset into two parts: the sorted portion on the left and the unsorted portion on the right. The DRAKON-diagram of this algorithm is shown in Figure 7.2.

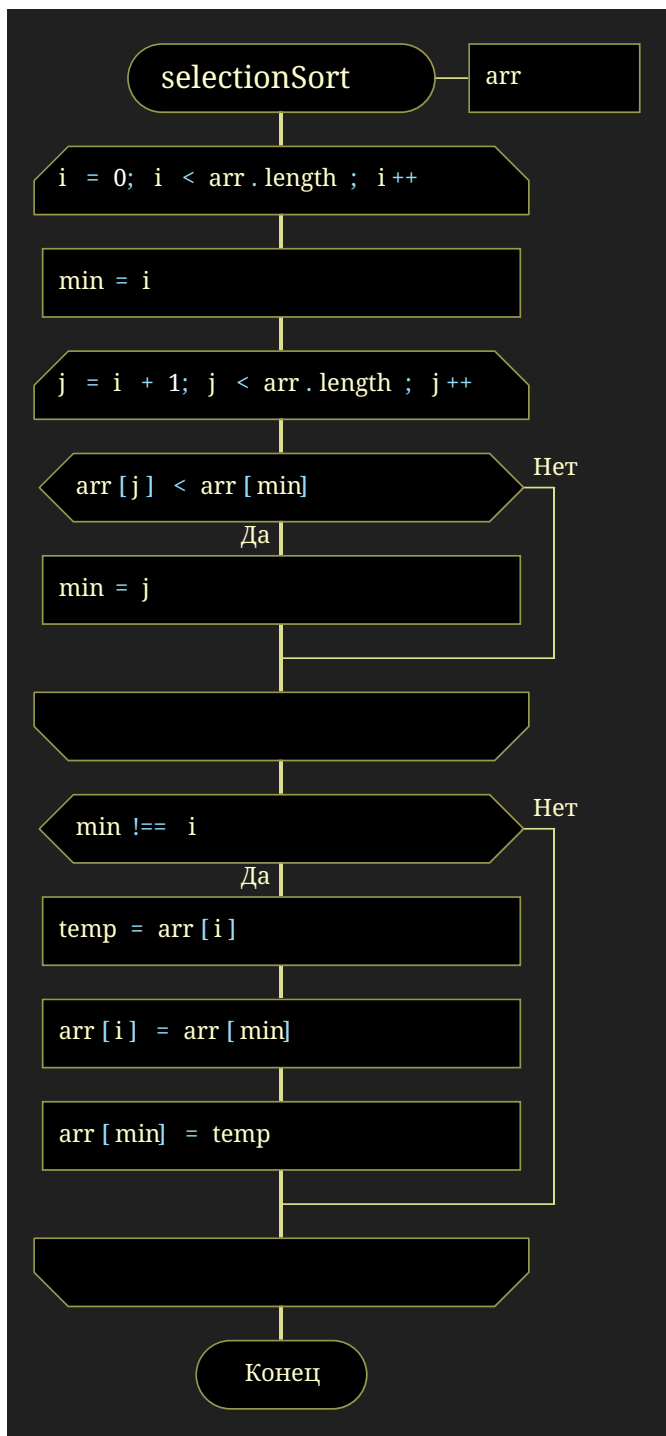


Figure 7.2. DRAGON-diagram of the Selection Sort algorithm

The idea is simple: in each iteration, the algorithm finds the smallest value in the unsorted part and swaps it with the first unsorted element. Initially, the sorted part is empty, and the unsorted part is the entire array.

For example, given the set [90, 12, 83, 24, 75, 38, 62, 41, 59, 10], the algorithm scans the array and finds the smallest element — 10. It then swaps 10 with the element at the first position (90). The sorted part now contains 10, and the process repeats with the remaining elements. The table below illustrates this process of selection and replacement.

90	12	83	24	75	38	62	41	59	10
<u>10</u>	12	83	24	75	38	62	41	59	90

<u>10</u>	12	83	24	75	38	62	41	59	<u>90</u>
<u>10</u>	<u>12</u>	83	24	75	38	62	41	59	<u>90</u>
<u>10</u>	<u>12</u>	<u>24</u>	83	75	38	62	41	59	<u>90</u>
<u>10</u>	<u>12</u>	<u>24</u>	<u>38</u>	75	83	62	41	59	<u>90</u>
<u>10</u>	<u>12</u>	<u>24</u>	<u>38</u>	<u>41</u>	83	62	75	59	<u>90</u>
<u>10</u>	<u>12</u>	<u>24</u>	<u>38</u>	<u>41</u>	<u>59</u>	62	75	83	<u>90</u>
<u>10</u>	<u>12</u>	<u>24</u>	<u>38</u>	<u>41</u>	<u>59</u>	<u>62</u>	75	83	<u>90</u>
<u>10</u>	<u>12</u>	<u>24</u>	<u>38</u>	<u>41</u>	<u>59</u>	<u>62</u>	<u>75</u>	<u>83</u>	<u>90</u>

Note to the table: Underlined items are already sorted; selected items are minimal in the current pass.

The time and space complexity of the Selection Sort algorithm are summarized in the table below:

Time Complexity	
Worst case	$O(n^2)$
Average case	$O(n^2)$
Best case	$O(n^2)$
Space Complexity	
All cases	$O(1)$

The time complexity is the same in all cases because the algorithm uses two nested loops:

- The outer loop runs n times (once per element)
- The inner loop runs $(n - 1)$ times per outer iteration
- This results in a total of $n * (n - 1)$ comparisons, which gives the time complexity of $O(n^2)$ regardless of the input order.

7.4. 7.4. Insertion Sort

Insertion Sort works by repeatedly taking an item from the unsorted part of the array and inserting it into its correct position in the sorted part. This process continues until all items are sorted.

This algorithm is similar to how people naturally sort playing cards or paper documents: taking one item at a time and inserting it into the correct place. The DRAKON-diagram for the Insertion Sort algorithm is shown in Figure 7.3.

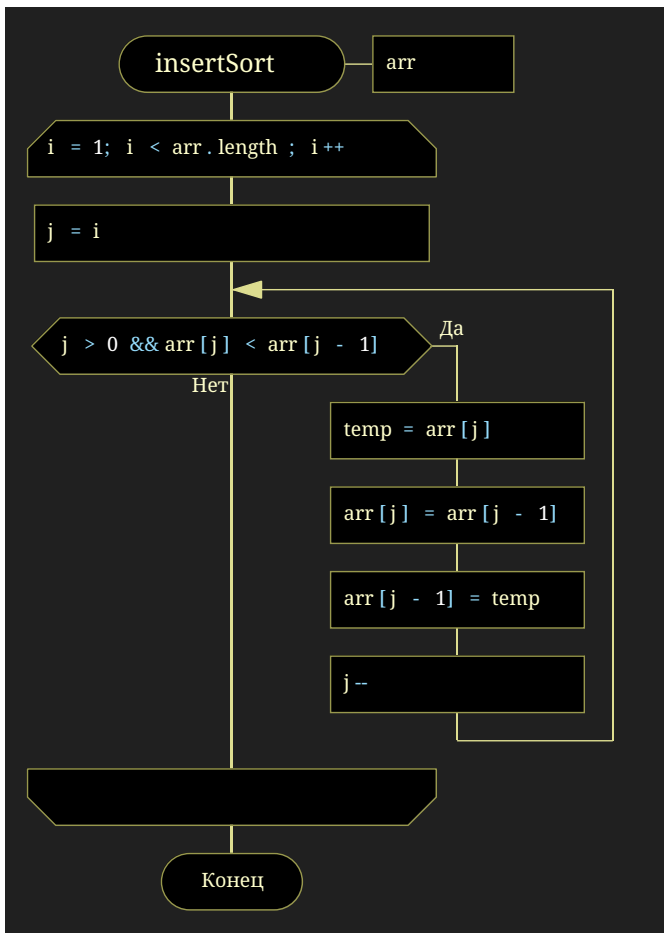


Figure 7.3. DRAKON-diagram of the Insertion Sort algorithm

In this algorithm, larger values are pushed to the right. The algorithm compares adjacent elements and swaps them if needed. This process is shown in the table below, where underscores mark the positions where swaps occurred.

Input array									
90	11	81	21	71	31	61	41	51	10
i = 1									
11	90	81	21	71	31	61	41	51	10
i = 2									
11	81	90	21	71	31	61	41	51	10
i = 3									
11	81	21	90	71	31	61	41	51	10
11	21	81	90	71	31	61	41	51	10
i = 4									
11	21	81	71	90	31	61	41	51	10
11	21	71	81	90	31	61	41	51	10
i = 5									
11	21	71	81	31	90	61	41	51	10

11	21	71	31	81	90	61	41	51	10
11	21	31	71	81	90	61	41	51	10
i = 6									
11	21	31	71	81	61	90	41	51	10
11	21	31	71	61	81	90	41	51	10
11	21	31	61	71	81	90	41	51	10
i = 7									
11	21	31	61	71	81	41	90	51	10
11	21	31	61	71	81	41	51	90	10
11	21	31	61	41	71	81	90	51	10
11	21	31	41	61	71	81	90	51	10
i = 8									
11	21	31	41	61	71	81	51	90	10
11	21	31	41	61	71	51	81	90	10
11	21	31	41	61	51	71	81	90	10
11	21	31	41	51	61	71	81	90	10
i = 9									
11	21	31	41	51	61	71	81	10	90
11	21	31	41	61	51	71	10	81	90
11	21	31	41	51	61	71	10	81	90
11	21	31	41	51	61	10	71	81	90
11	21	31	41	51	10	61	71	81	90
11	21	31	41	10	51	61	71	81	90
11	21	31	10	41	51	61	71	81	90
11	21	10	31	41	51	61	71	81	90
11	10	21	31	41	51	61	71	81	90
10	11	21	31	41	51	61	71	81	90

The time complexity of Insertion Sort depends on how the elements are arranged in the input array:

Worst case

when the array is sorted in reverse order. Each element must be compared with all previous elements and moved to the beginning of the array. The number of operations is $n(n - 1) / 2 \rightarrow O(n^2)$.

Average case

when the array is partially sorted. Each element is compared, on average, with half of the

previous elements: $(n^2) / 4 \rightarrow O(n^2)$.

Best case

when the array is already sorted. Each element is compared with only the one before it: $(n - 1) \rightarrow O(n)$.

The complexity summary is given in the table below:

Time Complexity	
Worst case	$O(n^2)$
Average case	$O(n^2)$
Best case	$O(n)$
Space complexity	$O(1)$

7.5. 7.5. Quick Sort

Quick Sort is a highly efficient sorting algorithm based on the divide-and-conquer strategy. Its general scheme includes the following steps:

1. Select a pivot (reference) element from the array slice.
2. Partition the array so that all elements smaller than the pivot are placed before it, and all greater or equal — after it.
3. Recursively apply this procedure to the left and right partitions.
4. As a result, the array becomes fully sorted.

The DRAKON-diagram of the Quick Sort algorithm is shown in Figure 7.4.

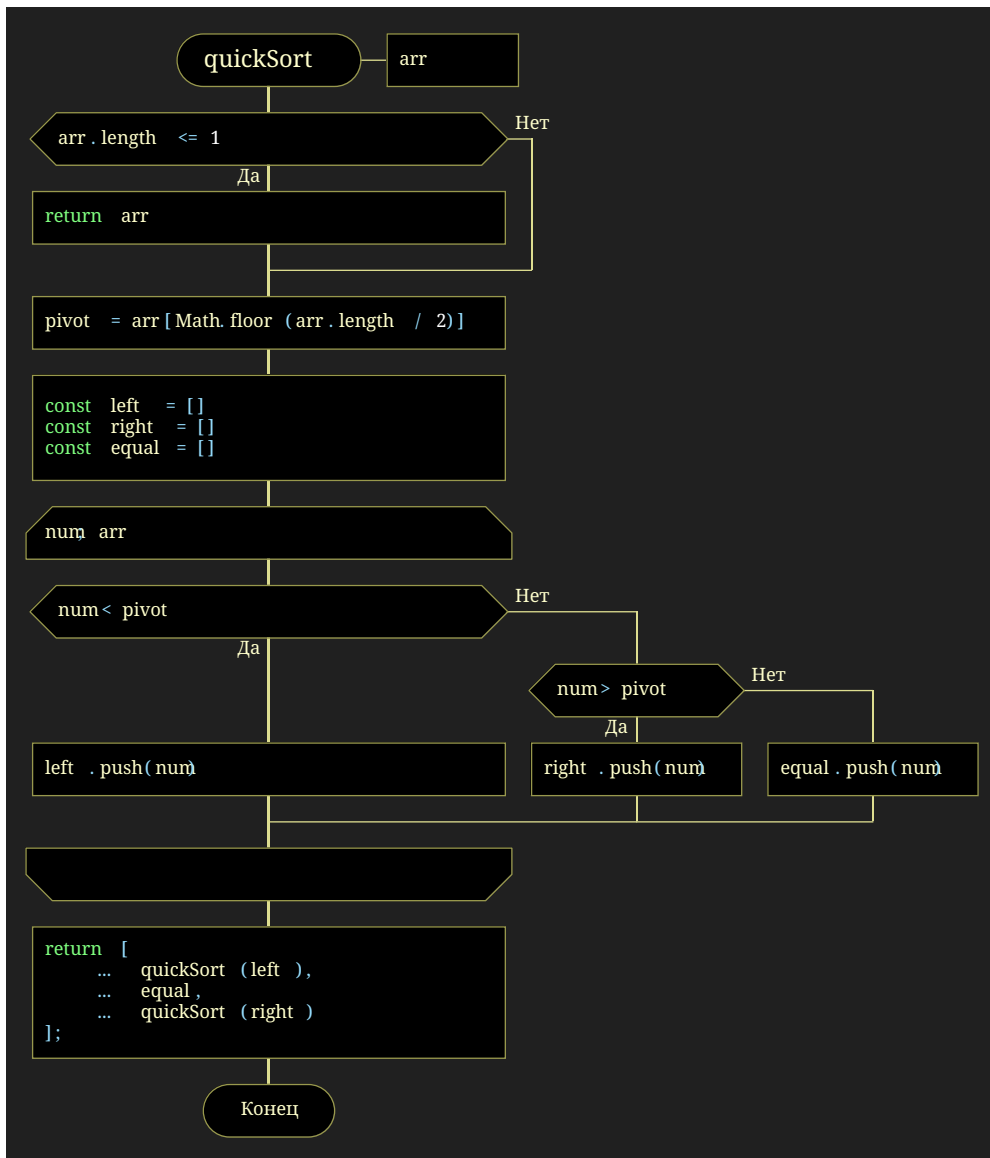


Figure 7.4. DRAGON-diagram of the Quick Sort algorithm

Let us look at an example in detail. Suppose we are sorting the array [78, 11, 81, 21, 71, 31, 61, 41, 51, 24], and we choose 31 as the pivot.

78	11	81	21	71	31	61	41	51	24
----	----	----	----	----	----	----	----	----	----

We create three temporary arrays:

- left[] for elements less than the pivot
- equal[] for elements equal to the pivot
- right[] for elements greater than the pivot

During execution, if an element from the left side is greater than the pivot, and one from the right side is smaller, the two elements are swapped. In our example, $78 > 31$ and $24 < 31$, so they are exchanged. This process continues until the subarray is partitioned correctly. Then, Quick Sort is applied recursively to the left and right partitions.

To better understand how recursive sorting builds the final result, the table below shows how the quickSort function combines the left, equal, and right parts at each step:

Step	Left	Pivot	Right	Result = left + equal + right
[78, 11, 81, 21, 71, 31 , 61, 41, 51, 24]	11, 21, 24	31	78, 81, 71, 61, 41, 51	–
[11, 21, 24]	11	21	24	11, 21, 24
[78, 81, 71, 61, 41 , 51]	41, 51	61	78, 81, 71	11, 21, 24
[41, 51]	41	51		11, 21, 24, 41, 51
[78, 81, 71]	78, 71	81		11, 21, 24, 41, 51, 61
[78, 71]		71	78	11, 21, 24, 41, 51, 61, 71, 78
[81]		81		11, 21, 24, 41, 51, 61, 71, 78, 81

The time complexity of Quick Sort depends on how well the pivot is chosen:

- Best case: the pivot is the median $\rightarrow O(n \log n)$
- Average case: still $O(n \log n)$, due to recursive halving
- Worst case: when the pivot is the smallest or largest item, resulting in unbalanced partitions $\rightarrow O(n^2)$

Quick Sort uses recursion. In each step, it partitions the array and performs $O(n)$ work. This leads to the recurrence relation $T(n) = 2T(n/2) + O(n)$, which solves to $O(n \log n)$.

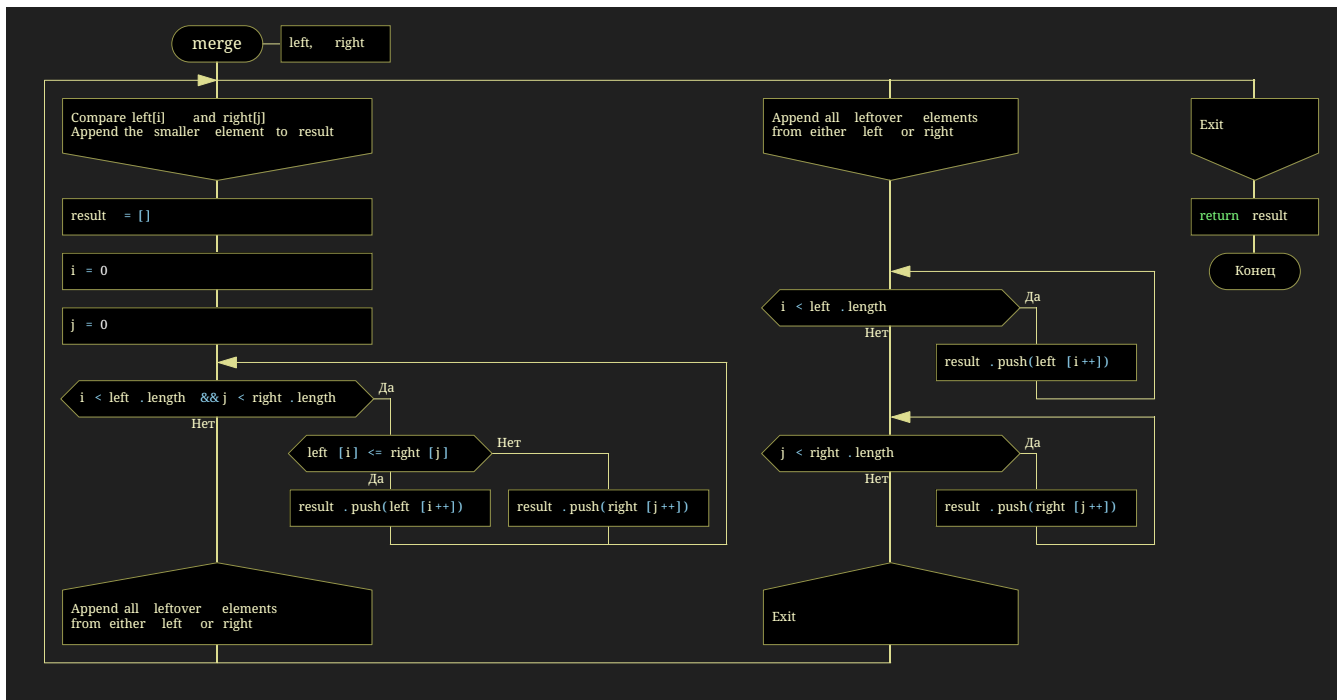
The complexity summary is shown below. Quick Sort is not stable by default.

Time Complexity	
Worst case	$O(n^2)$
Average case	$O(n \log n)$
Best case	$O(n \log n)$
Space Complexity	
Worst case	$O(n)$
Average case	$O(\log n)$

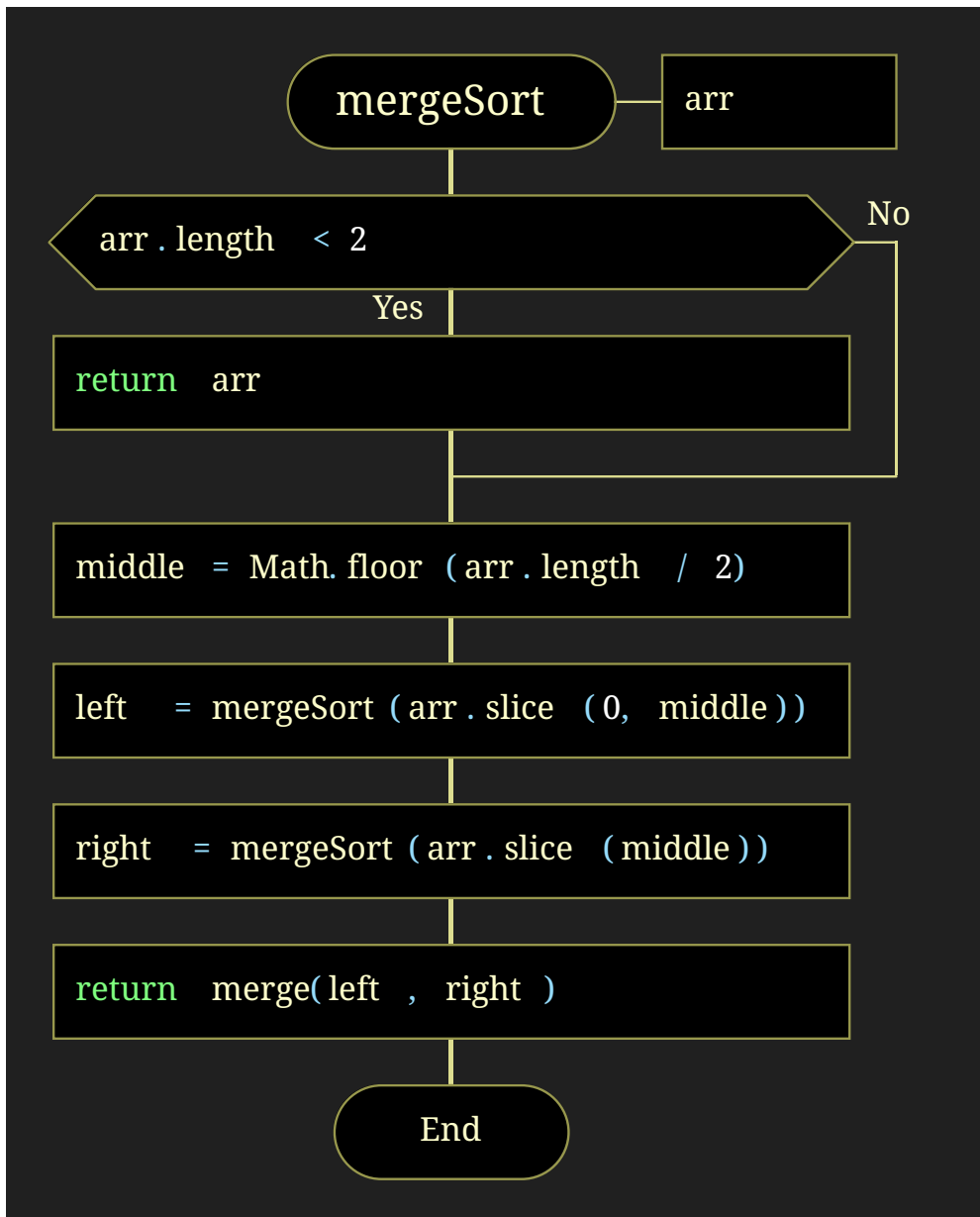
7.6. Merge Sort

Merge Sort is a classic divide-and-conquer algorithm. It works by recursively splitting the array into smaller parts until each part contains only one element (trivially sorted). After the splitting phase, the algorithm merges the sorted parts back together.

During merging, one element from each fragment is selected and compared. The smaller (or larger, depending on the desired order) element is added to the result array. The process continues until all elements are merged. Figure 7.5 illustrates this process:



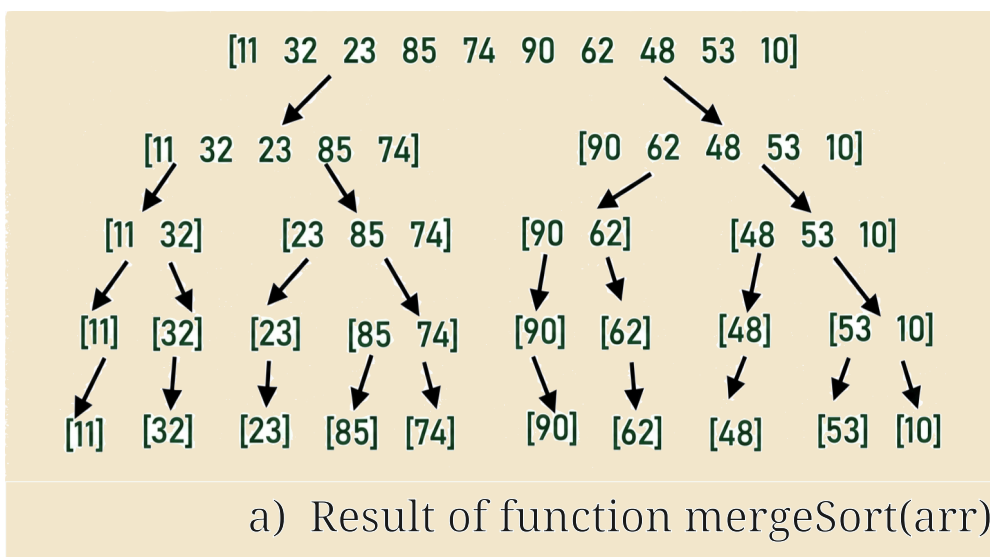
a)



b)

Figure 7.5. DRAKON-diagrams of Merge Sort algorithm: a) merge() function; b) mergeSort() function

A visual representation of splitting and merger phases are shown in Figure 7.6.



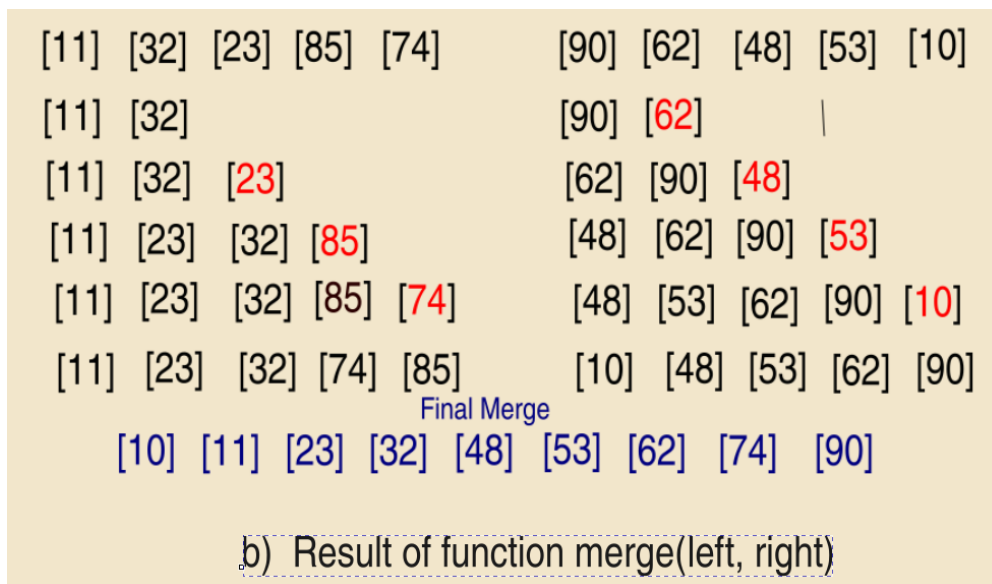


Figure 7.6. Visual representation of Merge Sort algorithm: a) splitting phase; b) merging phase

Merge Sort has a time complexity of $O(n \log n)$, where n is the number of elements in the array. The algorithm splits the array into two halves until only single elements remain. The number of split operations is $O(\log n)$, and each merge operation compares n elements, which gives the overall complexity of $O(n \log n)$.

The summary of time and space complexity is shown below:

Time Complexity	
Worst case	$O(n \log n)$
Average case	$O(n \log n)$
Best case	$O(n \log n)$
Space Complexity	
Worst case	$O(n)$
Average case	$O(n)$
Best case	$O(n)$

Merge Sort is a stable sorting algorithm, which means it preserves the relative order of equal elements in the input array. This stability is achieved because, during the merge phase, if two elements are equal, the one from the left subarray is placed before the one from the right subarray.

7.7. 7.7. Shell Sort

Shell Sort is a variation of Insertion Sort. It begins by comparing and sorting elements that are separated by a certain distance d . Then, the process is repeated for smaller values of d until $d = 1$, which is equivalent to a standard insertion sort.

The DRAKON-diagram of the Shell Sort algorithm is shown in Figure 7.7.

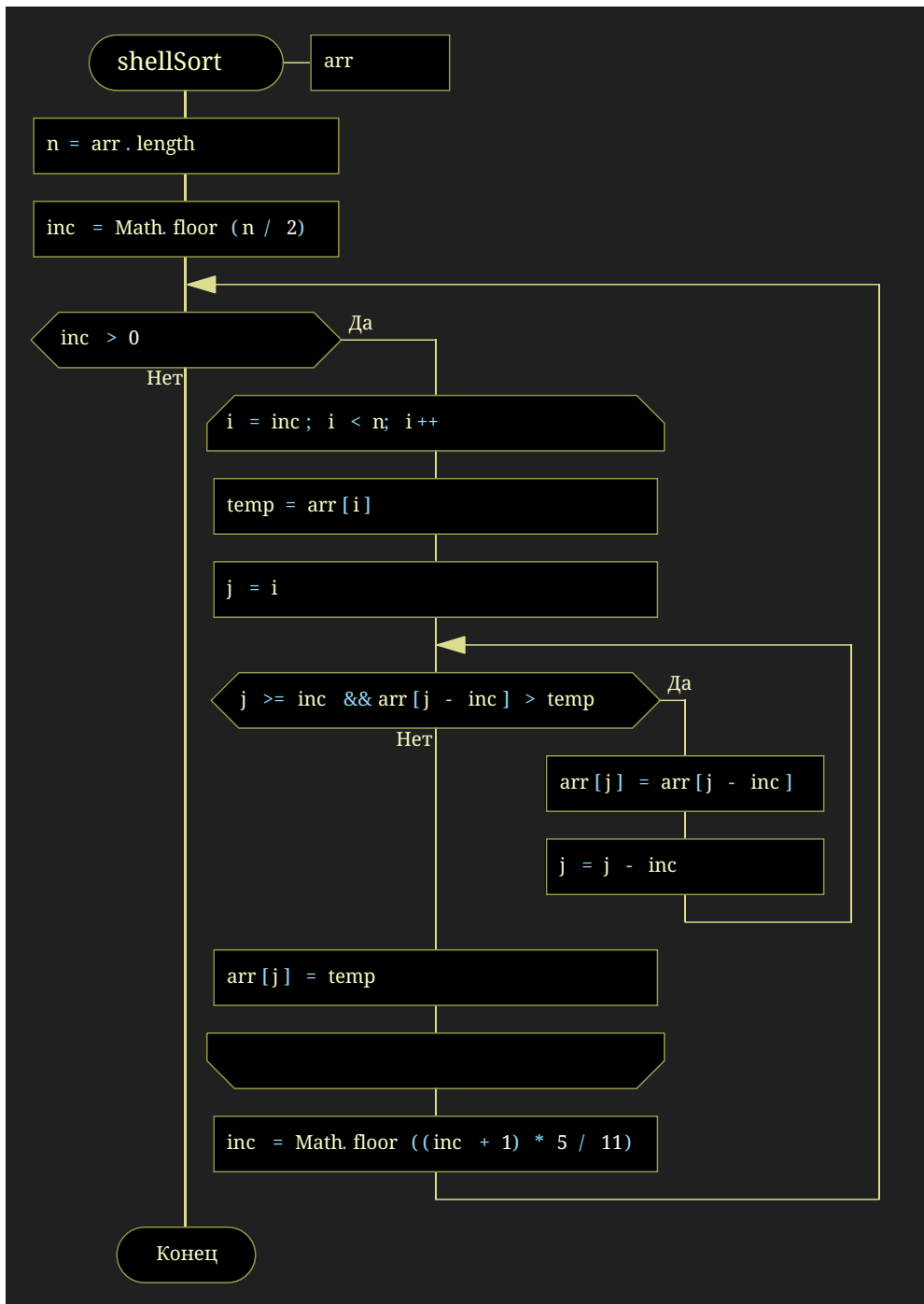


Figure 7.7. DRAKON-diagram of the Shell Sort algorithm

Shell Sort uses a three-step process common to many sorting algorithms:

1. Segmenting the array
2. Sorting within the segments
3. Merging the results into a sorted array

The array is partitioned in such a way that each item in a segment is a fixed number of positions apart from the others. This introduces a challenge: choosing the best distance **d**. A simple example of such a sequence is: $d = n / 2$, then $d = d / 2$, and so on, until $d = 1$.

The table illustrates how elements are swapped when a smaller value is found at position `arr[j]` compared to `arr[j - inc]`.

inc = 5

j	0	1	2	3	4	5	6	7	8	9
	90	-80	70	-60	50	-40	30	-20	10	0
	-40	-80	-20	-60	50	90	30	70	10	0
	-40	-80	-20	-60	0	90	30	70	10	50

inc = 2

j	0	1	2	3	4	5	6	7	8	9
	-40	-80	-20	-60	0	70	30	90	10	50
	-40	-80	-20	-60	0	70	10	90	30	50
	-40	-80	-20	-60	0	70	10	90	30	50
	-40	-80	-20	-60	0	50	10	70	30	90

inc = 1

j	0	1	2	3	4	5	6	7	8	9
	-40	-80	-20	-60	0	50	10	70	30	90
	-80	-40	-60	-20	0	50	10	70	30	90
	-80	-60	-40	-20	0	50	10	70	30	90
	-80	-60	-40	-20	0	10	50	70	30	90
	-80	-60	-40	-20	0	10	50	30	70	90
	-80	-60	-40	-20	0	10	30	50	70	90

The average time complexity of Shell Sort is estimated as $O(n \cdot \log^2 n)$, though this depends on multiple factors.

First, Shell Sort is essentially a generalization of Insertion Sort that uses gap-based comparisons. Second, the performance depends on the choice of gap sequence — that is, the values of d used during sorting. Different sequences can yield significantly different results.

Because Shell Sort works by refining the array through a series of passes with decreasing gaps, its time complexity is sensitive to these gap values. The optimal gap sequence is still a subject of research, and no single sequence is universally optimal.

As a result, although the average-case complexity is commonly cited as $O(n \cdot \log^2 n)$, the exact performance may vary depending on the input and the sequence used.

Shell Sort is not stable in its original form, but it can be modified to preserve stability if needed.

7.8. 7.8. Heap Sort

Heap Sort (pyramid sorting) can be seen as an improved version of Selection Sort: it divides the input data into sorted and unsorted regions, and then successively reduces the unsorted region by removing the largest item and moving it to the sorted region. The improvement is that a binary heap is used to find the highest value, not a linear search algorithm.

This algorithm is executed using the notion of a heap, which is a complete binary tree (see subsection 1.3). In a max-heap, parent nodes are greater than or equal to their children, whereas in a min-heap, parent nodes are less than or equal to their children. A binary heap tree can be of two types:

MinHeap

the parent node is always smaller than the child nodes

MaxHeap

the parent node is always greater than or equal to the child nodes

Figure 7.8 shows examples of both:

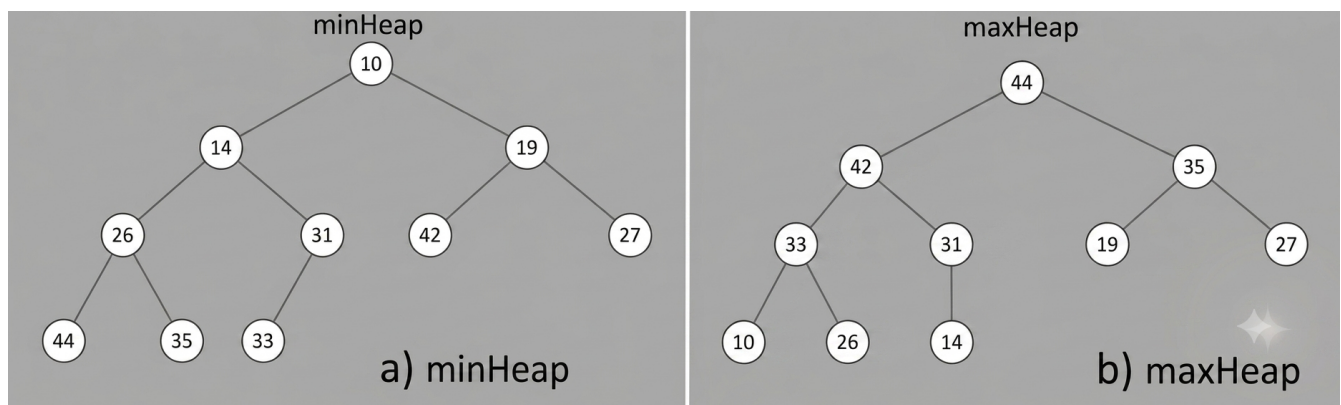


Figure 7.8. Binary tree examples

The sequence of heap construction steps is shown in the table below:

23	5	20	52	11	41	14
23	5	41	52	11	20	14
23	52	41	5	11	20	14
52	23	41	5	11	20	14
14	23	41	5	11	20	52
41	23	14	5	11	20	52
20	23	14	5	11	41	52
23	20	14	5	11	41	52
11	20	14	5	23	41	52
20	11	14	5	23	41	52
5	11	14	20	23	41	52

23	5	20	52	11	41	14
11	5	14	20	23	41	52
5	11	14	20	23	41	52

The Heap Sort algorithm uses three functions:

heapSort

performs the main sorting by rearranging nodes

heapify

compares adjacent nodes to maintain heap property

swap

swaps two nodes

The sequence of node movements in the heap is shown in Figure 7.9:

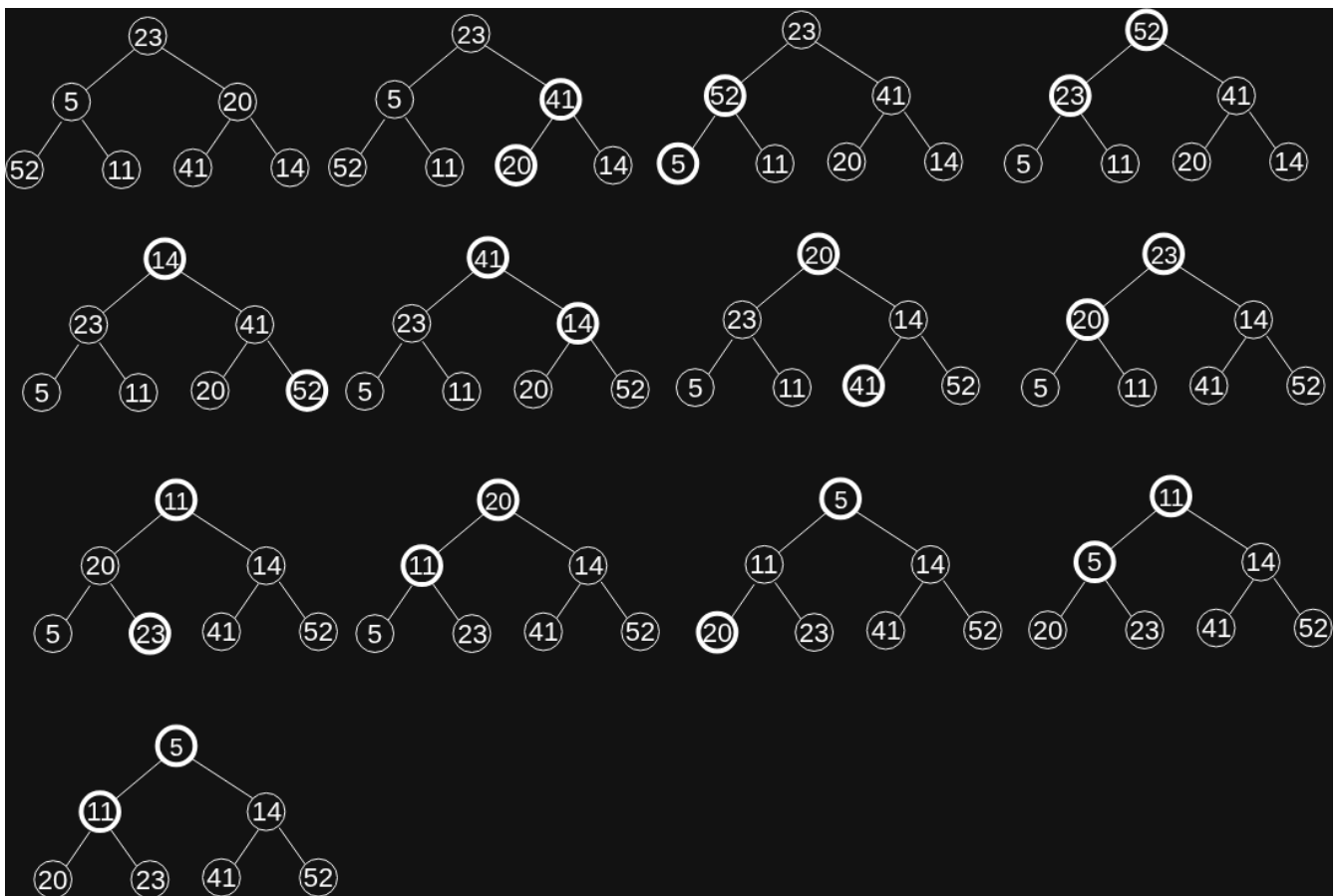


Figure 7.9. The dynamics of the pyramid sorting algorithm

DRAGON-diagrams of the Heap Sort algorithm are presented in Figure 7.10:

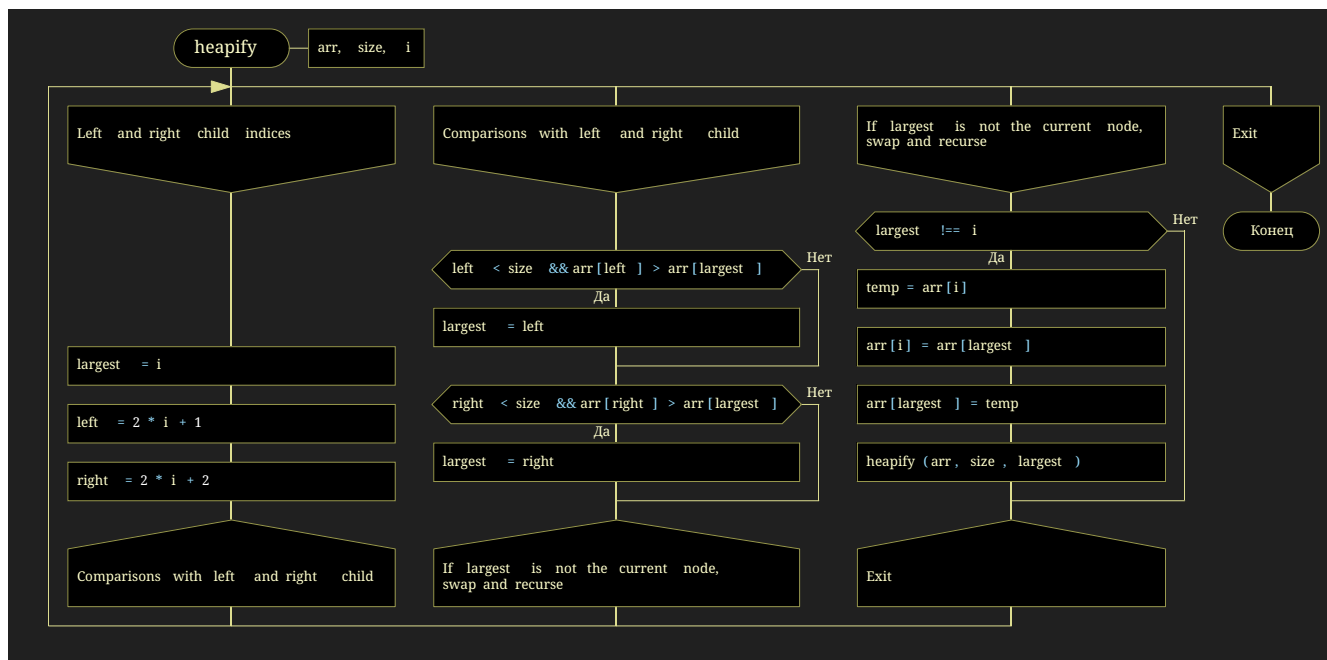
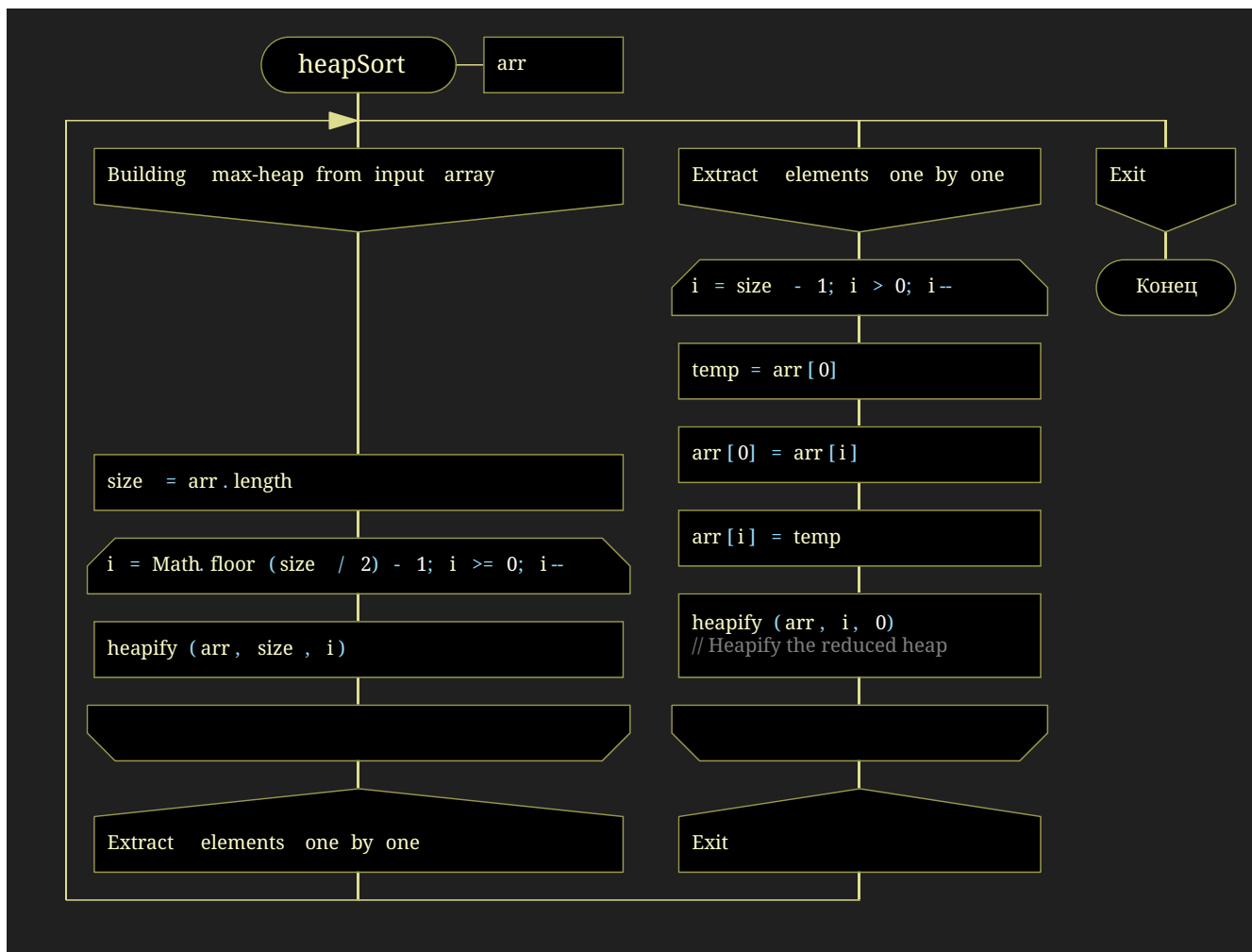


Figure 7.10. DRAKON-diagram of Heap Sort algorithm: a) heapSort(arr); b) heapify(arr, size, i)

The time complexity of Heap Sort is $O(n \log n)$ for all cases (worst, average, and best). The space complexity is $O(1)$ as it sorts in place. Heap Sort is not stable.

7.9.7.9. Comparison of Sorting Algorithms

Choosing a sorting algorithm is determined by the following factors:

- Time complexity
- Space complexity
- Stability

Knowing the strengths and weaknesses of each of the algorithms considered allows you to make a choice in favor of a particular sort. Each algorithm is unique and works best under certain conditions.

Time Complexity Comparison

Algorithm	Worst	Average	Best
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n^2)$	$O(n^2)$	$O(n)$
Quick Sort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Shell Sort	$O(n \cdot \log^2 n)$	$O(n \cdot \log^2 n)$	$O(n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

Space Complexity and Stability Comparison

Algorithm	Space Complexity	Stable
Bubble Sort	$O(1)$	Yes
Selection Sort	$O(1)$	No
Insertion Sort	$O(1)$	Yes
Quick Sort	$O(\log n)$ average, $O(n)$ worst	No
Merge Sort	$O(n)$	Yes
Shell Sort	$O(1)$	No
Heap Sort	$O(1)$	No

Some common sorting algorithms are inherently stable, such as Merge Sort, Insertion Sort, and Bubble Sort. Others, such as Quick Sort, Heap Sort, and Selection Sort, are unstable. However, stability can often be achieved in unstable algorithms by using additional space or modifying the implementation.

Chapter 8. Section 8. Binary Tree Algorithms

This section presents an advanced study of binary tree data structures and algorithms, spanning Binary Search Trees (BST), AVL trees, and Red-Black Trees (RBT), implemented using JavaScript and visual programming formalisms (DRAKON diagrams). A key contribution of this work is the author's innovative conceptual framework based on a node cluster (C-Cluster) representation for insertion and deletion operations. Within this model, the local spatial topology, color state, and structural relationships of candidate nodes are encoded into a compact character string (a textual cluster descriptor). This information image abstracts complex pointers and significantly simplifies the conditional logic of tree restructuring algorithms. Furthermore, this section introduces a novel mechanical analogy for Red-Black Tree balancing—modeling the structure as a suspended chandelier governed by gravitational equilibrium, where black nodes act as solid unit weights and red nodes act as weightless spacers. This mechanical paradigm renders abstract invariants (such as uniform black height and double-red prevention) visually intuitive and directly translates into the automated decision rules of a virtual balancing automaton. Additionally, the section covers foundational traversal strategies, canonical rotation primitives (LL, RR, LR, RL), and exact complexity trade-offs, providing both theoretical rigor and a complete visual reference framework.

8.1. 8.1. Representation of Binary Trees

A binary tree is a hierarchical data structure in which each node has at most two children, referred to as the left and right child. The topmost node is called the root, and nodes with no children are called leaves.

Binary trees as an abstract data type were discussed in Section 1. Here we will discuss the computer implementation of this type of data structure using the visual algorithmic language DRAKON and the JavaScript programming language. Recall the basic terminology of binary trees (Figure 8.1):

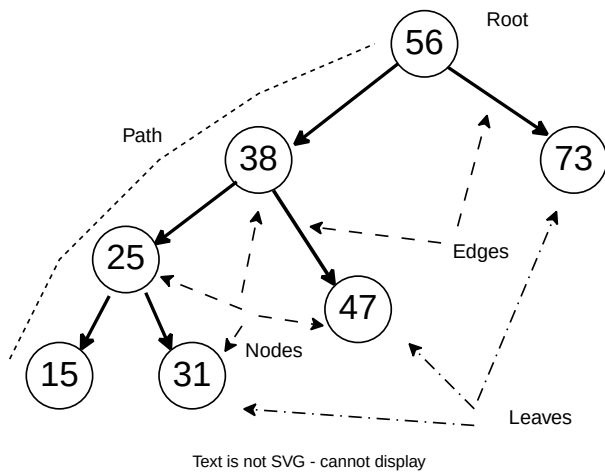


Figure 8.1. Basic tree terminology

- **Root:** The root of the tree is the only node with no incoming edges. It is the top node in the tree.
- **Node:** The basic element of the tree. Each node has data and two references that can point to zero or more children.
- **Edge:** A fundamental part of the tree used to connect two nodes.
- **Path:** An ordered list of nodes connected by edges.
- **Leaf:** A leaf node is a node that has no children.
- **Tree height:** The number of edges on the longest path between the root and a leaf.

Node level: The number of edges on the path from the root node to that node.

The information structure of a binary tree is organized as follows (Figure 8.2):

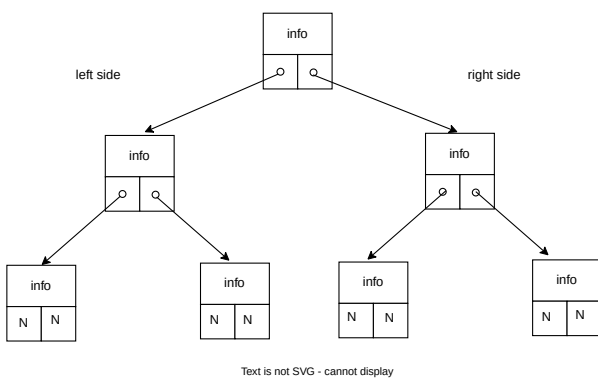


Figure 8.2. Binary tree structure

Several types of binary trees are discussed in the literature, the most important classification being based on node values:

- Binary Search Tree (BST)
- AVL Tree (Adelson-Velsky and Landis Tree)
- Red-Black Tree

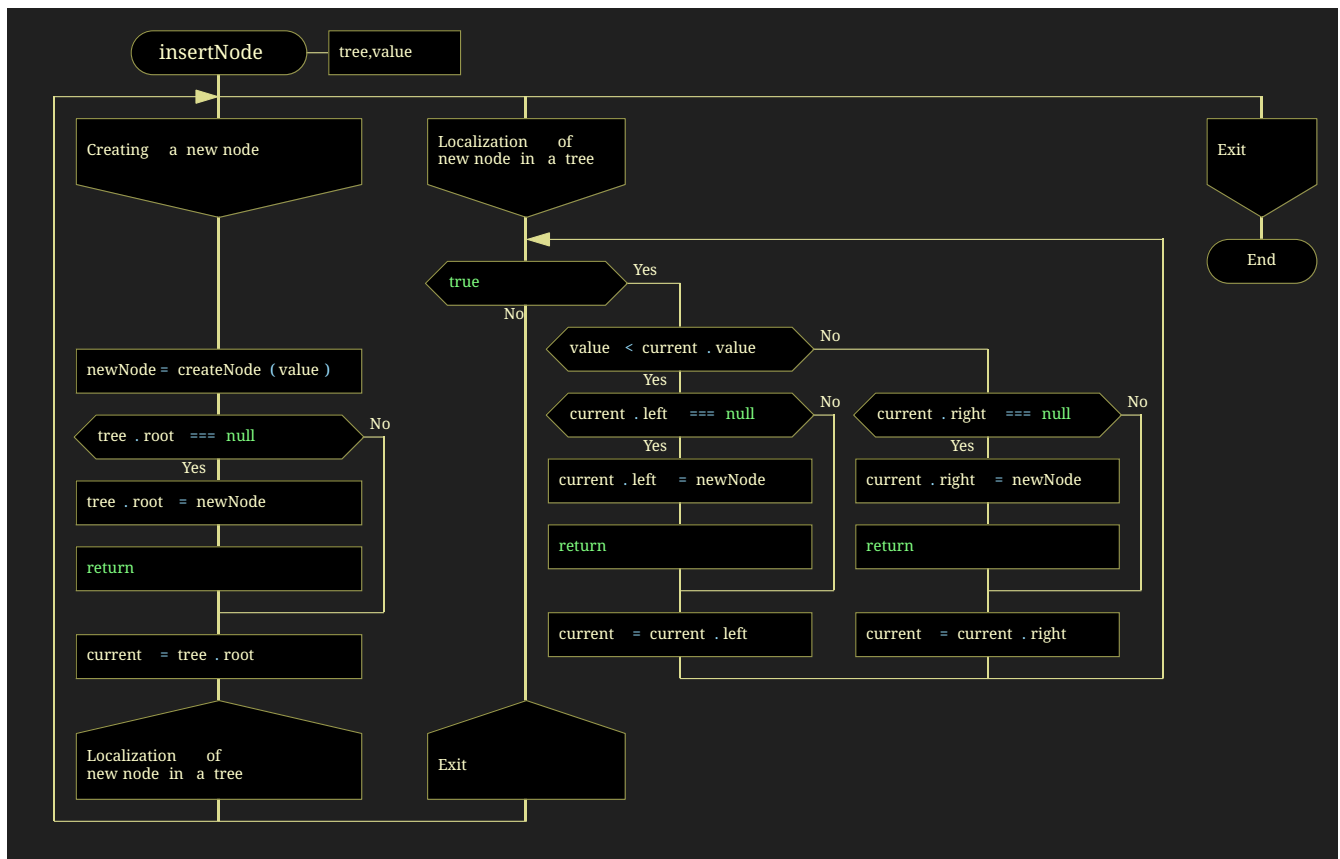
8.2. 8.2. Binary Search Tree

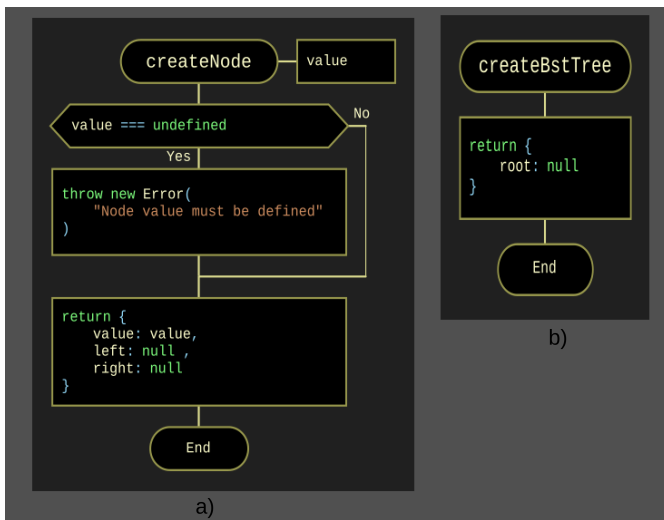
8.2.1. 8.2.1. Building a Binary Tree

If the tree is organized such that, for each node, all node values of its left subtree are less than this node and all values of its right subtree are greater, it is called a binary search tree (BST). A BST is a recursive data structure because each subtree is also a tree. A BST has the following properties:

- Each node retains a unique value.
- Each node has zero, one, or two child nodes.
- One node is referred to as the root node, which is at the top of the tree structure.
- Each node has only one parent node, except for the root node which has no parent.
- The value of each node is greater than that of its left child but less than that of its right child.

A binary search tree is built according to a specific algorithm. Consider the sequence of integers {11, 5, 17, 15, 1, 8, 19, 13, 21}, represented as a slice of numbers. First the root node {11} is formed, then in a loop for each node the method `insertNode(tree, value)` is called recursively (Figure 8.3a), which in turn calls the method `createNode(value)`, which creates a new node (Figure 8.3b) in the resulting tree (Figure 8.3c).





a)

Figure 8.3. Binary tree DRAGON-diagrams: a) building a tree by insert(node, value); b) creating a node

The complete process of creating a binary tree is shown in Figure 8.4. The first number 11 is written to the root of the tree. The second number 5 is less than the value in the tree root, so it is written to the left subtree. The next number 17 is greater than the root number, so it is written to the right subtree. Then the number 15 is greater than the value at the root of the tree, so it is written to the right subtree, but the right subtree is already built. The number 15 is compared to the number 17 at the root of the right subtree. Since the value to be added is less than the value in the root of the right subtree, we add the left subtree to this node. The final result is a binary search tree with three variants: node 5 is the parent of two children (1, 8), node 15 has only a left child, and node 19 has only a right child. This arrangement of nodes is chosen to demonstrate the node removal function, which will be discussed later.

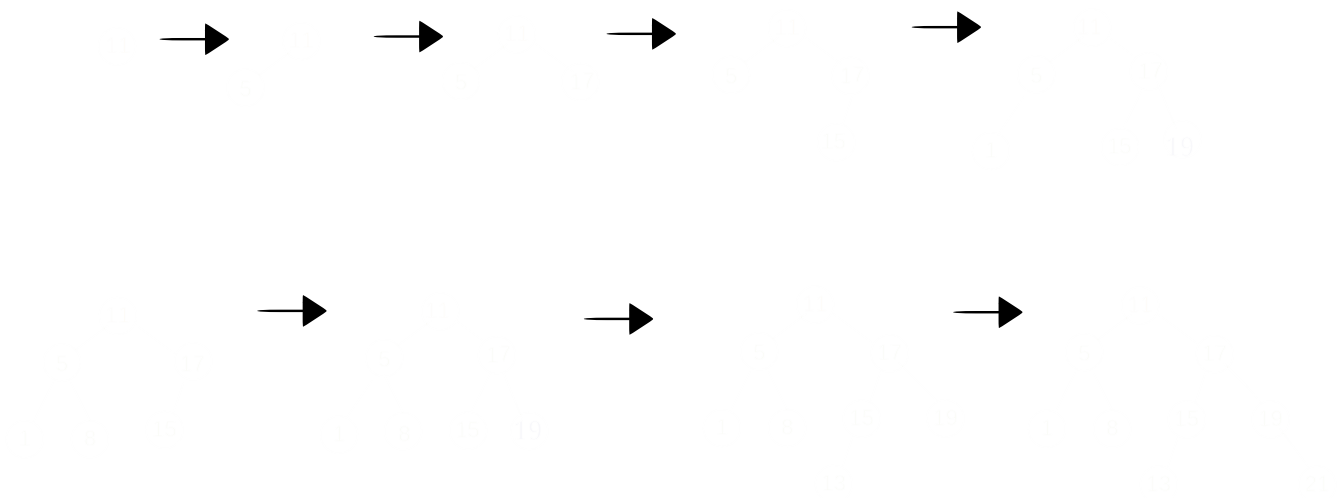


Figure 8.4. Building a binary tree

8.2.2. 8.2.2. Finding a Node Based on a Given Value

Another basic operation is the findNode(value) function. This function uses a JavaScript construct that recursively compares the value of val with the values of other nodes as it traverses the tree. The DRAGON-diagram of the findNode(node, value) function is shown in Figure 8.5.

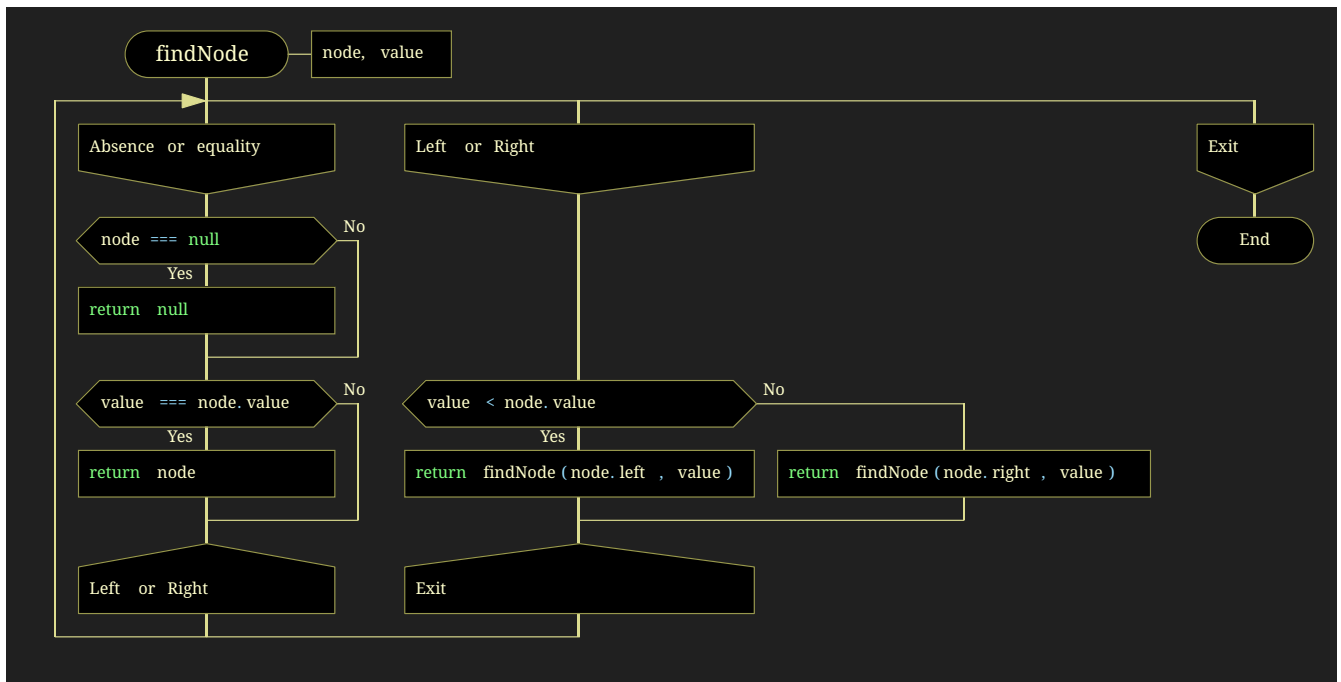


Figure 8.5. DRAGON-diagram of the node finding algorithm

8.2.3. 8.2.3. Deleting a Node with a Specified Value

The next basic operation is deleting a node with a specified value. The function `removeNode(node, value)` is used recursively.

Let us analyze in more detail the order in which nodes are moved in these variants.

Option a): Node (21) has no children (Figure 8.6).

In this case, this node is removed by changing the value of `node = null` in the parent node. Figure 8.6 shows the process of deleting node (21) and the corresponding DRAGON-diagram fragment.

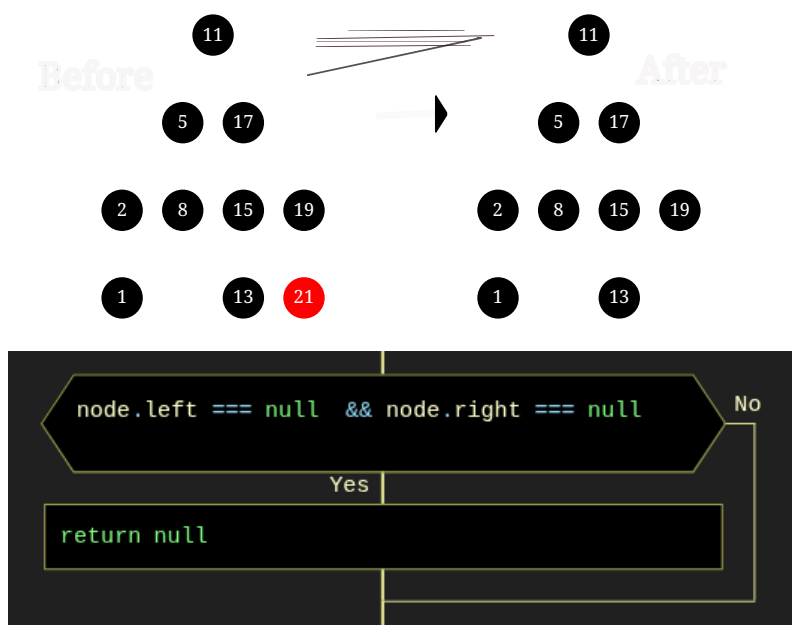


Figure 8.6. Deleting a node without children

Option b): Node (19) has a right child (Figure 8.7).

In this case, node (19) is removed from the tree by replacing its address in the parent node (17) with the address of its only child node (21). Figure 8.7 shows the process of removing node (19) and the corresponding DRAGON-diagram fragment.

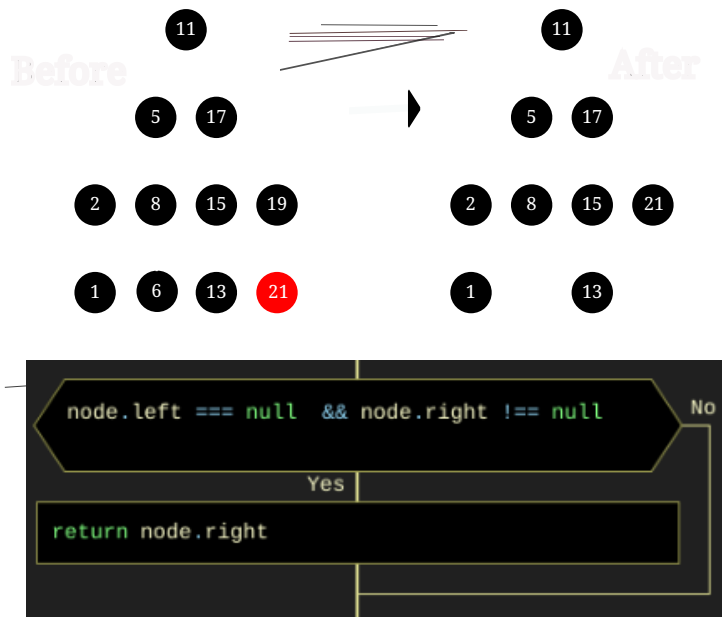


Figure 8.7. Deleting node (19) with a right child

Option c): Node (8) has a left child (Figure 8.8).

In this case, node (8) is removed from the tree by replacing its address in parent node (5) with the address of its only left child node (6). Figure 8.8 shows the process of removing node (8) and the corresponding DRAGON-diagram fragment.

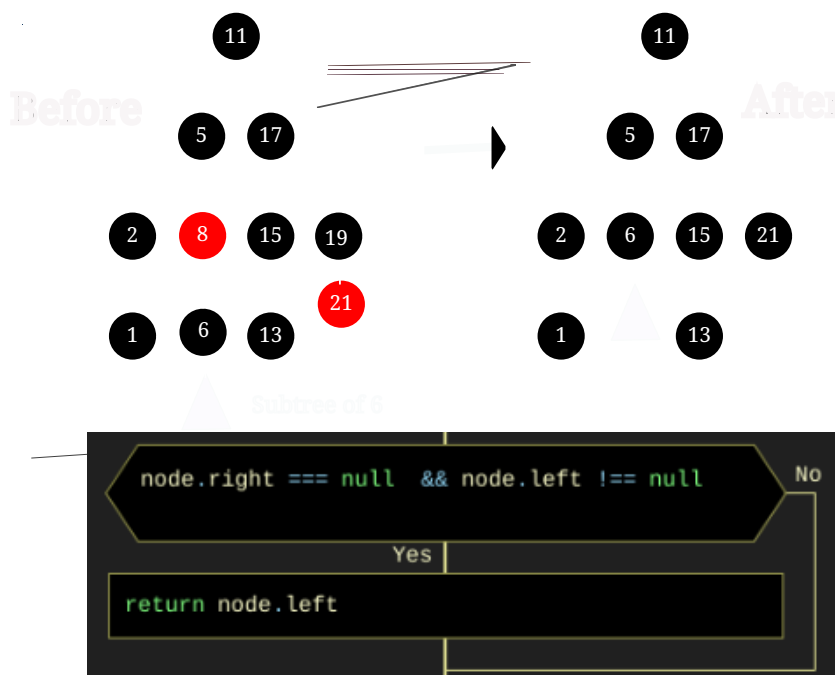


Figure 8.8. Deleting node (8) with a left child

Option d): Node (5) has two children (Figure 8.9). In this case the binary search tree is rearranged: node (2) moves to the place of node (5).

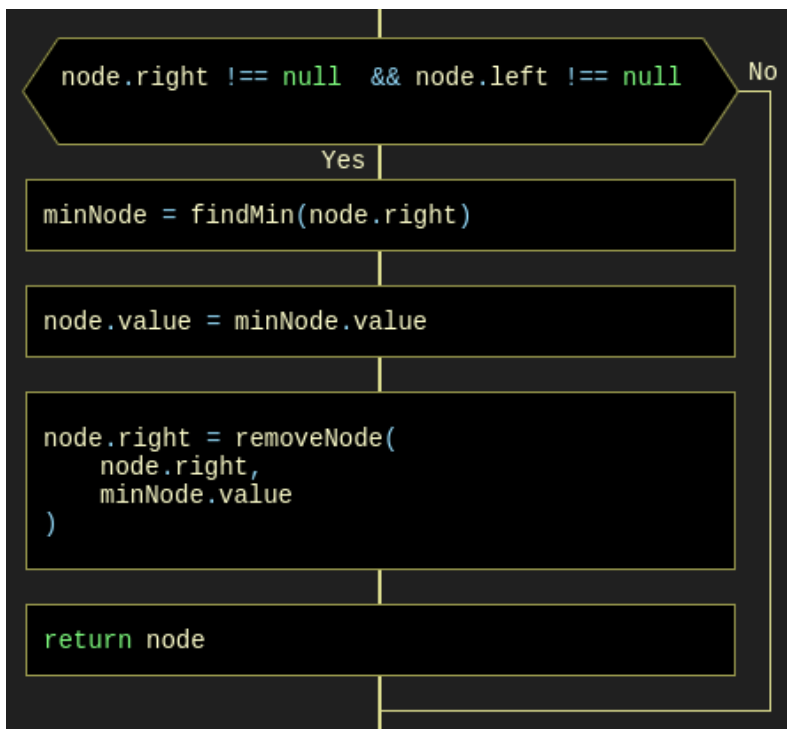
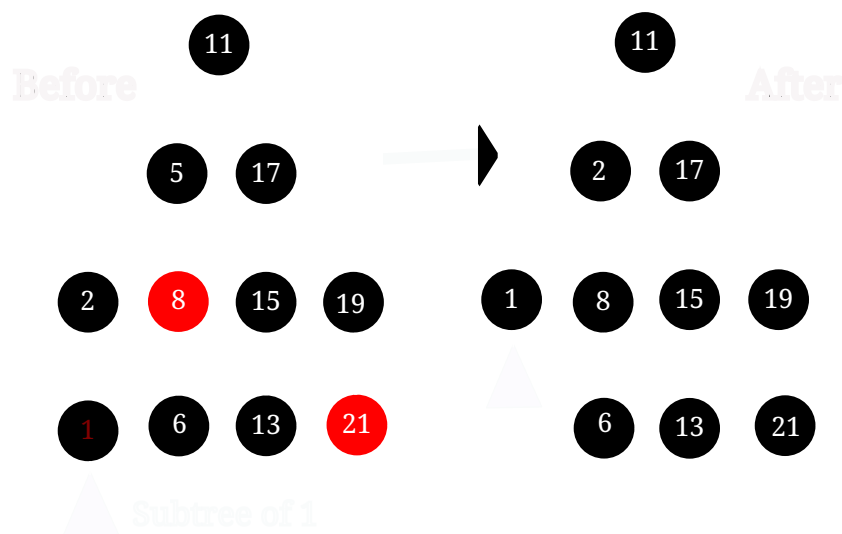


Figure 8.9. Deleting node (5) with two children

The DRAKON-diagram of the minimum value search method is shown in Figure 8.10.

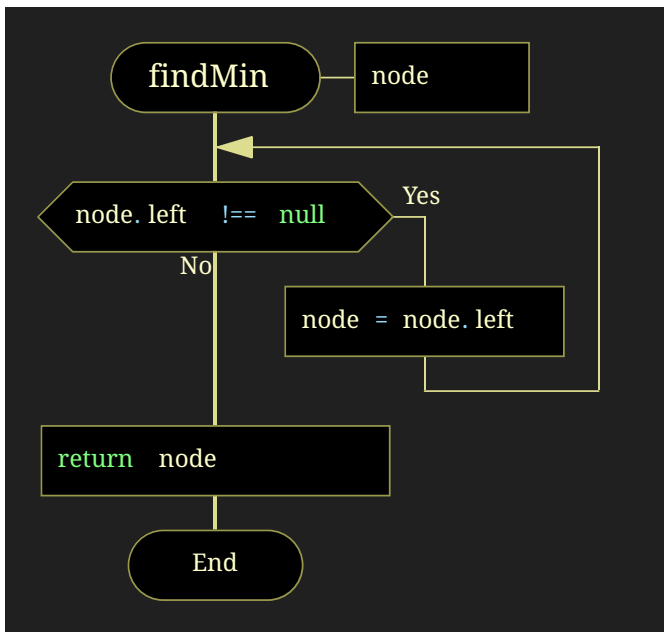


Figure 8.10. DRAKON-diagram of the minimum value search method

The complete DRAKON-diagram of node removal from a binary search tree is shown in Figure 8.11.

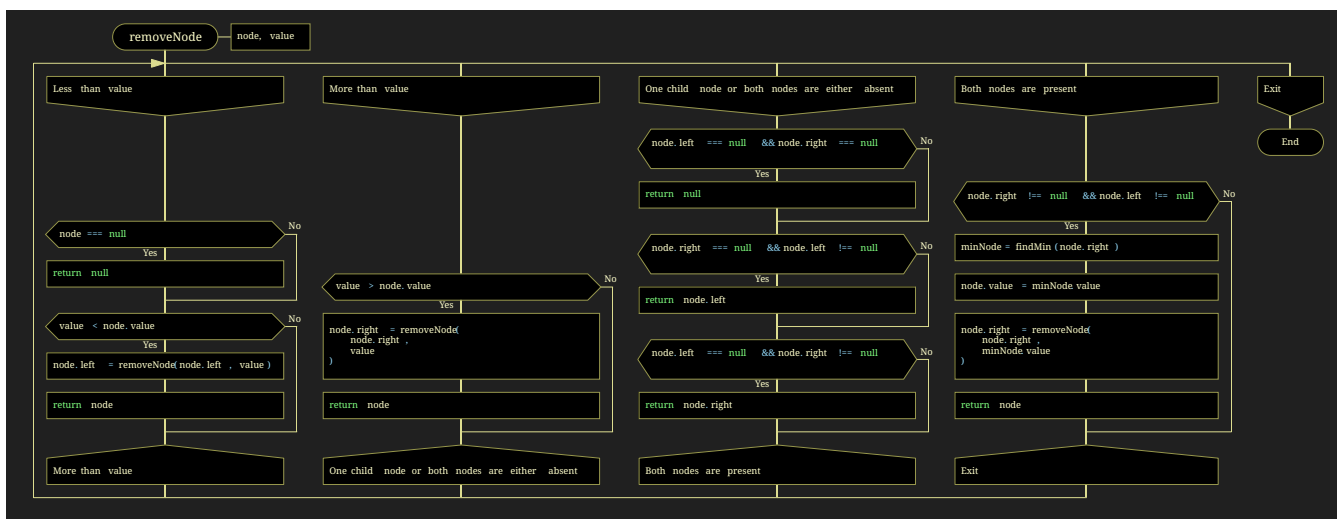


Figure 8.11. DRAKON-diagram of node removal

8.2.4. 8.2.4. Traversing the Binary Search Tree

One of the basic tree operations is traversing tree nodes. Unlike linear data structures, where elements are traversed in a linear order, tree nodes can be traversed in a variety of ways.

- Pre-order walk: Each ancestor node is visited before its descendants.
- In-order walk: The left subtree is visited from bottom to top, then the root node, then the right subtree.
- Post-order walk: Descendants are visited first, then ancestors.

All three cases, including the DRAKON diagrams, are shown in Figure 8.12.

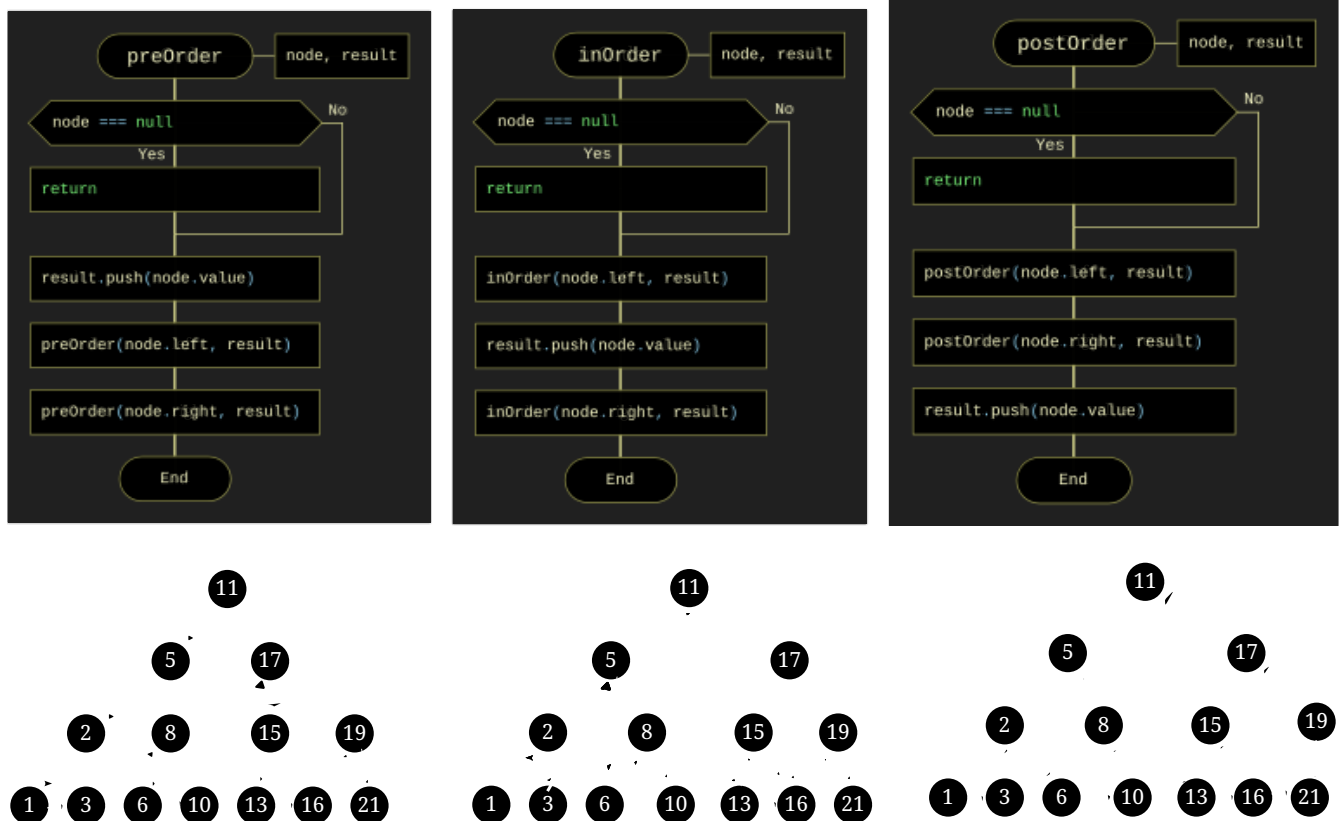


Figure 8.12. Three classical tree traversals: Pre-, In-, and Post-order

8.3. Self-Balancing Binary Trees (AVL Trees)

The efficiency of binary tree processing algorithms significantly depends on the order in which node values are provided during tree construction. In the most favorable sequence, the binary tree becomes balanced. In the worst-case sequence, the "tree" degenerates into a "branch" (Figure 8.13). As we will see below, restoring balance in such cases reduces to four canonical rotation patterns — right, left, right-left, and left-right rotation.

Figure 8.13. Most favorable (a) and worst-case (b) input data sequences

In such practically "degenerate" trees, the complexity of operations is determined by the number of nodes — that is, it is almost linear: $O(n)$. The left and right subtrees are unbalanced, which can be estimated by the balance factor (bf), which is equal to the difference between the heights of the left and right subtrees. Recall that the height of a tree is defined as the length of the longest branch in a subtree (sum of edges). For an ideal binary search tree (a tree where the number of nodes in the left subtree equals the number of nodes in the right subtree), this factor is 0 (Figure 8.14).

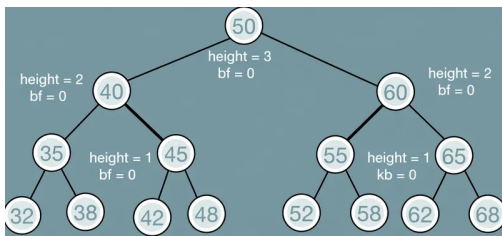


Figure 8.14. An ideal binary tree

In real life, ideal binary trees are almost never achieved. Programmers often try to build a tree where the height of the left subtree differs from the height of the right subtree by no more than 1. Such trees are called AVL trees; for these trees the complexity of operations is defined as $O(\log n)$, meaning the execution time of basic operations (search, delete) is significantly less than for BST trees.

The algorithms for such trees are based on the process of balancing the tree when a new node is inserted or an existing node is removed. The purpose of balancing is to reconstruct the tree so that the heights of the left and right subtrees do not differ by more than 1. The balance factor must satisfy the following conditions:

- The allowable values are $bf = -1, 0$, and $+1$.
- $bf = -1$ indicates that the right subtree "outweighs" the left subtree.
- $bf = +1$ indicates that the left subtree "outweighs" the right subtree.
- $bf = 0$ indicates that the tree contains an equal number of nodes on each side — that is, the tree is perfectly balanced.

The balancing algorithm in its most general form is presented in the following DRAGON diagram (Figure 8.15):

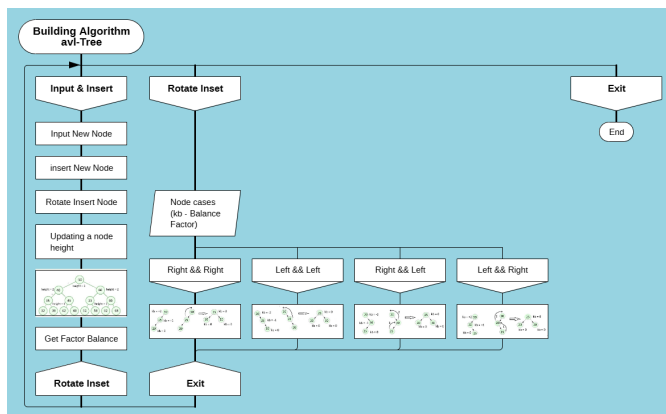


Figure 8.15. The balancing algorithm in its most general view

The balancing technique boils down to making

circular movements of the nodes in four variants:

- Right rotation (RR)
- Left rotation (LL)
- Right-left rotation (RL)
- Left-right rotation (LR)

Right rotation is performed when the root node has a balance factor $bf = +2$ and its left child has a balance factor $bf = +1$ (Figure 8.16):

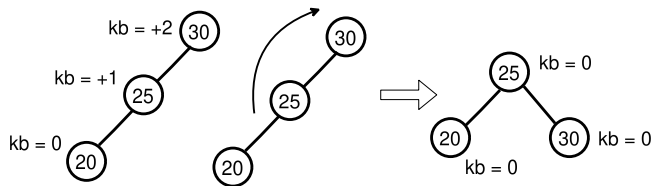


Figure 8.16. Right rotation

Left rotation is performed when the root node has a balance factor $bf = -2$ and its right child has a balance factor $bf = -1$ (Figure 8.17):

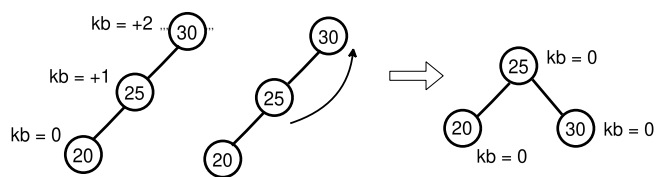


Figure 8.17. Left rotation

Right-left rotation is performed when the root node has a balance factor $bf = -2$ and its right child has a balance factor $bf = +1$ (Figure 8.18):

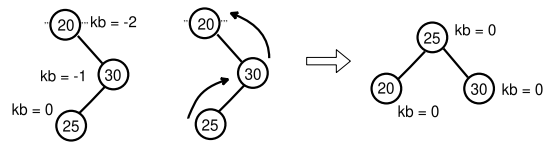


Figure 8.18. Right-left rotation

Left-right rotation is performed when a node has a balance factor $bf = +2$ and its left child has a balance factor $bf = -1$ (Figure 8.19):

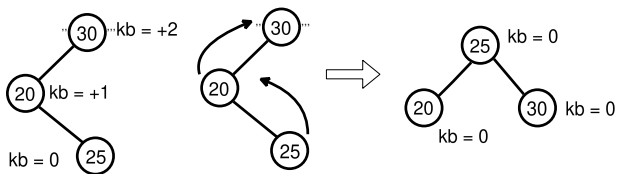


Figure 8.19. Left-right rotation

Consider the balancing process in more detail

with an example of tree formation when input data is received in this order: [1, 2, 5, 6, 8, 11, 13, 15, 17, 19, 21]. As shown above, such a tree is unbalanced.

Balance checks are started on the arrival of each new node using the `insertNode(node, value)` function, which determines the position of the new node in the left or right subtree relative to the root node. The choice of the balancing path is determined by the method `rotateInsert(node, value)`. A DRAKON-diagram of the algorithms of the above methods is shown in Figure 8.20.

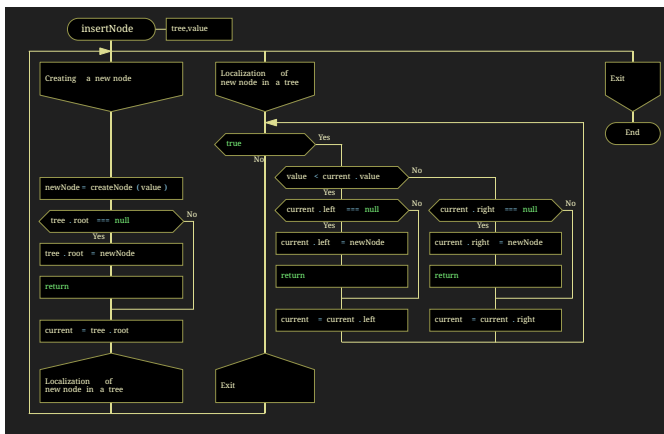


Figure 8.20. DRAKON-diagram depicting the method of inserting a node

The choice of one of the possible balancing paths (rotations) is determined by the rotateInsert(node, val) function (Figure 8.21).

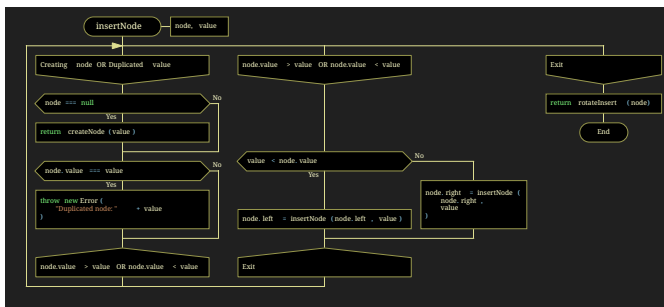


Figure 8.21. DRAKON-diagram depicting the rotation selection method

Depending on the location of the new node relative to the parent node and the balance factor values, the left-hand or right-hand rotation methods are performed (Figure 8.22):



Figure 8.22. Node rotation program codes

Let us examine the tree rebuilding process in detail as a result of using the `rotateInsert(node, val)` method on the example of entering the values of three nodes (5, 6, 8). After receiving the value of the last node (8), a "twig" appears instead of a tree that needs to be balanced. In this case the condition in the `rotateInsert(node, val)` method is met: the balancing factor is 2 ($bf = 2$) and the value of node (8) is greater than the parent node (6), so the `rotateLeft(node)` method

is called. The balancing process in this case is shown in Figure 8.23:

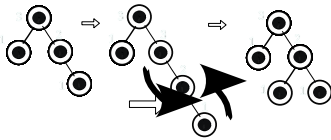


Figure 8.23. Example of left-left rotation balancing

The process of new nodes arriving and rebuilding the resulting AVL tree by balancing for the input set [1, 2, 5, 6, 8, 11, 13, 15, 17, 19, 21] is shown in Figure 8.24.

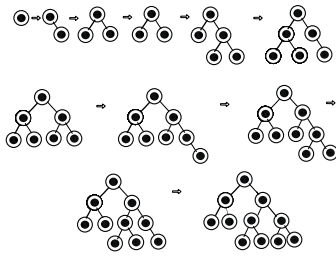


Figure 8.24. Process of building a balanced AVL tree

The DRAKON-diagram of the AVL tree basic operation `findNode(node, val)` is shown in Figure 8.25:

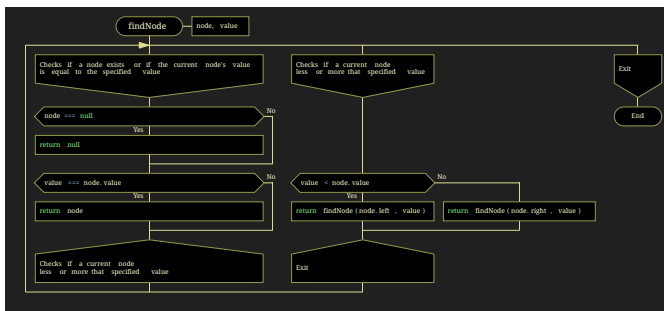


Figure 8.25. DRAKON-diagram depicting the node finding method

Another basic operation, deleting a node with a specified value, consists of the following steps. The node is searched from the root down through the branches to the node to be deleted. The following situations may occur (Figure 8.26):

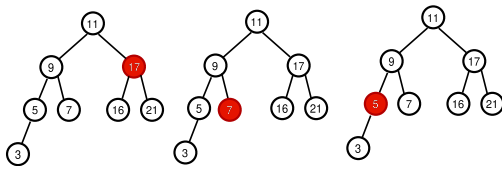


Figure 8.26. Cases for nodes to be removed

a) The node to be deleted has two non-empty children. b) The node to be deleted has no children. c) The node to be deleted has one child (left or right).

As with the other methods, a node with a given value is first recursively identified and then one of the options presented is selected. A DRAKON-diagram of the algorithm for deleting a node with a specified value is shown in Figure 8.27.

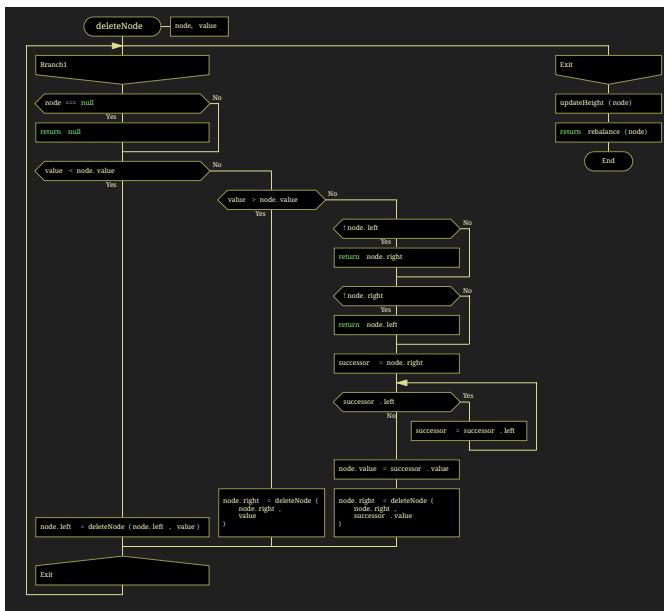


Figure 8.27. DRAKON-diagram depicting an algorithm for removing a node with a specified value

The `main()` program inputs an array of data, searches for a node with a specified value, and deletes a node with a specified value (Figure 8.28).

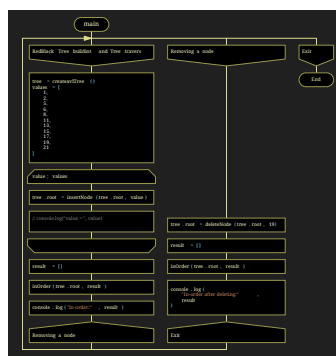


Figure 8.28. DRAKON-diagram of the main function

8.4. 8.4. Red-Black Trees

8.4.1. 8.4.1. Description of Red-Black Tree Properties as a Data Structure

A red-black tree is a variant of a self-balancing binary search tree in which the nodes are colored red or black according to a specific set of rules (Figure 8.29).

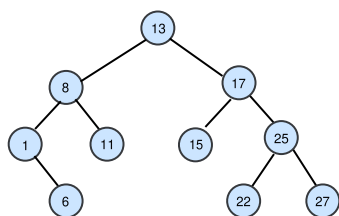


Figure 8.29. Red-black tree example

The nodes containing data (integers in this example) are called internal nodes. In addition, red-black trees contain imaginary "null" nodes associated with tree leaves, which are explicitly colored black (labeled "Null" in Figure 8.29). Red-black trees satisfy all properties of a binary search tree and must maintain the following specific invariants:

1. Each node is colored either red or black.
2. The root of the tree must be black. (Note: this invariant may be temporarily violated during the balancing process but must be restored before the operation completes.)
3. All leaves (NIL nodes) are black.
4. Both children of a red node must be black. Consequently, no two red nodes can appear

consecutively on any path.

5. Every simple path from a node to any of its descendant leaves must contain the same number of black nodes.

Unlike AVL trees, where balance is enforced by limiting the height difference between subtrees, red-black tree balance is achieved by strictly enforcing the properties listed above. Inserting or removing a node can disrupt these invariants. Balance is restored using two primary operations: recoloring nodes and/or restructuring the tree (or subtrees) using specific rotations. The most critical of these constraints are Properties 4 and 5.

- **Property 4** mandates that two red nodes cannot be adjacent.

- **Property 5** builds on this by enforcing a uniform black height for all paths from any node to its leaf descendants. Regardless of the specific path taken, the count of black nodes remains constant.

Mechanical Analogy for Balancing: The Chandelier and the Balancing Machine

Understanding the dynamic enforcement of red-black properties is crucial for developing an efficient balancing algorithm. However, this often presents significant challenges, as it requires tracking the state of multiple nodes simultaneously.

To facilitate conceptual clarity, we employ a mechanical analogy. Since the "root" of this data

structure is positioned at the top, a chandelier serves as an appropriate mechanical analogue — an elegant hanging structure where each node is a ball of a strictly defined color and weight, rigidly connected to form branches. The root is fixed at the top, and the weightless NIL leaves hang freely downward.

In this model:

- **Black balls** are solid and heavy, providing stability and physical equilibrium. Their weight is defined as 1 unit.
- **Red balls** are hollow, decorative, and virtually weightless (0 units). They serve as a flexible resource that can be "hardened" into black balls (weights) when necessary.

All NIL leaves are considered black and weightless termination points.

The structural equilibrium (balance) is defined by two simple conditions, which restate Properties 5 and 4 respectively in mechanical terms:

- The total weight (the number of black balls) on any path from the root to any leaf must be equal — this is Property 5, the uniform black height.
- Two red balls in a row are forbidden — this creates a "weak spot" in the chain, even though it does not alter the branch's overall weight; this is Property 4.

Black height is the fundamental metric in this engineering model, representing the total

"weight" (the number of black nodes) of a specific branch. Properties 4 and 5 work together to ensure structural integrity. While red nodes allow branches to stretch, the rules guarantee that the longest path (alternating black weights and red spacers) is never more than twice the length of the shortest path (consisting only of black weights). This constraint ensures that "tension" is evenly distributed, keeping the tree approximately balanced with a height of $O(\log n)$, where n is the number of nodes.

The entire lifecycle of the structure is managed by a virtual balancing automaton. It possesses an unlimited stock of spare black and red balls and performs four functions:

Insertion

The automaton hangs a new red ball (NIL replacement).

Deletion

It removes a ball and replaces it with a dummy, triggering rebalancing if necessary.

Balance Monitoring

After every insertion or deletion, it checks for equilibrium violations.

Balancing

If a violation is detected, the automaton performs local transformations: recoloring balls and/or rearranging connections (rotations) using its stock of spare parts.

To implement these operations, we define two custom data types: node and rbTree. A node includes fields for the key (numeric value), color (boolean: true for red, false for black), and pointers to left, right, and parent nodes. The rbTree structure is a reference-type variable that stores a pointer to the memory area containing the tree objects (Figure 8.30).



Figure 8.30. DRAKON-diagram of Node and rbTree description

Cluster Node Notation

To analyze the balancing process, we introduce an original cluster-based notation, developed specifically for this book, which focuses on a local C-Cluster of nodes involved in a transformation. This notation is essential for understanding the transformations applied by the automaton. The following designations are used (Figure 8.31):

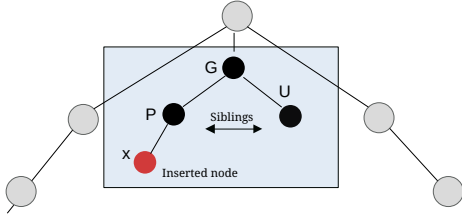


Figure 8.31. Designation of "kinship" degrees of RB tree nodes (Cluster x, P, G, U, S)

In this model:

x

The newly inserted red node.

P

Parent of **x**.

G

Grandparent of **x** (parent of **P**). In our model, **G** is always a solid 'weight' (black) before fixup begins.

U

Uncle of **x** (sibling of **P**). A missing (null) uncle is treated as a static black 'weight' (NIL leaf).

S

Sibling of x (the other child of P). Its color and position are crucial for determining the necessary rotation type.

For a compact description of the cluster arrangement, we introduce the following symbolic notation:

- A colon $:$ separates the node name from its color ($P:B$ describes a black parent).
- The comparison symbols $<$ and $>$ indicate relationship direction ($x<P$ means x is P 's left child).
- Commas and semicolons delimit attributes.

For example, the notation $x:R, x<P; P:R, P>G; G:B;$

U:R; U>G; describes a specific "zig-zag" violation where red node x is the left child of its red parent P , which in turn is the right child of the black grandparent G , who also has a red uncle U . Cluster nodes can be in a "zig-zig" (e.g., $x < P < G$) or "zig-zag" (e.g., $x < P > G$) configuration.

This concise notation significantly simplifies the balancing algorithms, making the transformation criteria visual and efficient.

8.4.2. 8.4.2. Inserting a New Node and the Balancing Process

A new node is inserted into the red-black tree using the `rbInsert(tree, key)` function, which follows standard BST rules and always adds a new red node. However, this addition may

introduce a "Double Red" conflict (violating Property 4). The rebalancing process is summarized in the mindmap in Figure 8.32.

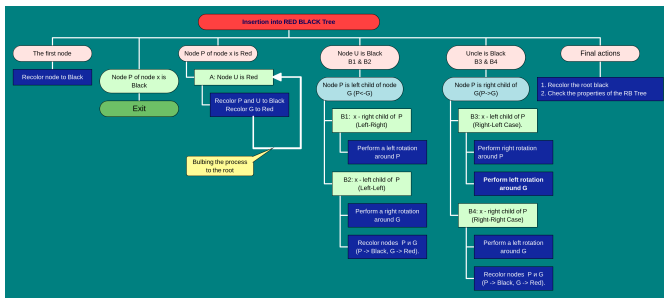


Figure 8.32. Mindmap of the insertion and balancing process

To restore balance, the automaton analyzes the local C-Cluster. Based on the colors and positions of the nodes, it executes the required fix: recoloring (replacing balls) or rotation

(rearranging connections). After a local fix, the problem is either resolved or propagates one level up the tree (e.g., a newly colored red ball now sits under a red parent), requiring the automaton to continue its work on the new cluster.

Balancing after insertion is categorized into cases based on the cluster's configuration. Table 8.1 summarizes these cases, their associated violations, unique signatures, fix criteria, and the mechanical actions applied by the automaton.

Table 8.1. Balancing during insertion in RBT

Cas e	Violation	Action Type	Cluster Descriptor	Ref ere nce Fig ure
Uncle is Red (Violation State)				
Case A	Double Red (U:R)	Recolorin g	x:R; P:R; G:B; U:R	Fig8 _33a _A_ Viol atio n.sv g
Uncle is Black (Violation State)				

Cas e	Violation	Action Type	Cluster Descriptor	Ref ere nce Fig ure
Case B1	Double Red (U:B)	Zig-Zig (LL)	x:R, x<P; P:R, P<G; G:B; U:B, U>G	Fig8 _33 b_B 1_L L_Vi olati on.s vg

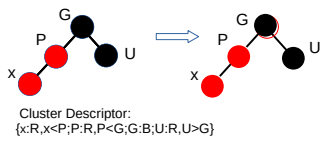
Cas e	Violation	Action Type	Cluster Descriptor	Ref ere nce Fig ure
Case B2	Double Red (U:B)	Zig-Zig (RR)	x:R, x>P; P:R, P>G; G:B; U:B, U<G	Fig8 _33c _B2_ RR_ Viol atio n.sv g

Cas e	Violation	Action Type	Cluster Descriptor	Ref ere nce Fig ure
Case B3	Double Red (U:B)	Zig-Zag (LR)	x:R, x>P; P:R, P<G; G:B; U:B, U>G	Fig8 _33 d_B 3_L R_Vi olati on.s vg

Cas e	Violation	Action Type	Cluster Descriptor	Ref ere nce Fig ure
Case B4	Double Red (U:B)	Zig-Zag (RL)	x:R, x<P; P:R, P>G; G:B; U:B, U<G	Fig8 _33e _B4_ RL_ Viol atio n.sv g

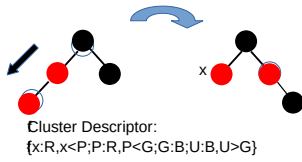
The visualizations of these distinct violation states — which serve as the 'Atlas' for insertion — are presented in Figures 8.33a-e.

Case A: Two adjacent red nodes



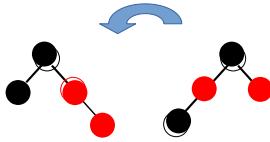
a) Case A — Recoloring (Red Uncle Violation)

Case B1: zig-zig (LL)



b) Case B1 — Zig-Zig (LL Double Red Violation)

Case B2: zig-zig (RR)

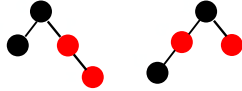


Cluster Descriptor:

$\{x:R, x>P; P:R, P>G; G:B; U:B, U<G\}$

c) Case B2 — Zig-Zig (RR Double Red Violation)

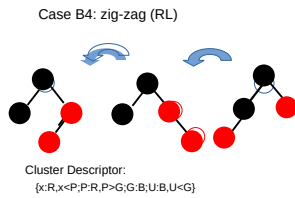
Case B3: zig-zag (LR)



Cluster Descriptor:

$\{x:R, x>P; P:R, P<G; G:B; U:B, U>G\}$

d) Case B3 — Zig-Zag (LR Double Red Violation)



e) Case B4 — Zig-Zag (RL Double Red Violation)

Figure 8.33. Atlas of options (violation states) for balancing after node insertion

The complete logic for red-black tree construction is encapsulated in the `rbInsert` function (Figure 8.34). It merges standard BST insertion with a call to the specialized `fixInsert` balancing routine to meticulously restore tree invariants.

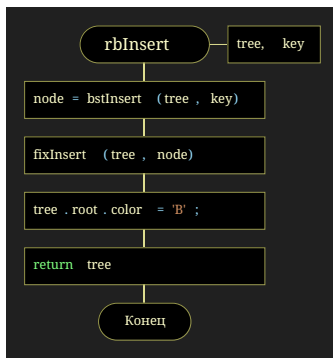


Figure 8.34. DRAKON-diagram of the function `rbInsert(tree, data)`

The complete decision logic of the balancing process is mapped out in the master DRAKON-diagram shown in Figure 8.35.

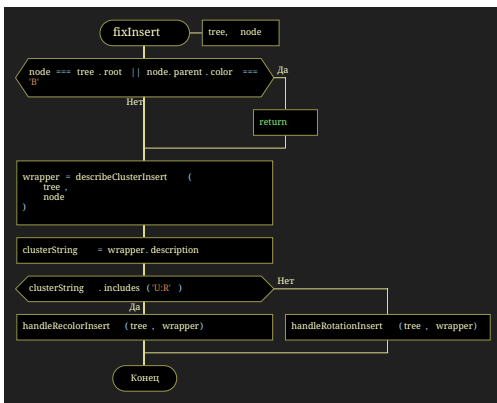


Figure 8.35. DRAKON-diagram of the tree balancing function `fixInsert(tree, data)`

The internal logic for the recoloring handler (Case A, Red Uncle) is shown in Figure 8.36, while the logic for the rotation handlers (Cases B1-B4, Black Uncle) is detailed in Figure 8.37.



Figure 8.36. DRAKON-diagram of the recoloring algorithm for a red uncle

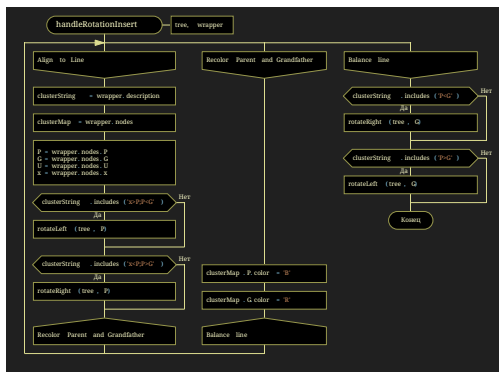
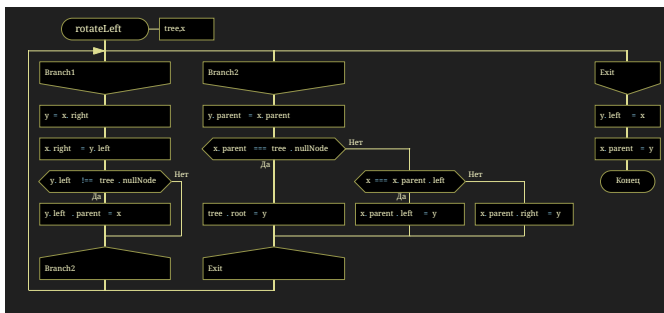
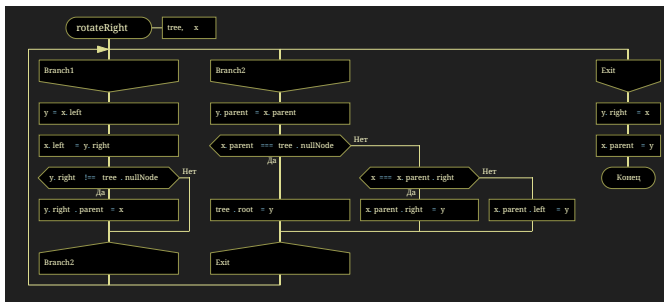


Figure 8.37. DRAKON-diagram of the rotation algorithms for a black uncle

The fundamental rotation primitives used by these handlers — the "building blocks" of restructuring — are detailed in Figures 8.38a and 8.38b.



a) Left-rotation primitive



b) Right-rotation primitive

Figure 8.38. DRAKON-diagrams of the fundamental rotation primitives

TIP

Deepen your understanding: experiment with the dedicated online visualization and simulation tool, Red-Black Tree Visualizer. This interactive platform allows you to insert and delete nodes, observe the balancing operations in real time, and visualize the tree structure dynamically. Engaging with this tool will enhance your comprehension of the balancing mechanisms described above.

8.4.3. 8.4.3. Deleting a Node: Restoring the Weight Balance

Node deletion in a red-black tree is a sophisticated operation because it must preserve the tree's "gravitational" balance. However, by adhering to our analogy of weights (black nodes) and zero-gravity spacers (red nodes), the logic becomes intuitive: complexity only arises when we lose a "weight."

The standard mechanical notation for the C-Cluster used in deletion operations is shown in Figure 8.39:

z

The node to be physically deleted, which triggers the Weight Debt (the "Double-Black"

focus).

p

Parent of **z**.

s

Sibling of **z**.

s.l, s.r

The left and right children of sibling **s** (the "nephews").

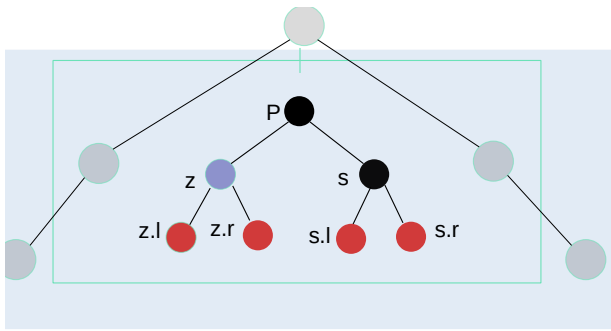


Figure 8.39. Cluster deletion nodes and their designations (z, P, s, s.l, s.r)

The complete deletion and rebalancing process is summarized in the mindmap in Figure 8.40.

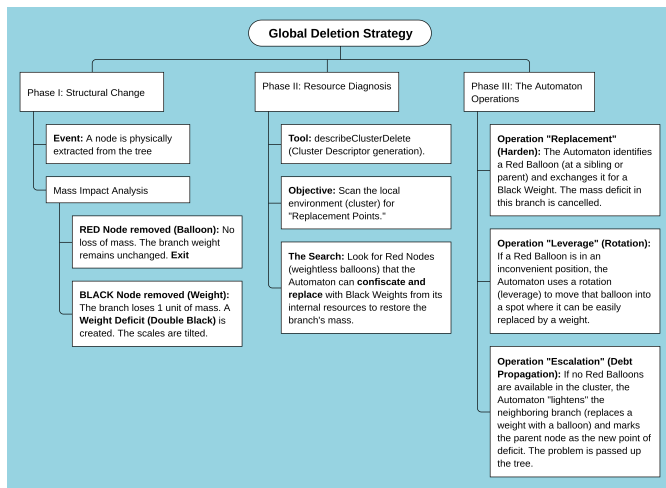


Figure 8.40. Mindmap summarizing RB tree deletion operations

The operation consists of three primary steps:

1. **Identification:** As in any standard BST, the

algorithm first locates the target node (z).

2. **Structural Deletion and Replacement:** We must physically remove the node in a way that minimizes damage:

- If node z has at most one child: we remove z and replace it with its non-NIL child (or a NIL node).
- If node z has two children: we cannot delete it directly. Instead, we find its inorder successor (the minimum node in its right subtree), swap their values, and then delete the successor. This ensures that the node being physically removed always has zero or one child.

3. **The Color Impact (The Weight Factor):**

Rebalancing necessity is determined entirely by the color of the physically deleted component:

- Deleting a Red Node: if the deleted node is red, no action is required. As red nodes are weightless, their removal does not alter any black-height paths, and Property 5 remains intact.
- Deleting a Black Node: if the deleted node is black, we have a problem. We have removed a "brick" from one side of the scale. This branch is now "lighter," violating Property 5 and creating a Weight Deficit.

This deficit is the core of the "Double Black" problem. It signals to our balancing automaton

that it must either:

1. Find a spare red node in the vicinity and 'harden' it into a weight.
2. Or redistribute weights from neighboring branches to fill the gap.

The "Double Black" Concept (The Weight Debt):

The node that occupies the position of the physically deleted black node is temporarily designated as "Double Black" (DB). In our chandelier analogy, this is a Weight Debt — a conceptual void indicating the path through this node is "lighter" than Property 5 requires. Correcting this violation employs a sequence of recoloring and rotations:

Weight Transfer

Shifting a black weight from a neighboring branch via rotation.

Color Hardening

Transforming a red node into a black weight to fill the void.

Debt Propagation

If the local cluster cannot cancel the void, the "debt" is moved up toward the parent. The parent then becomes the new focus of the balancing process until the deficit is cancelled at the root or resolved by a rotation.

The atlas of options for removing a node from the RB tree is presented in Figure 8.41.

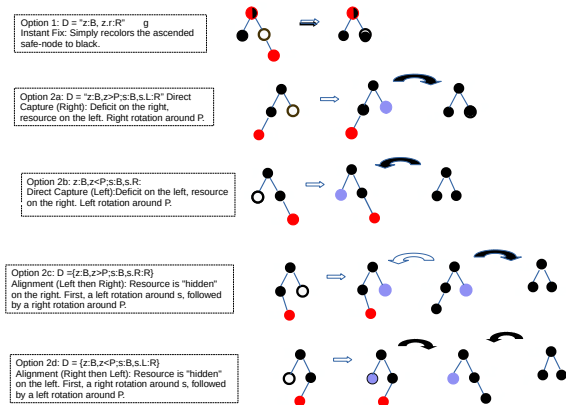


Figure 8.41. Atlas of options for node deletion from the RB tree

The DRAKON-diagram of the deleteNode function, which implements this complex structural logic, is shown in Figure 8.42.

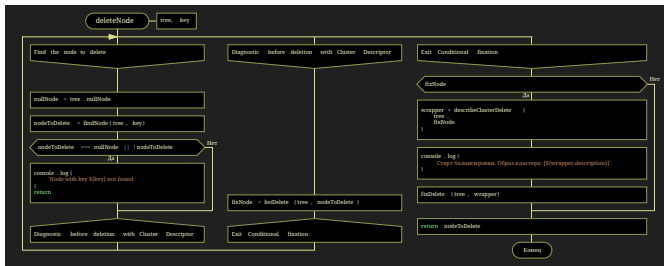


Figure 8.42. DRAKON-diagram of the function

`deleteNode(tree, data)`

To streamline rebalancing, a separate utility function analyzes the cluster state. The DRAKON-diagram for `deleteClusterDescriptor`, which generates the unique 'signature' string, is shown in Figure 8.43.

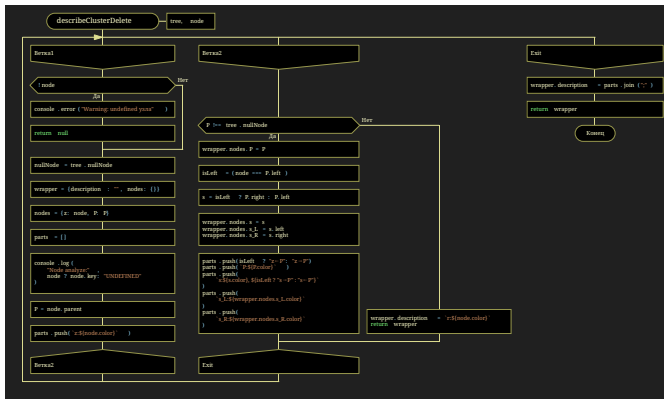


Figure 8.43. DRAKON-diagram of the descriptor function `deleteClusterDescriptor(tree, data)`

Once the cluster is analyzed, the balancing automaton function `fixDelete(tree, data)` acts upon the result. It makes all high-level balancing decisions based on the descriptor signature. Its DRAKON-diagram is presented in Figure 8.44.

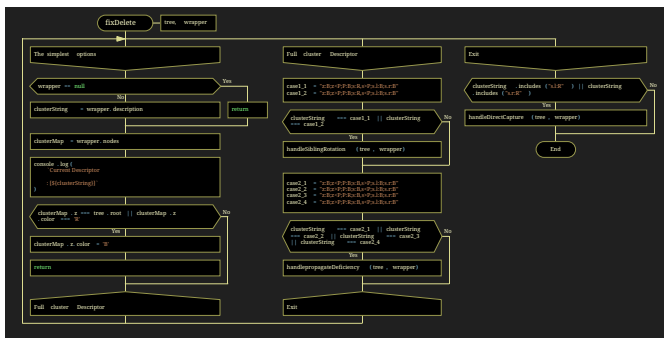


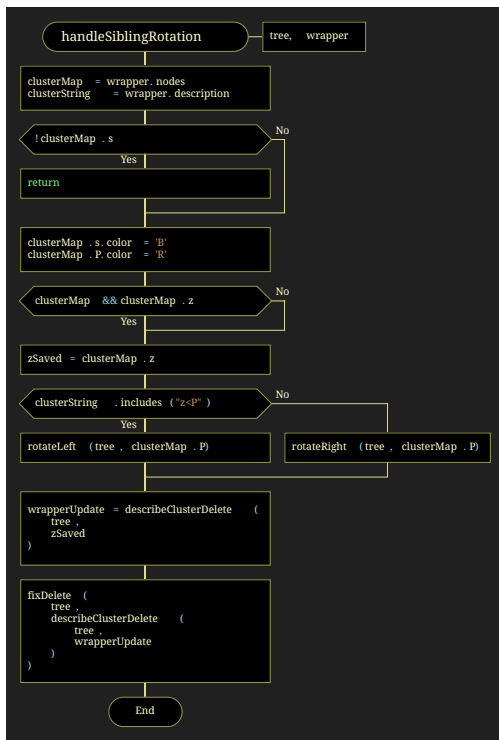
Table 8.2. Mapping automaton purpose to specialized function names (handle types)

Automaton Purpose in the Atlas	Proposed Function Name (handle type)
Maneuver with a Red Sibling (s:R) Goal: Make sibling s black so the automaton can capture a resource in the next step. The deficiency is not resolved.	handleSiblingRotation

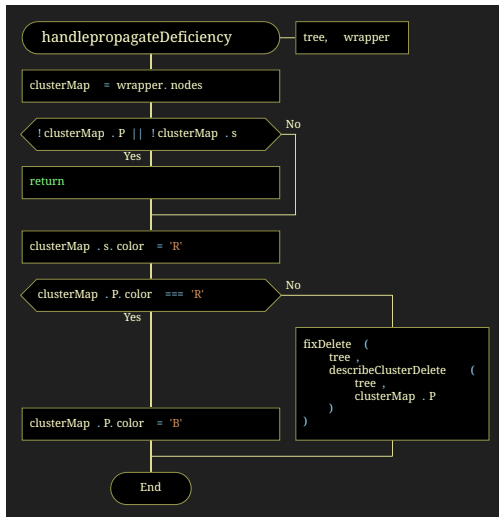
Automaton Purpose in the Atlas	Proposed Function Name (handle type)
Deficiency Propagation (s:B, no resources) Goal: "Dissolve" the deficiency (z) by recoloring sibling s to red, and move the deficiency (blue tag) to parent P.	propagateDeficiency

Automaton Purpose in the Atlas	Proposed Function Name (handle type)
Final Black Height Recovery (s.l:R or s.r:R) Goal: Find and capture a red resource (s.l or s.r) by performing rotations and recoloring. The deficiency is completely resolved (Options 2a-2d).	handleDirectCapture

The complete DRAKON-diagrams detailing the internal logic of these three specialized functions are shown in Figures 8.45a, 8.45b, and 8.45c.



a) handleSiblingRotation



b) propagateDeficiency

8.5. 8.5. Comparison of Tree Structures

A clear comparison of the dynamic behavior of each tree structure is presented in Table 8.3. While standard BST implementations risk degrading into linear structures with $O(n)$ complexity, self-balancing structures (AVL and red-black) guarantee $O(\log n)$ complexity, making them scalable for large datasets. Note that although red-black trees have dynamic complexities in insertions and deletions compared to BSTs, they eliminate the critical $O(n)$ failure mode, providing a predictable $O(\log n)$ experience.

Table 8.3. Comparison of time and space

complexities for tree operations

Tree Type	Operation	Average Time Complexity	Worst-Case Time Complexity	Space Complexity
BST	Insert	$O(\log n)$	$O(n)$	$O(n)$
	Find	$O(\log n)$	$O(n)$	$O(n)$
	Delete	$O(\log n)$	$O(n)$	$O(n)$
AVL Tree	Insert	$O(\log n)$	$O(\log n)$	$O(n)$
	Find	$O(\log n)$	$O(\log n)$	$O(n)$
	Delete	$O(\log n)$	$O(\log n)$	$O(n)$

Tree Type	Operation	Average Time Complexity	Worst-Case Time Complexity	Space Complexity
Red-Black Tree	Insert	$O(\log n)$	$O(\log n)$	$O(n)$
	Find	$O(\log n)$	$O(\log n)$	$O(n)$
	Delete	$O(\log n)$	$O(\log n)$	$O(n)$

8.6. 8.6. Conclusion

Trees are one of the most important data structures in computer science, used to represent hierarchical relationships and organize data. They provide efficient search, insert, and delete operations, making them ideal for use in a variety

of applications.

AVL trees, named after their creators Adelson-Velsky and Landis, are balanced binary search trees. They provide fast search, insertion, and deletion by keeping the height of the tree logarithmic relative to the number of nodes. However, AVL trees require additional insertion and deletion operations to maintain balance, which can be a disadvantage in some applications.

Red-black trees are another type of balanced binary search tree. They provide efficient search, insert, and delete operations, but can be more difficult to implement because of the additional properties they must support. They are widely used in many standard libraries.

The choice of a specific tree type depends on the specific requirements of the application. AVL trees are generally preferred when frequent searches are the most important operations, as they provide a more stringent balance than red-black trees. On the other hand, red-black trees can be more effective with frequent insertions and removals, as they require fewer rotations to maintain balance.

It is important to remember that both types of trees require additional memory to store balance information, and both can be difficult to implement. In some cases, other data structures, such as hash tables or B-trees, may be more appropriate. However, despite these challenges, trees continue to play a central role in computer science and are a key tool for any programmer.

Chapter 9. Section 9.

Graphs and Graph Algorithms

This section presents a comprehensive examination of graph data structures and foundational graph processing algorithms, spanning structural representations, systematic traversals, exhaustive route enumeration, and single-source shortest path optimization. The work explores

core conceptual definitions—covering directed, undirected, and weighted graphs along with topological vertex attributes (degrees, in-degrees, out-degrees, and self-loops)—and evaluates the operational trade-offs between adjacency matrices and adjacency lists in dynamic JavaScript environments. A central emphasis is placed on algorithmic execution and formal visualization using DRAKON diagrams. The section delivers detailed trace

9.1. 9.1. General Information. Basic Terminology

This section discusses data structures such as graphs. In mathematics, a graph is a structure consisting of vertices (nodes) and edges (connections) that connect these vertices.

In general, graphs share many similarities with trees, and one can consider trees as a special case of graphs. However, the practical value of graphs in solving real-world problems is much greater. Many tasks can be reduced to considering a set of objects whose properties are described by the relationships between them.

Such objects include:

- Electrical and electronic circuits
- Printed circuit boards
- Road maps
- Aviation routes
- Descriptions of structural constructions
- Game states and logic

Among the tasks that can be solved using graphs are:

- Finding the shortest path between two vertices
- Solving the problem of maximum capacity of pipelines, road networks, or computer

networks

- Distributing N workers to perform M different types of work
- Choosing the most efficient method of problem solving, among others.

Formally, a graph is denoted as $G = (V, E)$, where V is the set of vertices and E is the set of edges connecting them.

9.1.1. Directed and Undirected Graphs

A graph as a data structure is defined by a set of vertices and a set of connections between them. Depending on whether these connections have direction, graphs are divided into two main types:

undirected graphs and **directed graphs**.

Undirected Graphs

An **undirected graph** is a graph in which edges have no direction. An edge between vertices **u** and **v** represents only the fact that these two vertices are connected. In other words, the connection **u**→**v** is equivalent to the connection **v**→**u**.

Formally, an undirected graph can be written as:

$$G = (V, E)$$

where **V** is the set of vertices and **E** is the set of edges. Each edge connects two vertices.

The **degree** of a vertex v , denoted as $\deg(v)$, is the number of edges incident to v , that is, the number of edges connected to this vertex.

If loops are allowed, then a loop in an undirected graph contributes 2 to the degree of the vertex.

This is because a loop has two endpoints, and both endpoints are incident to the same vertex.

For example, if a vertex v has one ordinary edge connected to another vertex and one loop, then:

$$\deg(v) = 1 + 2 = 3$$

Directed Graphs

A **directed graph**, or **digraph**, is a graph in which every edge has a direction. Directed edges are often called **arcs**.

An arc from vertex u to vertex v is written as:

$$(u, v)$$

and represents a connection directed from u to v .

Unlike in an undirected graph, the arcs (u, v) and (v, u) are different connections.

For directed graphs, the degree of a vertex is divided into two separate characteristics:

- The **in-degree** of a vertex v , denoted as $\deg^-(v)$, is the number of arcs entering v .
- The **out-degree** of a vertex v , denoted as $\deg^+(v)$, is the number of arcs leaving v .

The **total degree** of a vertex in a directed graph is the sum of its in-degree and out-degree:

$$\deg(v) = \deg^-(v) + \deg^+(v)$$

If a directed graph contains a loop (v, v) , then this loop both enters and leaves the same vertex v . Therefore, it increases the in-degree by 1 and the out-degree by 1:

$$\deg^-(v) += 1$$

$$\deg^+(v) += 1$$

As a result, a loop increases the total degree of the vertex by 2.

In short, an undirected edge represents a symmetric connection between two vertices, while a directed edge represents an ordered connection. For this reason, directed graphs distinguish between incoming and outgoing arcs.

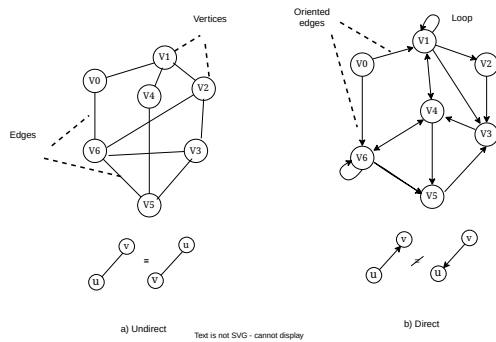


Figure 9.1. Graphical representation of (a) an undirected graph; (b) a directed graph (digraph)

9.1.2. Adjacency, Incidence, and Degree

A vertex and an edge are incident if the vertex is one of the endpoints of the edge.

Refined Terms for Directed Graphs (Digraphs):

- **In-degree ($\deg^-(v)$):** The number of edges directed into the vertex. In flow-based models, this represents the "input" capacity.
- **Out-degree ($\deg^+(v)$):** The number of edges directed away from the vertex. In state machines, this represents the number of possible state transitions.
- **Total Degree:** The sum of in-degree and out-degree.
- **Self-Loop (Digraph):** A special case where an edge starts and ends at the same vertex. It contributes exactly 1 to the in-degree and exactly 1 to the out-degree.

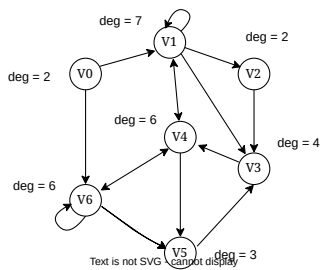


Figure 9.2. Basic graph parameters and node dimensions

9.2. 9.2. Graph Representation Methods

Solving problems related to processing a data set organized in the form of graphs requires modeling them in computer programs. In practice, two structures are typically used to

model the structure of a graph: the adjacency matrix and the adjacency list.

9.2.1. Adjacency Matrix

An adjacency matrix is a two-dimensional array that indicates the existence of a relationship between two vertices. If the graph has V vertices, then the adjacency matrix is a $V \times V$ array. In an undirected graph, the adjacency relationship is symmetric: if vertex u is adjacent to v , then v is also adjacent to u . This symmetry is reflected in the adjacency matrix, which is always symmetric about its main diagonal.

In contrast, for a directed graph (or digraph), the relationship is not necessarily symmetric. An edge (u, v) means vertex v is adjacent from

vertex u , and this is indicated in the matrix by a '1' at position $A[u][v]$ only.

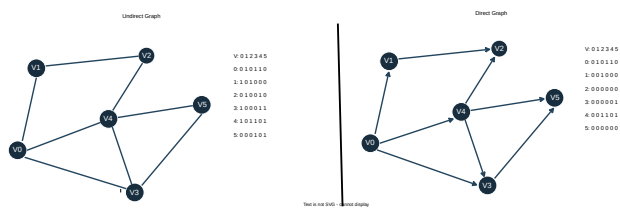


Figure 9.3. Adjacency matrix layout for undirected and directed graphs

9.2.2. Adjacency List

In some cases, a more memory-efficient way to represent a graph is to use an adjacency list. This method involves an array of linked lists (or nested arrays), where each individual list contains information about which vertices are

adjacent to a given one.

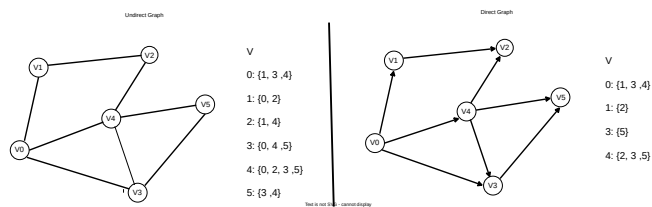


Figure 9.4. Adjacency list structural representation

9.2.3. Comparison and Complexity

The space complexity of representing a graph depends heavily on the chosen data structure:

- **Adjacency Matrix:** Requires $O(V^2)$ memory. This is memory-intensive, especially for large, sparse graphs (graphs with relatively few edges). It also often requires knowing the

number of vertices in advance.

- **Adjacency List:** Requires $O(V + E)$ memory, where E is the number of edges. This is much more efficient for sparse graphs, which are common in real-world applications.

In practice, the choice between these representations is a trade-off. An adjacency matrix is often preferable for dense graphs (graphs where the number of edges approaches V^2) and when frequent edge lookups are required ($O(1)$ complexity). An adjacency list is generally better for sparse graphs.

9.2.4. Weighted Graphs

Many real-life problems require modeling graphs

with numerical values associated with edges, such as distance, cost, or travel time. Such graphs are called weighted graphs. In an adjacency matrix, the weight is stored as the value of the matrix element (instead of '1'), and a special value (like ∞ or 0) can represent the absence of an edge. In an adjacency list, each element in the list is typically a structure containing both the adjacent vertex ID and the edge weight.

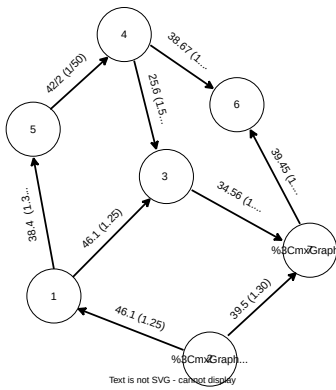


Figure 9.5. Adjacency list equipped with a 'weight' field

9.2.5. Graph Representation in JavaScript

Since JavaScript is a dynamic language, we rely on the flexibility of Objects, Arrays, and built-in

collections.

Adjacency Matrix in JavaScript

```
// Initialize a 3x3 matrix for an
undirected graph
const numVertices = 3;
const adjMatrix =
Array(numVertices).fill().map(() =>
Array(numVertices).fill(0));

function addEdgeMatrix(u, v) {
    adjMatrix[u][v] = 1;
    adjMatrix[v][u] = 1; // For
undirected graph
}

addEdgeMatrix(0, 1);
addEdgeMatrix(1, 2);
```

```
// Check adjacency (0(1))  
console.log(adjMatrix[0][1]); // 1
```

Adjacency List in JavaScript

```
const numVertices = 3;  
const adjList =  
  Array(numVertices).fill().map(() => []);  
  
function addEdgeList(u, v) {  
  adjList[u].push(v);  
  adjList[v].push(u); // For undirected  
graph  
}  
  
addEdgeList(0, 1);  
addEdgeList(1, 2);
```

```
// Check adjacency (O(degree(u)))
console.log(adjList[0].includes(1)); //
true
```

Weighted Graph in JavaScript

```
const weightedAdjList =
Array(numVertices).fill().map(() => []);

function addWeightedEdge(u, v, weight) {
    weightedAdjList[u].push({ node: v,
weight });
    weightedAdjList[v].push({ node: u,
weight }); // For undirected weighted
graph
}
```



```
addWeightedEdge(0, 1, 5);  
addWeightedEdge(1, 2, 10);
```

9.3.9.3. Fundamentals of Graph Traversal

Once a graph structure is established, the next logical task is to systematically visit all its vertices. This process is called graph traversal. There are two primary techniques: Depth-First Search (DFS) and Breadth-First Search (BFS).

The fundamental difference between DFS and BFS lies in the order in which they explore the graph, which stems from the underlying data structure they use to manage yet-to-be-visited

vertices:

1. **Depth-First Search (DFS)** uses a stack (often implemented via recursion or explicit arrays) to explore as deep as possible along each branch before backtracking. While DFS does not inherently find shortest paths, it is indispensable for tasks like detecting cycles, topological sorting, and exploring a graph when its overall structure is not known in advance.
2. **Breadth-First Search (BFS)** uses a queue to explore the graph level by level. It visits all direct neighbors of the starting vertex, then all of their neighbors, and so on. BFS is guaranteed to find the shortest path (in terms of the number of edges) in an unweighted

graph.

The step-by-step process for each algorithm is shown in the tables below for the example graph, starting from vertex V_0 .

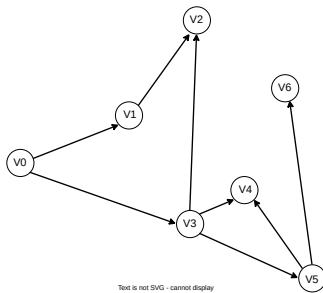


Figure 9.6. Reference directed graph for demonstrating DFS and BFS traversals

9.3.1. Depth-First Search (DFS) — Using Stack (LIFO)

Detailed DFS Trace Table from Vertex V_0

Step	Action	Current Vertex	Unvisited Neighbors	Stack Content (LIFO)	Visited Order
1	Initialization Push the start vertex onto the stack.	0	<i>[1, 3] (Both neighbors are unvisited. Due to the reverse i-- loop in your chart, they are pushed onto the stack as [3, 1])</i>	[3, 1]	[0]

Step	Action	Current Vertex	Unvisited Neighbors	Stack Content (LIFO)	Visited Order
2	Pop from stack Extract the last element of the stack.	1	[2] <i>([The only outgoing neighbor of vertex 1])</i>	[3, 2] <i>(Vertex v_2 is pushed on top of vertex 3)</i>	[0, 1]

Step	Action	Current Vertex	Unvisited Neighbors	Stack Content (LIFO)	Visited Order
3	Pop from stack Extract vertex V_2 .	2	[] ([Vertex V_2 is an absolute dead-end; it has no outgoing edges])	[3] (The stack shrinks; no new elements are added)	[0, 1, 2]

Step	Action	Current Vertex	Unvisited Neighbors	Stack Content (LIFO)	Visited Order
4	Pop from stack Extract vertex V_3 .	3	$[4, 5]$ <i>_ (Vertex V_3 has three neighbors: $[2, 4, 5]$. V_2 is already visited, only $[5, 4]$ are pushed in reverse order)]</i>	$[5, 4]$	$[0, 1, 2, 3]$

Step	Action	Current Vertex	Unvisited Neighbors	Stack Content (LIFO)	Visited Order
5	Pop from stack Extract vertex V_4 .	4	[] ([Vertex V_4 has no outgoing edges])	[5]	[0, 1, 2, 3, 4]

Step	Action	Current Vertex	Unvisited Neighbors	Stack Content (LIFO)	Visited Order
6	Pop from stack Extract vertex V_5 .	5	[6] ([Vertex V_5 has two neighbors: [4, 6]. Since V_4 was already visited in step 5, only vertex V_6 is pushed])	[6]	[0, 1, 2, 3, 4, 5]

Step	Action	Current Vertex	Unvisited Neighbors	Stack Content (LIFO)	Visited Order
7	Pop from stack Extract the final vertex.	6	[] ([Vertex V_6 has no outgoing edges])	[] (The stack is completely empty)	[0, 1, 2, 3, 4, 5, 6]

Step	Action	Current Vertex	Unvisited Neighbors	Stack Content (LIFO)	Visited Order
8	Termination The condition <code>stack.length > 0</code> returns No . Exit loop.	—	—	[]	Final Result: <code>[0, 1, 2, 3, 4, 5, 6]</code>

NOTE

Key Principle of DFS: The stack operates on the LIFO (Last In,

First Out) principle. The algorithm dives deep into the structural branches before backtracking.

9.3.2. Breadth-First Search (BFS) — Using Queue (FIFO)

Detailed BFS Trace Table from Vertex V_0

Step	Action	Current Vertex	Unvisited Neighbors	Queue Content (FIFO)	Visited Order
1	Initialization Enqueue the start vertex.	0	<i>[1, 3]</i> <i>(Both neighbors are unvisited. Due to the direct i++ loop in your chart, they are enqueued as [1, 3])</i>	<i>[1, 3]</i>	<i>[0]</i>

Step	Action	Current Vertex	Unvisited Neighbors	Queue Content (FIFO)	Visited Order
2	Dequeue (Shift) Extract the first element from the queue head.	1	[2] <i>([The only outgoing neighbor of vertex 1])</i>	[3, 2] <i>(Vertex V_2 is appended to the tail of the queue)</i>	[0, 1]

Step	Action	Current Vertex	Unvisited Neighbors	Queue Content (FIFO)	Visited Order
3	Dequeue (Shift) Extract vertex V_3 from the queue head.	3	$[4, 5]$ (Vertex V_3 has neighbors $[2, 4, 5]$. Vertex V_2 is skipped as it is already in the queue. Only $[4, 5]$ are appended)	$[2, 4, 5]$	$[0, 1, 3]$

Step	Action	Current Vertex	Unvisited Neighbors	Queue Content (FIFO)	Visited Order
4	Dequeue (Shift) Extract vertex V_2 .	2	$[]$ <i>([Vertex V_2 has no outgoing edges])</i>	$[4, 5]$ <i>(The queue shrinks from the front)</i>	$[0, 1, 3, 2]$

Step	Action	Current Vertex	Unvisited Neighbors	Queue Content (FIFO)	Visited Order
5	Dequeue (Shift) Extract vertex V_4 .	4	$[]$ ([Vertex V_4 has no outgoing edges])	$[5]$	$[0, 1, 3, 2, 4]$

Step	Action	Current Vertex	Unvisited Neighbors	Queue Content (FIFO)	Visited Order
6	Dequeue (Shift) Extract vertex V_5 .	5	[6] (Vertex V_5 has neighbors [4, 6]. Vertex V_4 is skipped as it is already visited. Only vertex V_6 is appended)	[6]	[0, 1, 3, 2, 4, 5]

Step	Action	Current Vertex	Unvisited Neighbors	Queue Content (FIFO)	Visited Order
7	Dequeue (Shift) Extract the final vertex.	6	[] ([Vertex V_6 has no outgoing edges])	[] (The queue is completely empty)	[0, 1, 3, 2, 4, 5, 6]

Step	Action	Current Vertex	Unvisited Neighbors	Queue Content (FIFO)	Visited Order
8	Termination The condition <code>queue.length > 0</code> returns No . Exit loop.	—	—	[]	Final Result: [0, 1, 3, 2, 4, 5, 6]

NOTE **Key Principle of BFS:** The queue

operates on the FIFO (First In, First Out) principle. The algorithm expands outwards like a wave — evaluating immediate neighbors before expanding deeper.

9.3.3. Comparison: Stack vs. Queue

The comparison of traversal paths clearly illustrates the fundamental difference between the two traversal strategies. DFS, using a stack, delves deep into the graph's branches, while BFS, using a queue, explores level by level (Figure 9.7).

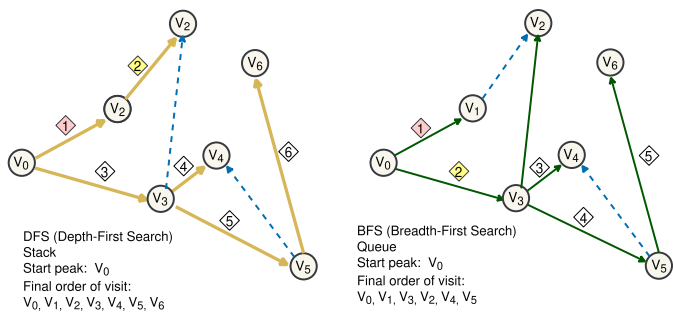


Figure 9.7. Comparison of DFS and BFS traversal orders from vertex 0

Feature	DFS (Stack)	BFS (Queue)
Data Structure	Stack (LIFO)	Queue (FIFO)

Feature	DFS (Stack)	BFS (Queue)
Traversal Order From <i>V_{start}</i>	0 → 1 → 2 → 3 → 4 → 5 → 6	0 → 1 → 3 → 2 → 4 → 5 → 6
Strategy	"Depth-first" — explores branches completely	"Breadth-first" — explores level by level
Target Applications	Cycle detection, Topological sort	Shortest path in unweighted networks

9.3.4. DRAKON-Diagrams for DFS and BFS

Before analyzing the practical considerations for selecting an appropriate traversal strategy, it is instructive to examine the algorithmic logic encoded in the implementation. The Drakon-diagrams below provide a visual representation of the control flow for both Depth-First Search (DFS) and Breadth-First Search (BFS) algorithms.

These diagrams decompose the traversal process into discrete decision points and operations, illustrating how the choice of underlying data structure — a stack for DFS versus a queue for BFS — fundamentally shapes the execution path.

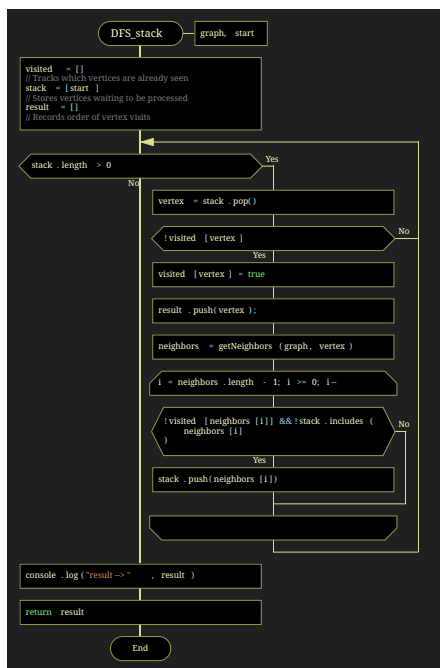


Figure 9.8a. Drakon-diagram illustrating the control flow of DFS (Depth-First Search)

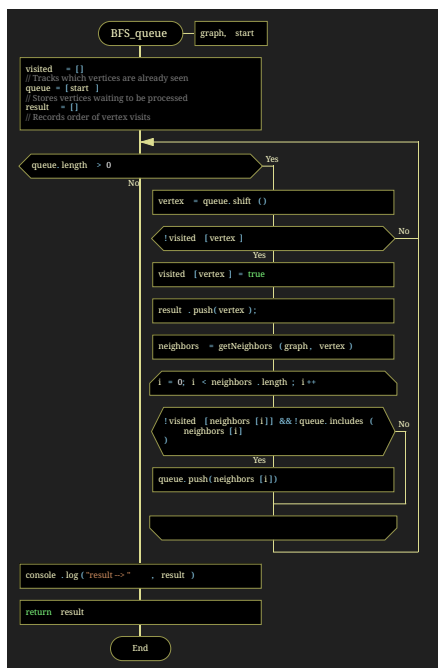


Figure 9.8b. Drakon-diagram illustrating the control flow of BFS (Breadth-First Search)

9.3.5. Choosing Between DFS and BFS

In practice, the choice between DFS and BFS depends on the specific problem requirements:

Task	Recommended Algorithm	Reason
Shortest path (unweighted)	BFS	Guarantees minimum number of traversed edges
Graph connectivity	DFS or BFS	Any systematic traversal satisfies this criteria

Task	Recommended Algorithm	Reason
Cycle detection	DFS	Highly efficient with tracking backtracking state
Topological sorting	DFS	Resolves dependencies naturally via post-order tracking

9.4. 9.4. Calculation of Paths and Travel Costs Between Two Vertices

This subsection examines the problem of finding all possible simple routes between two vertices in a weighted graph. For each route, the algorithm calculates two cumulative characteristics:

- total distance, measured in kilometers;
- total travel price, measured in arbitrary units.

The considered task differs from ordinary graph traversal. Classical BFS and DFS are usually used to determine the order of visiting vertices or to find one suitable path. In contrast, the algorithm

described here enumerates all possible simple paths from a source vertex to a target vertex and evaluates each of them according to its accumulated weights.

This makes it possible to solve the following practical tasks:

- find all possible routes between two selected vertices;
- calculate the total distance of each route;
- calculate the total travel price of each route;
- compare routes according to different optimization criteria;
- select the shortest, cheapest, or otherwise preferable route.

9.4.1. General Idea of the Algorithm

The algorithm is based on recursive backtracking. During the search, the program gradually constructs the current path from the source vertex to the target vertex. At each step, it selects one of the neighboring vertices, adds the corresponding edge characteristics to the accumulated sums, and continues the search recursively.

If the target vertex is reached, the current route is saved as a completed path. After that, the algorithm returns to the previous vertex, restores the former state, and tries another possible continuation.

Thus, the algorithm consists of two alternating

phases:

- forward movement to a neighboring vertex;
- backward restoration of the previous state.

This mechanism makes it possible to enumerate all simple paths without revisiting vertices already included in the current route.

The overall structure of the functions involved in the path search is shown in Figure 9.9.

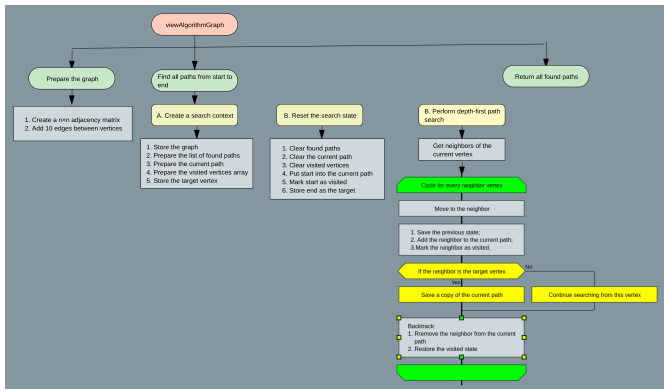


Figure 9.9. Mind map of path search functions

9.4.2. Functional Structure of the Path Search Module

The path search module is organized as a small group of cooperating functions. Each function performs a clearly separated role in the general

recursive process.

The main control function is `initPathSearch()`. It prepares the initial search context and starts recursion.

The recursive enumeration itself is performed by `findPathWithContext()`. This function checks the current vertex, processes its neighbors, and controls the transition from one vertex to another.

When the target vertex is reached, the function `handleFoundPath()` creates a structured description of the completed route and stores it in the resulting collection.

Two auxiliary functions support the backtracking

mechanism:

- `prepareMoveToNeighbor()` prepares the transition to a neighboring vertex and saves the current state;
- `backtrackFromNeighbor()` restores the previous state after the recursive call has finished.

This functional decomposition separates the logic of initialization, recursive search, result processing, forward movement, and backward restoration.

9.4.3. Description of Key Functions

`initPathSearch(graph, source, target)`

The function `initPathSearch()` is the entry point

of the path search module. It creates the search context, initializes all necessary data structures, and starts the recursive traversal of the graph.

Attribute	Description
Purpose	Initializes the search for all paths from the source vertex to the target vertex. Creates the search context, resets accumulated parameters, and starts recursive graph traversal.

Attribute	Description
Parameters	<code>graph</code> — graph object containing the adjacency matrix and edge weights. <code>source</code> — identifier of the source vertex. <code>target</code> — identifier of the target vertex.
Return Value	Array <code>allPaths</code> containing all discovered paths as <code>pathInfo</code> objects.

The initialization logic is shown in Figure 9.10.



Figure 9.10. DRAKON-diagram of the initialization function for path search

findPathWithContext(current, context)

The function `findPathWithContext()` is the core recursive function of the algorithm. It analyzes the current vertex and determines whether the target vertex has already been reached.

If the current vertex is the target, the completed path is processed by `handleFoundPath()`. Otherwise, the function checks all neighboring vertices and recursively continues the search through those neighbors that have not yet been visited in the current path.

Attribute	Description
Purpose	Performs recursive enumeration of all possible simple paths from the current vertex to the target vertex. Updates the current path, accumulated distance, and accumulated price.
Parameters	<code>current</code> — identifier of the current vertex. <code>context</code> — search state object containing <code>target</code>, <code>graph</code>, <code>currentPath</code>, <code>totalDistance</code>, <code>totalPrice</code>, <code>visited</code>, and <code>allPaths</code>.
Return Value	<code>void</code> . The results are accumulated in <code>context.allPaths</code> .

The recursive search logic is shown in Figure 9.11.

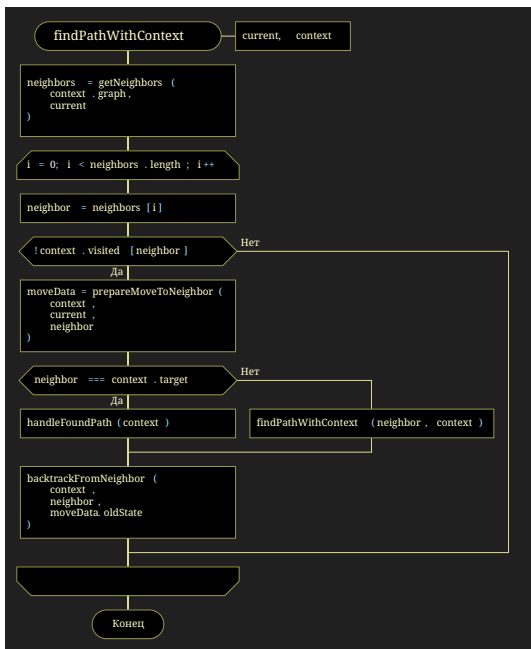


Figure 9.11. DRAKON diagram of the recursive path search function

handleFoundPath(context)

The function `handleFoundPath()` is called when the recursive search reaches the target vertex. At this moment, the current route is complete and can be transformed into a structured result.

The function creates a `pathInfo` object that contains the route itself, the list of vertices, total distance, and total price. This object is then added to the collection of found paths.

Attribute	Description
Purpose	Creates and stores a structured record of a completed path from the source vertex to the target vertex.

Attribute	Description
Parameters	<code>context</code> — search state object containing <code>currentPath</code> , <code>totalDistance</code> , <code>totalPrice</code> , and <code>allPaths</code> .
Return Value	<code>pathInfo</code> object with the fields <code>path</code> , <code>vertices</code> , <code>distance</code> , and <code>price</code> .

The path handling logic is shown in Figure 9.12.

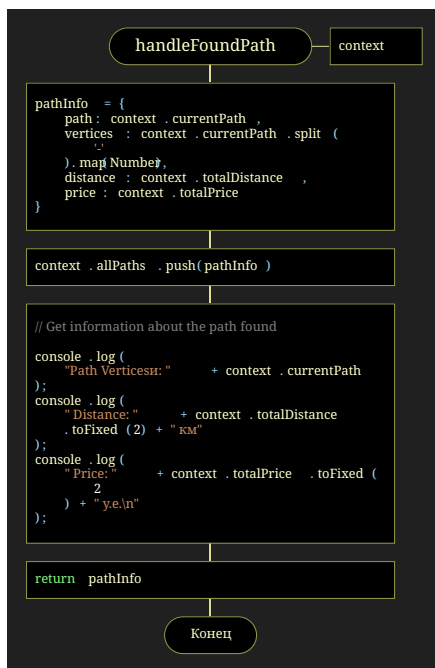


Figure 9.12. DRAKON diagram of the Path Handler function

prepareMoveToNeighbor(context, current, neighbor)

The function `prepareMoveToNeighbor()` prepares the forward movement from the current vertex to one of its neighbors. It reads the edge characteristics, updates the current path, increases the accumulated distance and price, and marks the neighbor as visited.

Before changing the context, the function saves the previous state. This saved state is required later for correct backtracking.

Attribute	Description
Purpose	Prepares transition to a neighboring vertex and saves the previous state of the search context.

Attribute	Description
Parameters	<code>context</code> — search state object. <code>current</code> — identifier of the current vertex. <code>neighbor</code> — identifier of the neighboring vertex.
Return Value	<code>oldState</code> object containing the previous values of <code>currentPath</code>, <code>totalDistance</code>, and <code>totalPrice</code>.

The transition preparation logic is shown in Figure 9.13.



Figure 9.13. DRAKON diagram of the Transition Preparation function

Reversing function

backtrackFromNeighbor(context, neighbor, oldState)

The function **backtrackFromNeighbor()** performs the reverse phase of the algorithm. After the

recursive processing of a neighboring vertex is completed, the algorithm must return to the previous state in order to try another possible route.

This function clears the visited flag for the processed neighbor and restores the path, distance, and price values saved in `oldState`.

Attribute	Description
Purpose	Restores the previous search state after processing a neighboring vertex. This enables exhaustive enumeration of alternative routes.

Attribute	Description
Parameters	<code>context</code> — search state object. <code>neighbor</code> — vertex whose processing has been completed. <code>oldState</code> — object containing the previous values of <code>currentPath</code>, <code>totalDistance</code>, and <code>totalPrice</code>.
Return Value	<code>void</code> . The search context is restored to the previous state.

The backtracking logic is shown in Figure 9.14.

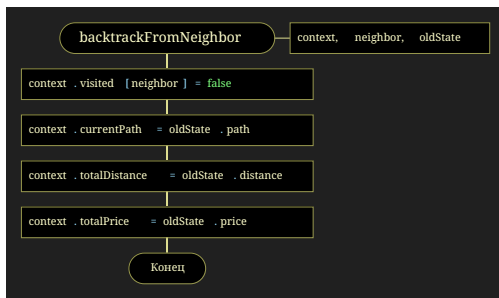


Figure 9.14. DRAKON diagram of the Backtracking function

9.4.4. Algorithmic Complexity Analysis

The implemented algorithm enumerates all simple paths between two vertices. Therefore, its running time may grow exponentially with the number of vertices.

In the worst case, especially for a dense graph, the number of possible simple paths can be very large. For this reason, the time complexity can be estimated as factorial or exponential, depending on the graph structure. A conservative upper estimate is:

$$O(V!)$$

where V is the number of vertices.

The space complexity is mainly determined by the recursion depth, the current path, and the `visited` array. Since a simple path cannot contain more than V vertices, the auxiliary space complexity is:

$O(V)$

This approach is suitable for small and medium-sized educational examples. However, for large dense graphs, exhaustive path enumeration becomes impractical. In such cases, specialized algorithms should be used depending on the optimization criterion, for example shortest path algorithms.

9.4.5. Program Execution Example

Using the example graph shown in Figure 9.15, the algorithm can find all simple paths from vertex 0 to vertex 5. Each path is represented as a sequence of vertices, along with the total distance and price calculated based on the edge weights.

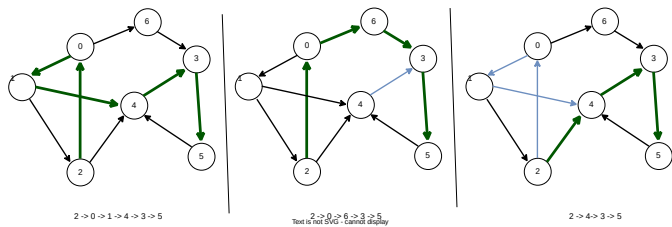


Figure 9.15. Graphical representation of the path search process from vertex 0 to vertex 5

The following fragment shows an example of output for paths from vertex 0 to vertex 5.

```
// === Find all paths from 0 to 5 ===
//
// Path Vertices: 0-1-4-3-5
// Distance: 101.28 km
// Price: 31.80 u.
//
```

```
// Path Vertices: 0-6-3-5  
// Distance: 94.21 km  
// Price: 21.60 u.
```

The first route has a greater distance and price. The second route is shorter and cheaper in this example. Therefore, the same set of found paths can be analyzed according to several criteria: minimum distance, minimum price, or a combined route quality indicator.

9.4.6. Summary

The developed module demonstrates how recursive backtracking can be applied to weighted graphs. Unlike simple traversal algorithms, this approach does not stop after

finding one path. Instead, it systematically enumerates all possible simple paths from the source vertex to the target vertex and calculates the quantitative characteristics of each route.

This makes the module useful as an educational example for understanding:

- recursive graph traversal;
- path construction;
- state saving and restoration;
- backtracking;
- analysis of weighted routes;
- comparison of alternative paths.

The same idea can be used as a basis for studying

transportation logistics, route planning, and optimization problems.

9.5. Dijkstra's Algorithm

Dijkstra's algorithm is designed to find the shortest paths from a given start vertex to all other vertices in a weighted graph, provided that all edge weights are non-negative. This condition is essential: Dijkstra's algorithm works correctly only when every edge weight is greater than or equal to zero.

The algorithm is named after the renowned Dutch computer scientist Edsger W. Dijkstra, who first conceived it in 1956. Today, Dijkstra's

algorithm is widely used in science, engineering, and everyday applications:

- **GPS navigation systems:** finding the fastest routes while taking into account distances and traffic conditions.
- **Telecommunications networks:** determining optimal paths for data routing, for example in OSPF and IS-IS protocols.
- **Robotics and games:** real-time path planning and object navigation through maps.

9.5.1. Graph Representation for Dijkstra's Algorithm

For our demonstration of Dijkstra's algorithm, we will use the graph shown in Figure 9.16. This

graph consists of six vertices, labeled from 0 to 5, connected by weighted edges.

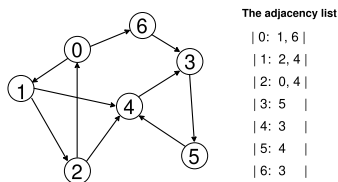


Figure 9.16. Target topology setup for Dijkstra shortest path calculation

In JavaScript, this weighted graph can be represented as an object whose keys are vertices and whose values describe adjacent vertices and edge weights:

```
const graph = {  
  0: { 1: 48.3, 2: 40.7, 5: 25.4 },
```

```
1: { 0: 48.3, 2: 37.5, 3: 29.8 },  
2: { 0: 40.7, 1: 37.5, 3: 34.9, 5:  
29.8 },  
3: { 1: 29.8, 2: 34.9, 4: 25.4 },  
4: { 3: 25.4, 5: 34.9 },  
5: { 0: 25.4, 2: 29.8, 4: 34.9 }  
};
```

This representation corresponds to an undirected weighted graph. For example, the edge between vertices **0** and **1** has weight **48.3**, and therefore it appears in both directions:

```
0: { 1: 48.3, ... }  
1: { 0: 48.3, ... }
```

9.5.2. How the Algorithm Works

Dijkstra's algorithm is based on a simple but powerful principle: it iteratively builds the set of vertices for which the shortest distance from the start vertex is already known, and uses these vertices to improve the distance estimates for the remaining vertices.

The mind map of Dijkstra's algorithm is presented in Figure 9.17.

[\[Fig9 17a DijkstraAlgo\]](#) |

[section_9/fig9_17a_DijkstraAlgo.svg](#)

Figure 9.17. Mind map for Dijkstra's algorithm

Let us examine the algorithm step by step. We start from vertex **0** and find the shortest paths from this vertex to all other vertices of the graph.

To track its progress, the algorithm maintains three essential data structures:

1. **Distance array (distances or d):** for each vertex v , $\text{distances}[v]$ stores the current length of the shortest known path from the start vertex s to v . Initially, $\text{distances}[s] = 0$, and $\text{distances}[v] = \text{Infinity}$ for all other

vertices.

2. **Visited array (`visited`):** a marker structure where `visited[v] = true` means that the shortest distance to vertex `v` has already been finalized. If `visited[v] = false`, the vertex is still unprocessed.
3. **Previous array (`previous`):** for each vertex `v`,

`previous[v]` stores the predecessor of `v` in the currently known shortest path. This structure is used later to reconstruct the actual shortest paths.

The algorithm can be understood through a city-exploration analogy. Imagine a network of cities connected by roads of different lengths. You start

at your home city and repeatedly choose the closest city whose shortest route has not yet been finalized. From that city, you check whether its roads provide shorter routes to neighboring cities.

In the classic implementation shown here, the algorithm uses a linear search to find the

unvisited vertex with the smallest current distance. This makes the implementation simple and clear for educational purposes.

The functions used in this implementation are summarized in the following table.

Functions used in the classic implementation of

Function	Arguments	Purpose	Content	Illustration
dijkstra	graph, startVertex	Main function of the algorithm.	Initializes auxiliary data structures, repeatedly selects the unvisited vertex with the	Fig9_17b_dD_dijkstra.svg

Function	Arguments	Purpose	Content	Illustration
getVertexList	graph	Builds the list of graph vertices.	Extracts all vertex names or indices from the graph representation and returns them as a list.	Fig9_18_getVertexList.svg

Function	Arguments	Purpose	Content	Illustration
initializeDistance	vertices, startVertex	Initializes the distance array.	Sets the distance from the start vertex to itself as 0. Sets the distance to all other vertices	Fig9_19_initData.svg

Function	Arguments	Purpose	Content	Illustration
initializeVisited	vertices	Initializes the visited marker array.	Marks all vertices as unvisited. For each vertex <i>v</i> , sets <code>visited[v] = false</code> .	Fig9_19_initData.svg

Function	Arguments	Purpose	Content	Illustration
initializePrevious	vertices	Initializes the predecessor array.	Sets the predecessor of each vertex to <code>null</code> . This structure is later used to reconstruct	Fig9_19_initData.svg

Function	Arguments	Purpose	Content	Illustration
findMinDistanceVertex	distances, visited	Finds the next vertex to process.	Searches among all unvisited vertices and returns the vertex with the smallest	Fig9_20_findMinDistanceVertex.svg

Function	Arguments	Purpose	Content	Illustration
markVertexVisited	visited, vertex	Marks the selected vertex as processed.	Sets <code>visited[vertex] = true</code> , meaning that the shortest distance to this vertex has been finalized.	Fig9_21_markVertexVisited.svg

Function	Arguments	Purpose	Content	Illustration
relaxEdges	graph, currentVertex, distances, previous,	Updates distance estimates for neighboring vertices.	For each outgoing edge from currentVertex to a neighboring vertex, checks	Fig9_2_2_relaxEdges.svg

9.5.3. DRAKON Diagrams of the Algorithm Functions

The implementation of Dijkstra's algorithm is divided into several small functions. Each of them performs one clearly defined operation. This decomposition makes the algorithm easier to understand and easier to verify.

The first helper function builds the list of graph vertices. This list is required to initialize the auxiliary data structures and to iterate over vertices during the search for the next vertex to process.

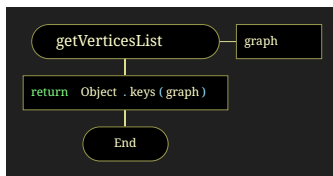


Figure 9.18. Building the list of graph vertices

The next group of functions initializes the main

data structures used by the algorithm: the distance array, the visited array, and the predecessor array.

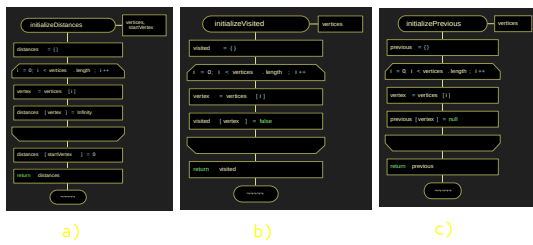


Figure 9.19. Initialization of distance, visited, and predecessor arrays

After initialization, the algorithm must repeatedly choose the unvisited vertex with the smallest currently known distance. This operation is performed by the `findMinDistanceVertex` function.

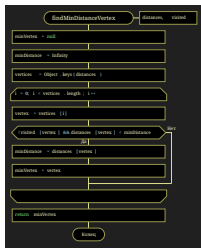


Figure 9.20. Finding the unvisited vertex with the minimum distance

If such a vertex is found, it becomes the current vertex. The algorithm then marks it as processed. This means that the shortest distance to this vertex is now considered final.

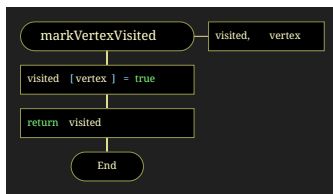


Figure 9.21. Marking the selected vertex as processed

The most important operation in the algorithm is

edge relaxation. During this step, the algorithm checks whether a shorter path to each neighboring vertex can be obtained through the current vertex.



Figure 9.22. Updating distance estimates for neighboring vertices

For an edge from vertex u to vertex v with weight $w(u, v)$, the relaxation condition is:

$$\text{distances}[u] + w(u, v) < \text{distances}[v]$$

If this condition is true, the algorithm updates the current distance estimate:

```
distances[v] = distances[u] + w(u, v)
```

and records the predecessor of vertex **v**:

```
previous[v] = u
```

Thus, relaxation performs two tasks at once:

- improves the shortest known distance to a vertex;
- stores information needed to reconstruct the shortest path.

9.5.4. General Control Flow

The overall logic of the algorithm can be described as follows:

1. Build the list of graph vertices.
2. Initialize the `distances`, `visited`, and `previous` structures.

3. Set the distance to the start vertex to 0.
4. Find the unvisited vertex with the smallest current distance.
5. Mark this vertex as visited.
6. Relax all outgoing edges from this vertex.
7. Repeat the process until there is no reachable

unvisited vertex left.

8. Return the final `distances` and `previous` structures.

A JavaScript implementation of the main function can be written as follows:

```
function dijkstra(graph, startVertex) {  
  const vertices =  
    getVerticesList(graph);  
  
  const distances =  
    initializeDistances(vertices,  
      startVertex);
```

```
    const visited =  
initializeVisited(vertices);  
    const previous =  
initializePrevious(vertices);  
  
    let currentVertex =  
findMinDistanceVertex(distances,  
visited);
```

```
while (currentVertex != null) {  
    markVertexVisited(visited,  
currentVertex);  
    relaxEdges(graph, currentVertex,  
distances, previous, visited);  
  
    currentVertex =
```

```
findMinDistanceVertex(distances,  
visited);  
}  
  
return {  
    distances: distances,  
    previous: previous  
};
```

```
}
```

The returned object contains two results:

- **distances** — the lengths of the shortest paths from the start vertex to all reachable vertices;
- **previous** — predecessor information that

allows the actual shortest paths to be reconstructed.

9.5.5. Result for the Example Graph

Let us run the algorithm from vertex 0 . Initially, the distance to vertex 0 is equal to 0 , while the distances to all other vertices are set to *Infinity*.

```
distances[0] = 0  
distances[1] = Infinity  
distances[2] = Infinity  
distances[3] = Infinity  
distances[4] = Infinity  
distances[5] = Infinity
```


After the algorithm finishes, the shortest distances from vertex 0 are:

Shortest distances from vertex 0

Vertex	Shortest distance from vertex 0	Previous vertex
0	0	null
1	48.3	0
2	40.7	0

Vertex	Shortest distance from vertex 0	Previous vertex
3	75.6	2
4	60.3	5
5	25.4	0

For example, the shortest path from vertex 0 to vertex 4 is reconstructed using the previous structure:

```
4 <- 5 <- 0
```

Reading this sequence in reverse order gives the

path:

0 -> 5 -> 4

The total length of this path is:

$$25.4 + 34.9 = 60.3$$

Similarly, the shortest path from vertex 0 to vertex 3 is:

$0 \rightarrow 2 \rightarrow 3$

with total length:

$$40.7 + 34.9 = 75.6$$

9.5.6. Complexity

In this classic implementation, the function `findMinDistanceVertex` performs a linear scan over all vertices. Since it can be called up to V times, where V is the number of vertices, the vertex-selection part has complexity:

$$O(V^2)$$

The relaxation of edges contributes:

$$O(E)$$

where E is the number of edges.

Therefore, the total time complexity of this classic implementation is:

$$O(V^2 + E)$$

For dense graphs, this is often written simply as:

$$O(V^2)$$

More advanced implementations use a priority queue or a min-heap to select the next vertex more efficiently, but the classic version shown

here is easier to understand and is well suited for explaining the main idea of Dijkstra's algorithm.

Chapter 10. Section

10. Data Structures

Selection in Practice

10.1. 10.1. Data Structure Selection Roadmap

Choosing an optimal data structure is a *multi-criteria decision problem* that requires consideration of *functional requirements*, *data characteristics*, the *operation profile*, and *physical*

hardware constraints. Functional requirements answer the question “What operations must be supported?”, the operation profile answers “How often and under what conditions will these operations be performed?”, and hardware constraints answer “What resources are available, and how do they affect performance?”

Functional requirements specify which *operations on data* a structure must support and which *properties* it must preserve. Examples include *lookup by key, insertion and deletion of elements, access by index, maintaining sorted order, and retrieving values within a specified range*. These requirements establish the set of suitable data

structures.

The *operation profile* describes how a structure will be used: *which operations* are performed most frequently, the *volume of data* they involve, and their *sequence*. For example, one application may primarily perform lookups, with infrequent updates; another may continuously receive new

elements and then process them sequentially. Even when the required operations are identical, differences in their *execution frequency* may lead to the selection of different structures.

The **hardware constraints** of the system impose additional limitations on the choice of data structures. These constraints include the

available RAM capacity, cache size and line size, memory access latency, memory bandwidth, pointer size and alignment requirements, parallel processing capabilities, and storage device characteristics. Each of these factors can significantly impact the performance of different data structures (Table 1).

Table 1. Hardware Characteristics and Constraints

Factor	Impact on data structure selection
<i>Available RAM capacity</i>	In addition to the data itself, a structure stores auxiliary information: pointers, indices, and reserved slots. This overhead may limit the maximum size of the structure.

Factor	Impact on data structure selection
<i>Cache capacity and cache line size</i>	Storing elements close together allows several adjacent values to be loaded in a single memory access and subsequently used without accessing RAM again.

Factor	Impact on data structure selection
<i>Memory access latency</i>	Particularly significant when following a chain of pointers: the address of the next element becomes available only after the previous element has been read.

Factor	Impact on data structure selection
<i>Memory bandwidth</i>	Limits the speed of bulk scanning, copying, and moving data, for example when inserting elements into an array.

Factor	Impact on data structure selection
<i>Pointer size and alignment requirements</i>	Affect the actual memory footprint of elements: pointers and alignment padding can substantially increase memory usage.

Factor	Impact on data structure selection
<i>Parallel processing capabilities</i>	When multiple cores are used, data partitioning and the frequency of access to shared memory become important. Contention over the same cache lines can reduce performance.

Factor	Impact on data structure selection
<i>Storage device characteristics</i>	When data does not fit in RAM, the number of storage accesses and the size of transferred blocks become important. This is one reason for using B-trees and B+ trees.

Before making a choice, you must define the mandatory operations (key-based lookup, indexed access, insertion, deletion, range queries) and the required data organization properties (key ordering, insertion-order preservation, element uniqueness, or allowing duplicates). Data structures that fail to meet these requirements

are ruled out from further consideration.

Data structure selection algorithm according to the characteristics of a specific task can be presented in five steps (Figure 10.1.)

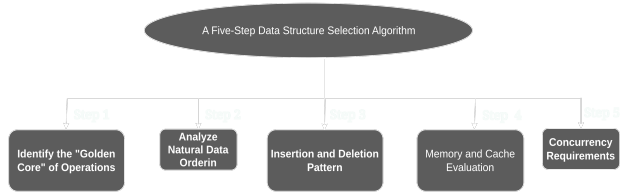


Figure 10.1. Multi-Criteria Memory-Oriented decision-making algorithm for data structure selection

In more detail Multi-Criteria Memory-Oriented decision-making algorithm for data structure selection is presented in the following tables (Tables 2.1–2.5).

*Table 2.1. Step 1: Identifying the Operation Profile
("Golden Core")*

Criterion / Condition	Candidate Data Structures	Engineering Rationale and Considerations
Mostly reading and searching	Arrays for indexed access or sequential traversal; hash tables for lookup by key; sorted arrays	Distinguish direct access, lookup by key, and sequential traversal. These operations have different costs and may favor different

Criterion / Condition	Candidate Data Structures	Engineering Rationale and Considerations
Mostly writing and insertion	Dynamic arrays for appending; linked lists for insertion at known locations; hash	The insertion location and subsequent access requirements determine which candidates are appropriate. For

Criterion / Condition	Candidate Data Structures	Engineering Rationale and Considerations
Mixed read/write workload	Dynamic arrays, hash tables, or balanced search trees, depending on the required	Compare the frequency and cost of each operation. Reserving capacity, where supported, can reduce reallocations or

NOTE

Operation frequencies alone do not determine the best structure: data volume, insertion locations, and the cost of each operation also matter. Ratios such as 80/20 or 50/50 illustrate workloads; they are not universal selection

thresholds. Stacks and queues specify access disciplines and can be implemented using different underlying structures.

Table 2.2. Step 2: Assessing the Need for Data Ordering

Criterion / Condition	Candidate Data Structures	Engineering Rationale and Considerations
Key order is required (range queries, ordered traversal,	Balanced search trees (AVL or red-black trees), B-trees or B+ trees, skip lists, and sorted	These structures support operations based on key order. The update frequency and storage environment help

Criterion / Condition	Candidate Data Structures	Engineering Rationale and Considerations
Key order is not required (individual lookups by key)	Hash table.	Expected $O(1)$ lookup is possible under suitable hashing and load-factor assumptions, assuming bounded-cost key hashing

NOTE

Key order and insertion order are different requirements: preserving insertion order does not make keys sorted. If only the minimum or maximum must be retrieved repeatedly, a heap may suffice; maintaining full key order

may be unnecessary.

Table 2.3. Step 3: Identifying the Insertion and Deletion Pattern

Criterion / Condition	Candidate Data Structures	Engineering Rationale and Considerations
Insertion and deletion at the end only	Dynamic array; a stack implemented using a dynamic array or linked nodes.	Dynamic-array append is amortized $O(1)$; removing the last element does not require shifting the remaining

Criterion / Condition	Candidate Data Structures	Engineering Rationale and Considerations
Insertion and deletion at the beginning only	Linked list with a head reference; deque with efficient operations at the front.	These implementations avoid the $O(n)$ element shifting required by a conventional contiguous array

Criterion / Condition	Candidate Data Structures	Engineering Rationale and Considerations
Insertion at one end and removal at the other, or changes at	Queue or deque implemented using a circular buffer or an appropriately linked	A circular buffer reuses vacant positions without shifting all elements. A linked implementation requires suitable

Criterion / Condition	Candidate Data Structures	Engineering Rationale and Considerations
Insertion or deletion at an internal position	Dynamic array for infrequent changes; linked list when the required node references are already	Array changes may shift $O(n)$ elements. A list can change links in $O(1)$, but locating the position may require an $O(n)$

Criterion / Condition	Candidate Data Structures	Engineering Rationale and Considerations
Insertion, update, or deletion by key	Hash table when key order is unnecessary; balanced search tree when ordered	Hash tables typically provide expected $O(1)$ key-based operations under suitable assumptions; balanced search

NOTE

The total cost includes locating the target and modifying the structure. Constant-time link changes require access to all necessary nodes; for example, deleting a node from a singly linked list generally requires its

predecessor. Changing an element by key and inserting at a numerical position are different tasks.

Table 2.4. Step 4: Assessing Memory and Cache Constraints (Hardware Filter)

Criterion / Condition	Candidate Data Structures	Engineering Rationale and Considerations
Small working set with frequent sequential access	Contiguous arrays or other compact representations.	Good spatial locality can reduce memory-access overhead. Element shifting may be acceptable for small arrays, but its

Criterion / Condition	Candidate Data Structures	Engineering Rationale and Considerations
Large data volume in RAM with frequent sequential processing	Arrays or chunked arrays.	Compact or block-based storage supports efficient scanning and reduces per-element pointer overhead. A large

Criterion / Condition	Candidate Data Structures	Engineering Rationale and Considerations
Large data volume in RAM with frequent localized changes	Balanced search trees for changes by key; linked lists when the necessary node references are	These candidates can avoid shifting a large contiguous suffix, but pointer overhead, allocation costs, and cache misses

Criterion / Condition	Candidate Data Structures	Engineering Rationale and Considerations
Data exceeds available RAM or is primarily stored on a storage	B-trees or B+ trees for indexed access and updates; block-oriented sequential layouts for	Match the layout to the access pattern. Block-oriented trees reduce the number of storage accesses for indexed

Criterion / Condition	Candidate Data Structures	Engineering Rationale and Considerations
Read-only data requiring lookup or traversal in key order	Sorted array with binary search.	A compact representation supports $O(\log n)$ binary search and efficient sequential traversal. Binary search has a

NOTE

Performance depends on the active working set, element layout, access pattern, and implementation; no universal cache-size or timing threshold determines the winner. In JavaScript, an ordinary Array

does not guarantee the same physical representation as a packed numeric buffer; TypedArray provides a clearer model for fixed-width numeric data. Validate candidate implementations using

representative data and workloads.

Table 2.5. Step 5: Determining Whether Concurrent Access Is Required

Criterion / Condition	Candidate Data Structures / Access Strategies	Engineering Rationale and Considerations
Several threads or workers process independently owned	Separate structures per thread or worker; partitioned data with explicit	Independent ownership can reduce contention. Exchanging data or combining results may still require coordination.

Criterion / Condition	Candidate Data Structures / Access Strategies	Engineering Rationale and Considerations
Several threads or workers access shared mutable data	Structures protected by locks; partitioned structures with appropriate coordination;	Choose according to the required consistency guarantees, contention level, and runtime support. Lock-free

Criterion / Condition	Candidate Data Structures / Access Strategies	Engineering Rationale and Considerations
Single-threaded execution without concurrent shared-memory	Simple implementations appropriate to the required operations.	No inter-thread synchronization is needed, reducing coordination overhead and implementation complexity.

NOTE

First identify which data is shared, which operations modify it, and which guarantees must hold during concurrent access. Lock-free does not mean synchronization-free or necessarily faster; such

algorithms require suitable atomic operations and careful implementation. In JavaScript, asynchronous execution alone does not imply parallel shared-memory access.

Conclusion

The five-step algorithm presented here makes it possible to identify data structures that meet the requirements of the task at hand. Functional requirements define the set of viable solutions, while the operation profile, the nature of data mutations, and the constraints of the computing

environment help compare the remaining candidates. The result is a well-grounded choice of one or more candidates, which must then be validated on representative data and workloads by measuring operation execution time and memory consumption.

Applying the hardware filter introduced in the

fourth step requires understanding how the elements of a structure are placed in computer memory and how the processor accesses them. Sequentially reading adjacent array elements versus following references between list nodes can incur substantially different time costs, even when processing the same number of elements.

The next subsection therefore examines the memory hierarchy and its impact on data structure performance. This will help explain why the theoretical advantages of a structure do not always translate into faster performance in a real computing environment.

10.2. 10.2. Memory Hierarchy and Its Impact on Data Structure Performance

Any practical choice of a data structure in a real

computing system must account not only for the abstract algorithmic complexity (Big-O) but also for the **physical organization of memory**. Ignoring this factor is one of the primary reasons why a theoretically optimal structure often performs slower in practice than a "simpler" alternative.

In modern computers, memory is organized as a **hierarchical pyramid** (Figure 10.2), where each level differs in capacity, access speed, and cost.

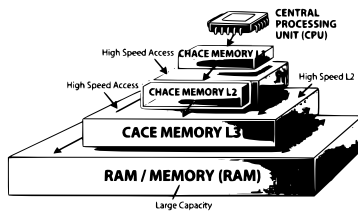


Figure 10.2. Memory hierarchy of a modern computer

10.2.1. 10.2.1. Characteristics of Memory Levels

Each memory level has unique parameters that determine how efficiently a particular data structure interacts with it. The figures below are **order-of-magnitude estimates for a typical modern system**, not measurements of any

specific processor — actual numbers vary with microarchitecture, process node, and generation of hardware.

Memory level comparison

Memo ry Level	Typica l Size	Access Time	Bandw idth	Manag ed By	Relati ve Energ y
CPU Regist ers	~1 KB	< 1 ns	~100 GB/s	Compil er	1×

Memo ry Level	Typica l Size	Access Time	Bandw idth	Manag ed By	Relati ve Energ y
L1 Cache	32–64 KB	~1 ns	~50 GB/s	Hardw are	2×

Memo ry Level	Typica l Size	Access Time	Bandw idth	Manag ed By	Relati ve Energ y
L2 Cache	256–51 2 KB	~3–5 ns	~30 GB/s	Hardw are	4×

Memo ry Level	Typica l Size	Access Time	Bandw idth	Manag ed By	Relati ve Energ y
L3 Cache	4–32 MB	~10–15 ns	~20 GB/s	Hardw are	8×

Memo ry Level	Typica l Size	Access Time	Bandw idth	Manag ed By	Relati ve Energ y
RAM	8–256 GB	~50–10 0 ns	~10–20 GB/s	OS/Pro gram	20×

Memo ry Level	Typica l Size	Access Time	Bandw idth	Manag ed By	Relati ve Ener gy
SSD/Di sk (swap)	256 GB – 4 TB	~50–20 0 μ s (SSD) / ~10 ms (HDD)	~0.5–5 GB/s	OS	1000×

IMPORTANT

Important: The difference between register access time (< 1 ns) and RAM access time (~ 100 ns) is **two orders of magnitude**. In the time the CPU waits for data from RAM, it

could have executed **hundreds** of operations. This is precisely why data structures that are "cache-friendly" outperform even those with better theoretical complexity.

10.2.2. 10.2.2. Key Concepts: Cache Line and Cache Miss

The CPU does not work with individual bytes but with fixed-size blocks called **cache lines** (typically 64 bytes). When a program accesses a variable, the entire cache line containing that variable is loaded into the cache.

- **Cache hit:** Data is already in the cache. Access takes ~1 ns.
- **Cache miss:** Data is not in the cache and must be loaded from RAM. Latency is ~100 ns (100× slower!).

For data structures, this means:

- **Arrays** (and any sequentially stored structures): When iterating over an array, the CPU loads one cache line (64 bytes = 8 elements of 8 bytes) and processes all 8 elements **without additional misses**. This achieves near-theoretical maximum speed.
- **Linked lists and trees:** Nodes are scattered

randomly across memory. Moving to the next node almost always causes a **cache miss**, because the adjacent node does not reside in the same cache line. The latency accumulates, and performance degrades by a factor of 5–20.

10.2.3. Prefetching Mechanism

Modern processors feature **hardware prefetchers** that analyze memory access patterns and proactively load data into the cache.

- For **arrays**, the prefetcher works perfectly: it detects linear traversal and begins loading

cache lines **ahead**, before the program even requests them. This effectively hides RAM latency.

- For **linked lists and trees**, the prefetcher is **useless**: pointer-based transitions are unpredictable, and the CPU cannot anticipate where the program will jump next.

IMPORTANT

Conclusion: If your data structure supports **sequential iteration** and you use it frequently in this mode, an array will be an order of magnitude faster than a tree or a list,

even if the tree has $O(\log n)$ complexity and the array has $O(n)$.

10.2.4. 10.2.4. Practical Rule: The "Cache Filter"

When selecting a data structure, **always** ask yourself three questions:

1. **Will the structure be iterated (traversed entirely) frequently?**

- YES → Prefer arrays and vectors (even if insertion is $O(n)$).
- NO → Trees and lists become viable options.

2. What is the access pattern?

- Sequential (in order) → Arrays win due to

prefetching.

- Random (by index or key) → Hash tables and trees provide the advantage.

3. How large is the data relative to the L3 cache size?

- Data fits in L3 (millions of elements) →

The cache effect is maximized; structures with poor locality (lists) perform catastrophically.

- Data does not fit in L3 (tens of millions) → The difference between arrays and trees becomes less critical because RAM is slow regardless.

10.2.5. 10.2.5. Illustrative Example: Iterating 1 Million Elements

Consider three data structures, each containing 1 million integers. We perform a full iteration and measure time (estimate for a modern Intel Core i7):

Structure	Memory Layout	Cache Misses per Element	Relative Time	Estimated Real Time
Array	Sequential	~0.125 (1 miss per 8 elements)	1×	~1 ms

Structure	Memory Layout	Cache Misses per Element	Relative Time	Estimated Real Time
Linked List	Random (fragmented)	~1 (almost always a miss)	~15×	~15 ms

Structure	Memory Layout	Cache Misses per Element	Relative Time	Estimated Real Time
Tree (depth-first traversal)	Random (worse than list)	~1–1.5 (double miss)	~20×	~20 ms

IMPORTANT

Important: This timing is only for iteration. If the structure is used for search (not iteration), the picture changes. A hash table provides $O(1)$ search but will iterate $5\times$ slower

than a tree. Therefore, **the choice always depends on the dominant operation.**

10.2.6. 10.2.6. Cache-Friendly Structures (Priority During Selection)

Based on memory hierarchy analysis, we can construct a **cache-locality rating** for major data structures:

Cache-locality rating

Friendliness Level	Data Structures	Reason
Excellent (★★★★)	Static Array, Dynamic Array (Vector), Sorted Array	Sequential storage, perfect prefetching

Friendliness Level	Data Structures	Reason
Good (★★★★)	Hash Table (open addressing), Deque (array-based)	Elements are scattered but within a single allocation

Friendliness Level	Data Structures	Reason
Fair (□□□□□)	Linked List (with node pool), Red-Black Tree (with custom allocator)	Nodes are scattered, but a pool improves locality

Friendliness Level	Data Structures	Reason
Poor (□□□□)	Naive Linked List, AVL/RBT with standard new/malloc	Each node is a separate allocation, maximal fragmentation

10.2.7. 10.2.7. Bridge to the DRAKON Decision Diagram

Understanding the memory hierarchy enables the introduction of a **new critical filter** into the data structure selection algorithm (see the DRAKON diagram in Section 10.5):

- If **cache locality** is a priority (high-load systems, game engines, financial applications) → **favor arrays** and sequentially stored structures.
- If **cache locality** is not critical (background tasks, batch processing, infrequent access) → trees and lists can be chosen without concern.

This filter is inserted **after** the questions about operation frequency and **before** the questions about memory constraints, as shown in Figure 10.2 (see Section 10.5).

Based on the material in this subsection, the following procedure can be proposed for assessing the resources available for a specific

task.

*Procedure for Assessing Available Resources for a
Specific Task*

What to Assess	What Needs to Be Determined	How It Affects the Decision
Data volume and expected growth	Number of elements, size of a single element, possible maximum volume	Helps assess whether the data will fit in available memory

What to Assess	What Needs to Be Determined	How It Affects the Decision
Total memory consumption	Beyond the useful data itself: references, indices, reserved capacity,	Compact source data does not necessarily mean low memory consumption

What to Assess	What Needs to Be Determined	How It Affects the Decision
Available RAM	How much memory can be allocated to the program, accounting for the OS, runtime	A margin should be left for data growth and peak consumption during

What to Assess	What Needs to Be Determined	How It Affects the Decision
Working set and locality	Which portion of the data is actively used; whether accesses are sequential or reference-	Helps assess the benefit of compact placement and the likelihood of data reuse from cache

What to Assess	What Needs to Be Determined	How It Affects the Decision
Nature of memory and storage exchange	Whether bulk reads/copies predominate, or numerous scattered accesses	In the first case, throughput matters most; in the second, the latency of individual

What to Assess	What Needs to Be Determined	How It Affects the Decision
Parallel processing capability	Whether data and work can be split across executors; which data will remain shared	Indicates whether additional cores can be leveraged without excessive

What to Assess	What Needs to Be Determined	How It Affects the Decision
Actual program metrics	Time for characteristic operations, peak memory consumption, behavior under	Allows preliminary estimates to be verified and implementations compared in the target